Подготовка к РК1

Программирование I · язык C++

Темы 1–7: от алгоритмов и переменных до массивов, функций, строк и основ ООП

Все программы написаны для Borland C++ (Builder) 6.0, консольное приложение.
Каждый пример скомпилирован и запущен, вывод в PDF — настоящий.

Лекция → примеры и нюансы → задачи с решениями → тест → ключи

Содержание

Лекция 1. Парадигмы, алгоритмы, синтаксис C++, типы данных, указатели . . 5

1.1 Программирование и парадигмы . . . 5
1.2 Классификация языков программирования . . . 5
1.3 Алгоритм . . . 5
1.4 Алфавит и лексемы языка C++ . . . 6
1.5 Структура программы в Borland C++ 6.0 . . . 8
1.6 Линейная структура (следование) . . . 10
1.7 Типы данных . . . 10
1.8 Перечисляемый тип (enum) . . . 12
1.9 «Множественность» данных: массивы . . . 13
1.10 Указатели и адреса памяти . . . 13
1.11 Задачи . . . 14
1.12 Тест по теме . . . 17
1.13 Контрольные вопросы из программы (проверь себя устно) . . . 19

Лекция 2. Операции и выражения . . . 20

2.1 Основные понятия . . . 20
2.2 Арифметические операции . . . 20
2.3 Операции сравнения (равенства) . . . 22
2.4 Операции присваивания . . . 23
2.5 Логические операции . . . 23
2.6 Побитовые операции . . . 24
2.7 Операция «запятая» . . . 24
2.8 Условная операция ?: . . . 24
2.9 Операция sizeof . . . 25
2.10 Операция разрешения области видимости :: . . . 25
2.11 Операции доступа к элементам: . и -> . . . 25
2.12 Приоритет и ассоциативность . . . 26
2.13 Преобразование типов в выражениях . . . 26
2.14 Задачи . . . 26
2.15 Тест по теме . . . 29
2.16 Контрольные вопросы из программы (проверь себя устно) . . . 32

Лекция 3. Ветвление: if, switch, goto. Цикл for . . . 33

3.1 Управляющие структуры . . . 33
3.2 Условный оператор if . . . 33
3.3 Оператор выбора switch . . . 34
3.4 Оператор безусловного перехода goto . . . 35
3.5 Понятие цикла . . . 36
3.6 Цикл со счётчиком for . . . 36

3.7 Задачи . . . 37
3.8 Тест по теме . . . 44
3.9 Контрольные вопросы из программы (проверь себя устно) . . . 47

Лекция 4А. Циклы while и do...while, break и continue . . . 48

4А.1 Цикл с предусловием while . . . 48
4А.2 Цикл с постусловием do...while . . . 48
4А.3 Операторы прерывания цикла . . . 49
4А.4 Типичные задачи на while . . . 50
4А.5 Задачи . . . 50
4А.6 Тест по теме . . . 54

Лекция 4Б. Одномерные массивы . . . 57

4Б.1 Что такое массив . . . 57
4Б.2 Объявление . . . 57
4Б.3 Инициализация . . . 57
4Б.4 Шаблон ввода и вывода . . . 58
4Б.5 Типовые алгоритмы . . . 58
4Б.6 Сортировка . . . 60
4Б.7 Задачи (лабораторная по одномерным массивам) . . . 60
4Б.8 Тест по теме . . . 69

Лекция 4В. Двумерные массивы (матрицы) . . . 72

4В.1 Что это . . . 72
4В.2 Шаблон: ввод, вывод и вложенные циклы . . . 72
4В.3 Типовые алгоритмы . . . 73
4В.4 Задачи (лабораторная по многомерным массивам) . . . 74
4В.5 Тест по теме . . . 83
4В.6 Контрольные вопросы из программы (проверь себя устно) . . . 85

Лекция 5. Функции . . . 86

5.1 Зачем нужны функции . . . 86
5.2 Определение функции и вызов . . . 86
5.3 Объявление (прототип) и определение . . . 87
5.4 Параметры и аргументы . . . 88
5.5 Передача параметров по значению . . . 88
5.6 Передача параметров по ссылке . . . 89
5.7 Спецификатор const у параметров . . . 89
5.8 Параметры со значениями по умолчанию . . . 90
5.9 Встроенные (inline) функции . . . 90
5.10 Локальные и глобальные переменные. Область видимости . . . 90
5.11 Массивы как параметры функций . . . 91
5.12 Перегрузка функций . . . 92

5.13 Рекурсия . . . 92
5.14 Задачи . . . 93
5.15 Тест по теме . . . 99
5.16 Контрольные вопросы из программы (проверь себя устно) . . . 102

Лекция 6. Тип char и строки . . . 104

6.1 Тип char . . . 104
6.2 Строки как массивы символов . . . 104
6.3 Ввод и вывод строк . . . 105
6.4 Доступ к символам и перебор строки . . . 106
6.5 Библиотека . . . 106
6.6 Класс string . . . 107
6.7 Типичные ошибки . . . 108
6.8 Задачи . . . 108
6.9 Тест по теме . . . 112
6.10 Контрольные вопросы из программы (проверь себя устно) . . . 115

Лекция 7. Основные понятия и принципы ООП . . . 116

7.1 Что такое ООП . . . 116
7.2 Базовые понятия . . . 116
7.3 Главные принципы ООП . . . 117
7.4 Состояние и поведение. Взаимодействие объектов . . . 121
7.5 Моделирование реальных сущностей . . . 122
7.6 Преимущества и применение . . . 122
7.7 Задачи . . . 123
7.8 Тест по теме . . . 128
7.9 Контрольные вопросы из программы (проверь себя устно) . . . 131

Приложения. Шпаргалка, типичные ошибки и итоговый тест по РК1 . . . 132

П.1 Шаблон консольной программы для Borland C++ 6.0 . . . 132
П.2 Типичные ошибки новичка . . . 132
П.3 Шпаргалка . . . 133
П.4 Примеры типа «изменить побочную диагональ / отсортировать» . . . 134
П.5 Итоговый тест по темам 1–7 . . . 135
П.6 Задачи для тренировки (реши сам, затем проверь на компьютере) . . . 141
П.7 План повторения перед РК1 . . . 142

Лекция 1. Парадигмы, алгоритмы, синтаксис C++, типы данных, указатели

Эта лекция — фундамент. Всё дальнейшее (циклы, массивы, функции) строится на том, что здесь: как устроена программа, какие бывают типы данных и что на самом деле происходит с переменной в памяти.

1.1 Программирование и парадигмы

Программирование — процесс создания программ: описание алгоритма на языке, понятном компьютеру. **Парадигма программирования** — это стиль, набор идей и правил, по которым мы строим программу.

Парадигма	Суть	Пример языков
Процедурная (императивная)	Программа = последовательность команд и вызовов процедур/функций. Данные и функции отдельно	C, Pascal, Basic
Объектно-ориентированная (ООП)	Программа = набор взаимодействующих объектов. Данные и методы внутри объекта	C++, Java, C#, Python
Функциональная	Программа = вычисление функций, без изменения состояния	Haskell, Lisp
Логическая	Программа = факты и правила, ответ ищет сам интерпретатор	Prolog
Декларативная	Описываем ЧТО нужно получить, а не КАК	SQL, HTML

> **Запомни.** C++ — **мультипарадигменный** язык: на нём можно писать и процедурно (как мы делаем первые 6 тем), и объектно-ориентированно (тема 7 и дальше).

1.2 Классификация языков программирования

- **Машинный язык** — коды из нулей и единиц, понятные процессору. Зависит от процессора.
- **Ассемблер** — мнемоники вместо кодов (`MOV AX, 5`). Это уже язык, но по-прежнему привязан к железе. Оба — языки **низкого уровня**.
- **Языки высокого уровня** — близки к человеческому: C++, Pascal, Java, Python. Не зависят от процессора, один оператор = много машинных команд.

По способу исполнения:

Тип	Как работает	Примеры
Компилируемые	Компилятор целиком переводит исходник в .exe, потом запускаем. Быстро	C, C++, Pascal
Интерпретируемые	Интерпретатор переводит и выполняет строку за строкой. Медленнее, но удобно	Python, JavaScript

Путь программы на C++ в Borland: **исходный текст (.cpp)** → препроцессор (обрабатывает `#include`, `#define`) → компилятор (получает объектный файл `.obj`) → компоновщик (подключает библиотеки, получает `.exe`).

> **Запомни.** Ошибки бывают трёх видов: **синтаксические** (опечатка, нет `;` — компилятор не соберёт), **семантические/ошибки времени выполнения** (деление на ноль — программа падает), **логические** (программа работает, но считает не то — самые коварные).

1.3 Алгоритм

Алгоритм — точная конечная последовательность действий, которая приводит от исходных данных к результату.

Свойства алгоритма (их часто спрашивают):

Свойство	Что означает
Дискретность	Алгоритм разбит на отдельные шаги
Определённость (детерминированность)	Каждый шаг понятен и однозначен
Конечность	Алгоритм заканчивается за конечное число шагов
Массовость	Подходит для целого класса задач, а не для одного набора данных
Результативность	Всегда приводит к результату
Понятность	Исполнитель знает, как выполнять каждую команду

Способы записи алгоритма

1. **Словесный** — обычными предложениями: «взять два числа, сложить, вывести».
2. **Псевдокод** — полуформальная запись: `если a > b то max := a иначе max := b`.
3. **Блок-схема** — графическая запись фигурами и стрелками.
4. **Программа** — запись на языке программирования.

Фигуры блок-схем

Фигура	Значение
Овал	Начало / конец алгоритма
Параллелограмм	Ввод / вывод данных
Прямоугольник	Действие (вычисление, присваивание)
Ромб	Условие (ветвление): выходы «да» / «нет»
Шестиугольник	Заголовок цикла со счётчиком (for)
Стрелки	Порядок выполнения

Три основных типа алгоритмов (структуры)

Линейный (следование) — действия выполняются одно за другим. **Ветвящийся** — выбор пути по условию. **Циклический** — повтор действий, пока выполняется условие.

Схема



Какой тип алгоритма какой оператор C++ реализует: линейный — подряд идущие операторы; ветвящийся — `if`, `switch`; циклический — `for`, `while`, `do...while`.

1.4 Алфавит и лексемы языка C++

Алфавит C++ состоит из: прописных и строчных **латинских** букв (A–Z, a–z), цифр 0–9, знака подчёркивания _, специальных символов (`+ - * / % = < > ( ) { } [ ] ; , . : " ' # & | ! ~ ^ ?`) и пробельных символов.

> **Нюанс.** Русские буквы **не входят** в алфавит C++. Их можно использовать только внутри строк `"Привет"` и комментариев. Назвать переменную `сумма` нельзя — только `summa` или `sum`.

Лексема — минимальная значимая единица языка (как слово в предложении). Виды лексем: идентификаторы, ключевые слова, константы, операции, разделители.

Идентификаторы

Идентификатор — имя, которое программист даёт переменной, константе, функции, типу. Правила:

- состоит из латинских букв, цифр и _;
- **не может начинаться с цифры**;
- не может совпадать с ключевым словом;
- **регистр важен**: `Sum`, `sum`, `SUM` — три разных имени;
- пробелов внутри быть не может (используй `total_sum` или `totalSum`).

Допустимо	Недопустимо	Почему
`x`, `sum1`, `_temp`, `Age`, `total_sum`	`1sum`	начинается с цифры
`maxValue`, `a_1`	`total sum`	пробел
`int_x`	`int`	ключевое слово
`price2`	`my-var`	дефис — это знак минус

Ключевые слова

Это зарезервированные слова, которые имеют особый смысл для компилятора, их нельзя использовать как имена: `int`, `float`, `double`, `char`, `bool`, `void`, `if`, `else`, `for`, `while`, `do`, `switch`, `case`, `default`, `break`, `continue`, `return`, `const`, `enum`, `class`, `struct`, `new`, `delete`, `true`, `false`, `sizeof`, `goto` и др.

> **Запомни.** `main`, `cout`, `cin`, `getch` — **не ключевые слова**, это имена функций и объектов из библиотек. Но называть так свои переменные не стоит.

Константы

Константа — значение, которое не меняется. Виды литералов:

Вид	Примеры	Заметки
Целые десятичные	`25`, `-7`, `0`	
Целые восьмеричные	`031` (=25)	начинаются с 0
Целые шестнадцатеричные	`0x1F` (=31)	начинаются с 0x
Вещественные	`3.14`, `0.5`, `2.5e-3`, `1e5`	`e` — степень десяти
Символьные	`'A'`, `'7'`, `'\n'`	в одинарных кавычках, один символ
Строковые	`"Hello"`, `"a"`	в двойных кавычках, в конце скрытый `\0`
Логические	`true`, `false`	тип `bool`

Суффиксы: `10L` — long, `10U` — unsigned, `3.14f` — float (без суффикса вещественное число — `double`).

Именованные константы создаются двумя способами:

Код

```
#define PI 3.14159          // директива препроцессора: подстановка текста, без ; и типа
const double E = 2.71828;   // константа с типом; менять нельзя
const int N = 10;           // именно такие константы можно использовать как размер массива
```

> **Нюанс.** Попытка изменить константу — ошибка компиляции: `const int a = 5; a++;` не соберётся (в тестах: «код выдаст ошибку»). Константу обязательно инициализируют при объявлении.

Escape-последовательности

Управляющие символы внутри строк и символьных констант записываются через обратную косую черту:

Запись	Значение
`\n`	перевод строки
`\t`	горизонтальная табуляция
`\\`	сам символ `\`
`\'` и `\"`	одинарная и двойная кавычка
`\0`	нулевой символ (конец строки)
`\a`	звуковой сигнал
`\b`	возврат на шаг (backspace)
`\r`	возврат каретки в начало строки

Код

```
#include <iostream.h>
int main()
{
    cout << "Стол\tСтул\n";
    cout << "Путь: C:\\temp\n";
    cout << "Он сказал: \"Привет\"" << endl;
    return 0;
}
```

Результат выполнения

```
Стол□Стул
Путь: C:\temp
Он сказал: "Привет"
```

Комментарии

Комментарий компилятор игнорирует. Два вида: однострочный `// до конца строки` и многострочный `/* ... */`.

Код

```
int a = 5;      // это комментарий до конца строки
/* а это комментарий
   на несколько строк */
```

> **Нюанс.** Многострочные комментарии **нельзя вкладывать** друг в друга: `/* ... /* ... */ ... */` — первый же `*/` закроет комментарий, остаток вызовет ошибку.

1.5 Структура программы в Borland C++ 6.0

Как создать консольную программу в C++ Builder 6: **File → New → Other → Console Wizard**, выбрать язык C++ и отметить **Console Application**, не ставя галочку VCL. Запуск — клавиша **F9**. Мастер создаёт заготовку с `#pragma hdrstop`, `#include <condefs.h>`, `#pragma argsused` и `int main(int argc, char* argv[])`. Всё это можно оставить, а можно упростить до минимума.

Минимальная программа:

Код

```
#include <iostream.h>   // подключаем ввод-вывод: cin, cout
#include <conio.h>      // getch() — ждать нажатия клавиши

int main()              // с функции main() начинается выполнение ЛЮБОЙ программы
{                       // начало тела функции
    cout << "Hello, C++!" << endl;   // вывод
    getch();            // пауза, чтобы окно не закрылось
    return 0;           // программа завершилась успешно
}                       // конец тела функции
```

Результат выполнения

```
Hello, C++!
```

Общая схема программы на C++:

1. **Директивы препроцессора** (`#include`, `#define`) — строки с `#`, точка с запятой в конце не ставится.
2. **Глобальные объявления** (константы, переменные, прототипы и определения функций).
3. **Функция** `main()` — точка входа. Тело заключено в **фигурные скобки** `{ }`.
4. Внутри — **операторы**, каждый заканчивается `;`.

Что нужно знать про структуру:

- **Библиотечные функции** лежат в библиотечных файлах; их описания (прототипы) — в заголовочных файлах `.h`, которые подключаются через `#include`.
- `main` может быть `void main()` (так любят в учебных тестах: `return;` в конце означает `void`) или `int main()` с `return 0;`. В Borland 6 работают оба варианта.
- Переменные можно объявлять **в любом месте до первого использования** (не обязательно в начале, как в Pascal).
- Регистр важен: `Main` или `COUT` — ошибка.

Заголовок	Что даёт
`<iostream.h>`	`cin`, `cout`, `endl`
`<conio.h>`	`getch()`, `clrscr()` (очистка экрана) — это Borland-специфика
`<math.h>`	`sqrt`, `pow`, `sin`, `fabs`, `ceil`, `floor`, `log` и т.д.
`<stdlib.h>`	`rand()`, `random()`, `randomize()`, `abs`, `atoi`
`<stdio.h>`	`printf`, `scanf`, `gets`, работа с файлами
`<string.h>`	`strlen`, `strcpy`, `strcat`, `strcmp`
`<iomanip.h>`	`setw`, `setprecision` — форматирование вывода
`<ctype.h>`	`isdigit`, `isalpha`, `toupper`, `tolower`

Нюанс. В Borland C++ 6.0 допустимы обе формы подключения: старая `#include <iostream.h>` (что даёт `cout` сразу) и новая `#include <iostream>` + `using namespace std;`. Если взять `<iostream>` и забыть `using namespace std;`, то `cout` придётся писать как `std::cout`.

Нюанс. Русский текст в консоли. Консоль Windows по умолчанию использует кодировку, из-за которой русские буквы в `cout` превращаются в «кракозябры». Решение: в начале `main` написать `setlocale(LC_ALL, "Russian");` (нужен `#include <locale.h>`). Если не помогло — добавить `system("chcp 1251");` (`<stdlib.h>`) или в самом начале вызвать `SetConsoleOutputCP(1251);` (`<windows.h>`). Узнай на лабе, какой способ работает у вашего преподавателя, и придерживайся его.

Вывод и ввод

Код

```
cout << "Текст" << a << endl;   // вывод: << «кладёт» значения в поток; endl — новая строка
cin >> a >> b;                  // ввод: значения разделяются пробелом или Enter
```

- `endl` и `"\n"` оба переводят строку (`endl` ещё сбрасывает буфер).
- Строка в кавычках в `cout` нужна для пояснения пользователю; без неё программа тоже работает (в тестах: «для наглядности, можно убрать»).
- `cin >> x` для `char` читает один символ, пропуская пробелы.

В стиле C используются `printf` и `scanf` из `<stdio.h>`:

Формат	Тип
`%d` или `%i`	int (десятичное целое)

Формат	Тип
%u	unsigned int
%o	восьмеричное целое
%x	шестнадцатеричное целое
%f	вещественное (float, и в printf — double)
%e	вещественное в экспоненциальной форме
%c	символ
%s	строка

Код

```
#include <stdio.h>
int main()
{
    int a = 255;
    double x = 3.14159;
    printf("a = %d, hex = %x, oct = %o\n", a, a, a);
    printf("x = %f, %.2f, %e\n", x, x, x);
    printf("символ %c, строка %s\n", 'Z', "abc");
    return 0;
}
```

Результат выполнения

```
a = 255, hex = ff, oct = 377
x = 3.141590, 3.14, 3.141590e+00
символ Z, строка abc
```

1.6 Линейная структура (следование)

В **линейной программе** операторы выполняются строго по порядку: ввод → вычисление → вывод. Это основа лабораторной №1.

Код

```
#include <iostream.h>
#include <conio.h>

int main()
{
    int a, b, sum;
    cout << "Введите a: -> ";
    cin >> a;
    cout << "Введите b: -> ";
    cin >> b;
    sum = a + b;                  // вычисление
    cout << "Сумма = " << sum << endl;
    getch();
    return 0;
}
```

Пример работы программы

```
Введите a: -> 3
Введите b: -> 4
Сумма = 7
```

> **Запомни.** Алгоритм любой линейной задачи: **1)** что вводим; **2)** что вычисляем и по какой формуле; **3)** что выводим. Прежде чем писать код, запиши входные данные, формулу и тип результата.

1.7 Типы данных

Тип определяет, **сколько памяти** занимает значение и **какие операции** с ним можно делать. В Borland C++ 6.0 (32-битная Windows):

Тип	Размер, байт	Диапазон	Что хранит
`char`	1	-128 ... 127	символ или маленькое целое
`unsigned char`	1	0 ... 255	
`short`	2	-32768 ... 32767	короткое целое
`unsigned short`	2	0 ... 65535	
`int`	4	-2147483648 ... 2147483647	целое
`unsigned int`	4	0 ... 4294967295	
`long`	4	как у `int`	длинное целое
`float`	4	примерно ±3.4·10^38, ~7 знаков	вещественное одинарной точности
`double`	8	примерно ±1.7·10^308, ~15 знаков	вещественное двойной точности
`long double`	10	±1.1·10^4932	повышенная точность
`bool`	1	`true` / `false` (внутри 1 и 0)	логическое значение

Нюанс. Типа `real` в C++ **нет** (он есть в Pascal). Вещественные — `float` и `double`. Логический тип называется `bool` (его значения: `true`, `false`; в выражениях `true` = 1, `false` = 0).

Размер любого типа или переменной узнаёт оператор `sizeof`:

Фрагмент

```
cout << sizeof(char) << " " << sizeof(short) << " " << sizeof(int) << " ";
cout << sizeof(float) << " " << sizeof(double) << " " << sizeof(bool) << endl;
```

Результат выполнения

```
1 2 4 4 8 1
```

Объявление и инициализация переменных

Переменная — именованная область памяти, в которой хранятся данные определённого типа; её значение может меняться при работе программы. Объявление: `тип имя;` или сразу с начальным значением `тип имя = значение;`.

Код

```
int a;                  // объявлена, значение НЕ определено (мусор!)
int b = 5;              // объявлена и инициализирована
int x, y = 3, z;        // можно несколько через запятую
float f = 2.5f;
char c = 'A';
bool ok = true;
```

Нюанс. Локальная переменная без инициализации содержит **случайный мусор** из памяти. Пользоваться ею до присвоения нельзя — типичный источник «странных» результатов. Глобальные переменные, наоборот, автоматически обнуляются.

Переполнение

Если результат не помещается в диапазон типа, происходит **переполнение**: число «заворачивается» по кругу.

Фрагмент

```
short s = 32767;
s = s + 1;                  // вышли за верхнюю границу
unsigned char c = 255;
c = c + 1;                  // 255 + 1 -> 0
cout << s << " " << (int)c << endl;
```

Результат выполнения

```
-32768 0
```

Преобразование типов

- **Автоматическое** (в выражениях): тип «повышается» до более широкого. `short + int` → `int`; `int + double` → `double`; `float + double` → `double`; `long + float` → `float`.
- **Явное**: `(int)3.99` даст 3 (дробная часть **отбрасывается**, не округляется); `(double)a / b`.
- При присваивании справа налево значение приводится к типу левой части: `int k = 3.99;` → `k = 3`.

Фрагмент

```
int a = 7, b = 2;
cout << a / b << " " << a % b << " " << (double)a / b << " " << a / 2.0 << endl;
int k = 3.99;
cout << k << " " << (int)(-3.99) << endl;
```

Результат выполнения

```
3 1 3.5 3.5
3 -3
```

Нюанс. Деление двух целых — **целочисленное**: `5/2 = 2`, `7/2 = 3`, `7/3 = 2`. Чтобы получить дробный результат, хотя бы одно число должно быть вещественным: `5/2.0 = 2.5`. Остаток от деления `%` работает только с целыми: `5%2 = 1`, `7%2 = 1`.

1.8 Перечисляемый тип (enum)

enum — тип, значения которого — именованные целые константы. Нумерация по умолчанию идёт с 0, но можно задать числа вручную.

Код

```
#include <iostream.h>

enum Color { RED, GREEN, BLUE };               // RED=0, GREEN=1, BLUE=2
enum Day { MON = 1, TUE, WED, THU, FRI };      // MON=1, TUE=2, ... FRI=5

int main()
{
    Color c = GREEN;
    Day d = FRI;
    cout << c << " " << d << endl;             // выведутся номера: 1 5
    if (c == GREEN)
        cout << "Зелёный" << endl;
    switch (d)
    {
        case MON: cout << "Начало недели" << endl; break;
        case FRI: cout << "Пятница!" << endl; break;
    }
    int n = BLUE + 10;                         // enum можно использовать как int: 12
    cout << n << endl;
    return 0;
}
```

Результат выполнения

```
1 5
Зелёный
Пятница!
12
```

Нюанс. Значение `enum` нельзя ввести через `cin >> c;` напрямую: вводят число в `int`, а потом преобразуют `c = (Color)n;`. Присвоить `enum`-переменной «просто число» без приведения — ошибка в C++.

1.9 «Множественность» данных: массивы

В программе курса «множественность» — это работа со множеством однотипных данных. В C++ типа `set` (как в Pascal) нет; для набора однотипных значений используется **массив**. Подробно массивы — в лекции 4, а пока запомни форму:

Код

```
int a[5] = {4, 8, 15, 16, 23};   // 5 целых; индексы от 0 до 4
cout << a[0] << a[4];            // 4 и 23
```

Массив — последовательность значений одного типа под одним именем. Номер элемента (индекс) всегда начинается с **0**. Все элементы массива связаны тем, что у них одинаковый **тип**.

1.10 Указатели и адреса памяти

Память компьютера — это ячейки с номерами (**адресами**). У каждой переменной есть адрес. **Указатель** — переменная, которая хранит адрес другой переменной.

Запись	Смысл
`&a`	адрес переменной `a` (операция взятия адреса)
`int *p;`	`p` — указатель на `int` (звёздочка при объявлении)
`p = &a;`	записываем в `p` адрес `a`
`*p`	значение по адресу в `p` (разыменование)
`*p = 10;`	записать 10 в ту переменную, на которую указывает `p`

Код

```
#include <iostream.h>
int main()
{
    int a = 5;
    int *p = &a;          // p указывает на a
    cout << a << " " << *p << endl;   // 5 5
    *p = 9;               // меняем a через указатель
    cout << a << endl;    // 9
    int b = 1;
    p = &b;               // теперь p указывает на b
    *p = *p + 100;
    cout << a << " " << b << endl;    // 9 101
    return 0;
}
```

Результат выполнения

```
5 5
9
9 101
```

Указатель и массив

Имя массива — это адрес его первого элемента. Прибавление числа к указателю сдвигает его на столько **элементов**, а не байтов.

Код

```
#include <iostream.h>
int main()
{
    int a[4] = {10, 20, 30, 40};
    int *p = a;                       // то же, что &a[0]
    cout << *p << " " << *(p + 1) << " " << *(p + 3) << endl;   // 10 20 40
    p++;                              // сдвиг на один int (на 4 байта)
    cout << *p << " " << p[1] << " " << a[2] << endl;           // 20 30 30
    return 0;
}
```

Результат выполнения

```
10 20 40
20 30 30
```

Нулевой указатель и динамическая память

- Указатель, который ни на что не указывает, равен `0` (или `NULL`). Разыменовывать его нельзя — аварийное завершение.
- Указатель без инициализации указывает «в никуда» — опасно.
- **Динамическая память** выделяется оператором `new` во время работы программы, освобождается `delete`. Это способ получить массив нужного размера, когда `n` вводится с клавиатуры.

Код

```
#include <iostream.h>
int main()
{
    int n;
    cout << "Сколько чисел: -> ";
    cin >> n;
    int *a = new int[n];            // массив из n int в динамической памяти
    for (int i = 0; i < n; i++)
    {
        cout << "Число " << i + 1 << ": -> ";
        cin >> a[i];
    }
    int s = 0;
    for (int i = 0; i < n; i++)
        s += a[i];
    cout << "Сумма = " << s << endl;
    delete [] a;                    // освободить память (для массива — с [])
    return 0;
}
```

Пример работы программы

```
Сколько чисел: -> 4
Число 1: -> 7
Число 2: -> 1
Число 3: -> 5
Число 4: -> 2
Сумма = 15
```

> **Нюанс.** Важно для массивов: ввод `n` и объявление `int a[n];` в Borland C++ 6.0 **не компилируется** («Constant expression required»). Размер статического массива должен быть константой. Либо `const int N = 100; int a[N];` и используем только первые `n` элементов, либо `new`.

> **Нюанс.** `p++` у указателя на `int` сдвигает на 4 байта, а `*p++` — это «взять значение, потом сдвинуть». Не путай с `(*p)++`, которое увеличивает **значение**, на которое указывает `p`.

1.11 Задачи

Задача 1.1. Длина окружности и площадь круга

Ввести радиус `R`, вывести длину окружности и площадь круга. Число π задать константой `3.14159`. Вывод — с двумя знаками после запятой (для этого нужен `<iomanip.h>`: `fixed` и `setprecision(2)`).

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <iomanip.h>
#include <conio.h>

const double PI = 3.14159;

int main()
{
    double r, len, area;
    cout << "Введите радиус: -> ";
    cin >> r;
    len = 2 * PI * r;
    area = PI * r * r;
    cout << fixed << setprecision(2);       // фиксированная запись, 2 знака после точки
    cout << "Длина окружности = " << len << endl;
    cout << "Площадь круга = " << area << endl;
    getch();
    return 0;
}
```

Пример работы программы

```
Введите радиус: -> 2.5
Длина окружности = 15.71
Площадь круга = 19.63
```

Задача 1.2. Цифры трёхзначного числа

Дано трёхзначное число. Найти сумму его цифр и число, записанное в обратном порядке. Подсказка: сотни `n / 100`, десятки `n / 10 % 10`, единицы `n % 10`.

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <conio.h>

int main()
{
    int n, a, b, c;
    cout << "Введите трёхзначное число: -> ";
    cin >> n;
    a = n / 100;            // сотни
    b = n / 10 % 10;        // десятки
    c = n % 10;             // единицы
    cout << "Сумма цифр = " << a + b + c << endl;
    cout << "Обратное число = " << c * 100 + b * 10 + a << endl;
    getch();
    return 0;
}
```

Пример работы программы

```
Введите трёхзначное число: -> 472
Сумма цифр = 13
Обратное число = 274
```

Задача 1.3. Обмен значений двух переменных

Поменять местами значения `a` и `b` двумя способами: с третьей переменной и без неё (на арифметике).

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <conio.h>

int main()
{
    int a, b, t;
    cout << "Введите a и b: -> ";
    cin >> a >> b;
    t = a;  a = b;  b = t;                  // способ 1: через временную переменную
    cout << "Способ 1: a = " << a << ", b = " << b << endl;
    a = a + b;  b = a - b;  a = a - b;      // способ 2: без третьей переменной
    cout << "Способ 2 (обратно): a = " << a << ", b = " << b << endl;
    getch();
    return 0;
}
```

Пример работы программы

```
Введите a и b: -> 3 8
Способ 1: a = 8, b = 3
Способ 2 (обратно): a = 3, b = 8
```

Задача 1.4. Секунды в часы, минуты, секунды

Ввести количество секунд (целое), вывести в формате `ч:мин:сек`. Используй `/` и `%`.

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <conio.h>

int main()
{
    long t;
    int h, m, s;
    cout << "Введите секунды: -> ";
    cin >> t;
    h = t / 3600;
    m = t % 3600 / 60;
    s = t % 60;
    cout << h << " ч " << m << " мин " << s << " с" << endl;
    getch();
    return 0;
}
```

Пример работы программы

```
Введите секунды: -> 3725
1 ч 2 мин 5 с
```

Задача 1.5. Работа с указателями

Объявить две переменные `a`, `b` и два указателя на них. Вывести значения через указатели, увеличить `a` на 10 через указатель, а затем обменять `a` и `b`, используя только указатели.

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <conio.h>

int main()
{
    int a, b, t;
    int *pa = &a, *pb = &b;
    cout << "Введите a и b: -> ";
    cin >> a >> b;
    cout << "Через указатели: " << *pa << " " << *pb << endl;
    *pa = *pa + 10;
    cout << "После +10: a = " << a << endl;
    t = *pa;  *pa = *pb;  *pb = t;          // обмен через указатели
    cout << "После обмена: a = " << a << ", b = " << b << endl;
    getch();
    return 0;
}
```

Пример работы программы

```
Введите a и b: -> 4 9
Через указатели: 4 9
После +10: a = 14
После обмена: a = 9, b = 14
```

1.12 Тест по теме

Тест по теме — лекция 1

1. Что такое идентификатор в C++?

а) Любое число, записанное в тексте программы
б) Имя, которое программист даёт объекту программы (переменной, функции, константе)
в) Оператор, выполняющий присваивание
г) Зарезервированное слово, понятное только компилятору

2. Какое из имён **нельзя** использовать как идентификатор?

а) `total_sum`
б) `Sum1`
в) `_count`
г) `2days`

3. Различаются ли в C++ прописные и строчные буквы в именах?

а) Различаются только в ключевых словах
б) Различаются только в именах функций
в) Нет: это одно и то же имя
г) Да: `Sum` и `sum` — разные идентификаторы

4. Какое из слов является ключевым (зарезервированным) в C++?

а) `getch`
б) `while`
в) `cout`
г) `Main`

5. Внутри каких символов пишется многострочный комментарий в C++?

а) `(* ... *)`
б) `{ ... }`
в) `< ... >`
г) `/* ... */`

6. Какая escape-последовательность переводит курсор на новую строку?

а) `\\`
б) `\0`
в) `\n`
г) `\t`

7. Какого типа данных **нет** в C++?

а) `bool`
б) `char`
в) `real`
г) `double`

8. Диапазон значений типа `char`:

а) от 0 до 1023
б) от 0 до 65535
в) от -128 до 127
г) от -32768 до 32767

9. Чему равно значение выражения `7/2`, если числа целые?

а) 3.5
б) 2
в) 3
г) 4

10. Чему равно `7%3`?

а) 0
б) 2
в) 1
г) 2.33

11. Какой тип получится при смешивании в выражении типов `float` и `double`?

а) `float`
б) `double`
в) `int`
г) `long`

12. Что делает запись `int *p = &a;`?

а) Записывает в `p` значение переменной `a`
б) Объявляет указатель `p` и записывает в него адрес переменной `a`
в) Создаёт копию переменной `a`
г) Объявляет переменную `a` по адресу `p`

13. Что выведет программа?

```
int a = 5;
int *p = &a;
*p = 9;
cout << a;
```

а) 5
б) ошибка компиляции
в) 9
г) адрес переменной `a`

14. Чему равно значение `WED` в описании `enum Day { MON, TUE, WED };`?

а) 0
б) 3
в) 1
г) 2

15. С какой функции начинается выполнение любой программы на C++?

а) `start()`
б) с первой функции в файле
в) `getch()`
г) `main()`

16. Какое объявление создаёт массив из 10 вещественных чисел?

а) `int a[10];`
б) `float a(10);`
в) `float a[1..10];`
г) `float a[10];`

17. Какая директива подключает средства ввода-вывода `cin` и `cout` в стиле Borland?

а) `#include <math.h>`
б) `#include <conio.h>`
в) `#include <iostream.h>`
г) `#include <string.h>`

18. Какой формат `printf` используется для вывода вещественного числа?

а) `%s`
б) `%f`
в) `%c`
г) `%d`

19. Какая фигура блок-схемы обозначает проверку условия?

а) параллелограмм
б) прямоугольник
в) овал
г) ромб

20. Что означает свойство алгоритма «конечность»?

а) Каждая команда записана на языке программирования
б) Алгоритм заканчивается за конечное число шагов
в) Алгоритм записан не более чем в десяти шагах
г) Алгоритм можно применять только к одному набору данных

21. Что произойдёт при компиляции `int const a = 5; a++;`?

а) Программа выведет 0
б) Переменная `a` станет равна 6
в) Переменная `a` останется 5 без ошибки
г) Ошибка: константу нельзя изменять

Ключи (проверяй себя только после того, как ответил сам): 1–б, 2–г, 3–г, 4–б, 5–г, 6–в, 7–в, 8–в, 9–в, 10–в, 11–б, 12–б, 13–в, 14–г, 15–г, 16–г, 17–в, 18–б, 19–г, 20–б, 21–г

1.13 Контрольные вопросы из программы (проверь себя устно)

1. Что такое алгоритм? Каковы его основные свойства? (1.3)
2. Основные способы записи алгоритма. (1.3)
3. Что такое блок-схема? Какие символы используются? (1.3)
4. Какие бывают основные типы алгоритмов? В чём различие линейного, ветвящегося и циклического? (1.3)
5. Общая структура программы на C++. (1.5)
6. Основные элементы синтаксиса языка. (1.4)
7. Что такое идентификаторы, ключевые слова, константы, переменные? (1.4, 1.7)
8. Что такое комментарии и escape-последовательности? Примеры. (1.4)
9. Основные типы данных C++. (1.7)
10. Что такое указатель? В чём разница между `&` и `*`? (1.10)

Лекция 2. Операции и выражения

Выражения — это то, из чего состоит почти каждая строка программы. На РК часто дают задачи вида «что будет выведено» и «как записать формулу на C++», поэтому здесь важна точность: приоритеты, целочисленное деление, префиксный и постфиксный `++`.

2.1 Основные понятия

- **Операция (оператор-знак)** — действие над данными: `+`, `*`, `==`, `&&`.
- **Операнд** — то, над чем выполняется операция (числа, переменные, вызовы функций).
- **Выражение** — комбинация операндов и операций, которая имеет **значение** определённого типа: `a * 2 + 1`.
- По числу операндов: **унарные** (один: `-a`, `!ok`, `i++`), **бинарные** (два: `a + b`) и одна **тернарная** (три: `?:`).
- **Приоритет** определяет, какая операция выполняется первой, **ассоциативность** — в каком направлении (слева направо или справа налево) выполняются операции одного приоритета.

2.2 Арифметические операции

Операция	Название	Пример	Результат
`+`	сложение	`7 + 2`	9
`-`	вычитание	`7 - 2`	5
`*`	умножение	`7 * 2`	14
`/`	деление	`7 / 2` и `7 / 2.0`	3 и 3.5
`%`	остаток от деления (только целые)	`7 % 2`	1

Фрагмент

```
cout << 17 / 5 << " " << 17 % 5 << " " << -17 / 5 << " ";
cout << -17 % 5 << " " << 17.0 / 5 << endl;
```

Результат выполнения

```
3 2 -3 -2 3.4
```

Нюанс. Операция `/` на двух целых — это **целая часть** от деления («div» из Pascal): `5/2 = 2`, `7/3 = 2`, `1/2 = 0`. Знак остатка при `%` совпадает со знаком делимого: `-17 % 5 = -2`. Операция `%` к `float` и `double` **не применяется** (для них есть `fmod(x, y)` из `<math.h>`).

Нюанс. Классическая ловушка: `double r = 1 / 2;` даст **0**, потому что деление выполняется на целых и лишь потом результат превращается в `double`. Правильно: `1.0 / 2`.

Инкремент и декремент

`++` увеличивает переменную на 1, `--` уменьшает на 1. Применяются **только к переменным** (не к константам и выражениям: `5++` и `(a+b)++` — ошибка).

Форма	Название	Что происходит
`b = a++;`	постфиксная	сначала `b` получает старое `a`, потом `a` увеличивается
`b = ++a;`	префиксная	сначала `a` увеличивается, потом `b` получает новое `a`

Фрагмент

```
int a = 5, b;
b = a++;                          // b = 5, a = 6
cout << a << " " << b << endl;
b = ++a;                          // a = 7, b = 7
cout << a << " " << b << endl;
int i = 10;
cout << i++ << " ";               // выведет 10, затем i станет 11
cout << ++i << endl;              // i станет 12 и выведет 12
```

Результат выполнения

```
6 5
7 7
10 12
```

> **Нюанс.** Не пиши в одном выражении два изменения одной переменной: `i = i++;`, `a[i] = i++;`, `x = x++ + ++x;` — результат не определён стандартом и зависит от компилятора.

Следующий пример «на трассировку»: `x = 0; x++; x = x + 1; x = x * x;` — после каждого шага x равен 1, 2, 4. Итог: **4**. А для `x = 0; x++; x = x * 1; x = x * x;` будет 1, 1, 1 — итог **1**.

Математические функции (<math.h>)

Запись	Что делает
`sqrt(x)`	квадратный корень
`pow(x, y)`	x в степени y
`fabs(x)`	модуль вещественного числа
`abs(n)`	модуль **целого** числа (<stdlib.h>)
`ceil(x)`	округление вверх: `ceil(3.226)` = 4
`floor(x)`	округление вниз: `floor(3.63)` = 3
`sin(x)`, `cos(x)`, `tan(x)`	тригонометрия, аргумент в **радианах**
`asin(x)`, `acos(x)`, `atan(x)`	обратные функции (арксинус, арккосинус, арктангенс)
`exp(x)`	e в степени x
`log(x)`	натуральный логарифм ln x
`log10(x)`	десятичный логарифм lg x
`fmod(x, y)`	остаток от деления вещественных

Фрагмент

```
cout << sqrt(16.0) << " " << pow(2.0, 10) << " " << fabs(-3.5) << endl;
cout << ceil(3.226) << " " << floor(3.63) << " " << log10(1000.0) << endl;
cout << sqrt(pow(5.0, 4)) << endl;            // sqrt(625) = 25
```

Результат выполнения

```
4 1024 3.5
4 3 3
25
```

> **Нюанс. Корень n-й степени:** пятый корень из x — это `pow(x, 1.0/5)`. Запись `pow(x, 1/5)` даст `pow(x, 0)` = 1, потому что `1/5` — целое деление и равно 0. Кубы и квадраты: `pow(a, 3)` или просто `a*a*a`.

> **Нюанс. Градусы в радианы:** `x_rad = x_deg * 3.14159 / 180`. В C++ нет функции `round()` в старых Borland — округление до целого пишут так: `floor(x + 0.5)`. Результат `ceil`/`floor` имеет тип `double`; чтобы получить `int`, пиши `(int)ceil(x)`.

Запись формул в линейной форме

Дробные «этажами» формулы нужно записать в одну строку с правильными скобками.

Формула	Запись на C++
(a + b) / (c − d)	`(a + b) / (c - d)`
a / (2 · b)	`a / (2 * b)` — а не `a / 2 * b` (это (a/2)·b)
√(b² − 4ac)	`sqrt(b * b - 4 * a * c)`
x² + 3x	`x * x + 3 * x`
|x − 5|	`fabs(x - 5)`
1 / (1 + x²)	`1 / (1 + x * x)` (при `x` типа `double`)

Случайные числа (Borland)

В `<stdlib.h>`: `randomize();` — один раз в начале программы запускает генератор (иначе при каждом запуске будет одна и та же последовательность). `random(n)` возвращает целое от **0 до n-1**. Старый универсальный способ: `rand()` (целое 0…32767).

Код

```
randomize();
int a = random(10);           // 0..9
int b = random(100) + 1;      // 1..100
int c = random(21) - 10;      // -10..10   (общая формула: A + random(B - A + 1) даёт [A; B])
```

Нюанс. В тестах встречается вопрос «как получить случайное число в диапазоне [-20, 50]» с «правильным» ответом `random(70)-20`, а вопрос «в каком диапазоне лежит `random(20)-5`» с ответом [-5, 15]. Если вычислять строго, `random(70)-20` даёт числа от -20 до **49**, а `random(20)-5` — от -5 до **14**. Верхняя граница у `random(n)` — это `n-1`. Знай это, но на тесте отвечай так, как принято в их банке вопросов, если нужного точного варианта нет.

2.3 Операции сравнения (равенства)

Операция	Смысл
`==`	равно
`!=`	не равно
`<`, `>`	меньше, больше
`<=`, `>=`	меньше или равно, больше или равно

Результат сравнения — **логическое значение**: `true` (1) или `false` (0). Например, `(1 != 1)` — ложь (0), `(3 < 5)` — истина (1).

Нюанс. `=` — присваивание, `==` — сравнение. Запись `if (a = 5)` не сравнивает, а **присваивает** `a` значение 5 и всегда истинно (5 ≠ 0). Знак `=` **не является** знаком сравнения (вопрос «что не знак сравнения» в тестах).

Нюанс. Двойное неравенство `0 < x < 10` в C++ **не работает как в математике**: сначала вычисляется `0 < x` (0 или 1), и это число сравнивается с 10 — всегда истина. Правильно: `x > 0 && x < 10`.

Нюанс. Вещественные числа сравнивают «с допуском»: `fabs(a - b) < 0.000001`, потому что `0.1 + 0.2 == 0.3` может оказаться ложью из-за ошибок округления.

Фрагмент

```
int x = 20;
cout << (0 < x < 10) << " " << (x > 0 && x < 10) << " " << (1 != 1) << endl;
```

Результат выполнения

```
1 0 0
```

2.4 Операции присваивания

Простое присваивание `a = b;` записывает значение справа в переменную слева. Присваивание само является выражением, поэтому можно **цепочкой**: `a = b = c = 0;` (выполняется справа налево).

Составные операции сокращают запись:

Запись	Эквивалент
`s += x`	`s = s + x`
`s -= x`	`s = s - x`
`s *= x`	`s = s * x`
`s /= x`	`s = s / x`
`s %= x`	`s = s % x`

Фрагмент

```
int x = 2;
x *= (x + 1);              // x = x * (x + 1) = 2 * 3 = 6
cout << x << " ";
int n = 17;
n %= 3;                    // n = n % 3 = 2
cout << n << " ";
int y = 7;
y /= 2;                    // целочисленное деление: 3
cout << y << " ";
int k = 4;
k *= 1;                    // k = k * 1 -- не меняется!
cout << k << endl;
```

Результат выполнения

```
6 2 3 4
```

> **Запомни.** Из выражений `x++`, `x = x + 1`, `x += 1` и `x *= 1` **только** `x *= 1` **не увеличивает** переменную на 1 (стандартный вопрос теста). `s += x` — это то же самое, что `s = s + x`.

2.5 Логические операции

Операция	Название	Смысл
`&&`	И («логическое умножение»)	истина, если **оба** операнда истинны
`||`	ИЛИ («логическое сложение»)	истина, если **хотя бы один** истинен
`!`	НЕ (отрицание)	меняет истину на ложь

В C++ **любое число, не равное 0, — истина**, а 0 — ложь. Приоритет: сначала `!`, потом `&&`, потом `||`.

Фрагмент

```
int a = 5, b = 0;
cout << (a && b) << " " << (a || b) << " " << (!a) << " " << (!b) << endl;
bool ok = (a > 3) && !(b > 0);
cout << ok << endl;                        // 1
```

Результат выполнения

```
0 1 0 1
1
```

> **Нюанс. Ленивое вычисление.** В `a && b` если `a` ложно, то `b` вообще **не вычисляется** (результат и так ясен); в `a || b`, если `a` истинно, `b` тоже пропускается. Этим пользуются для защиты: `if (d != 0 && x / d > 1)` — деления на ноль не будет.

Законы де Моргана: `!(a && b)` равно `!a || !b`, а `!(a || b)` равно `!a && !b`.

2.6 Побитовые операции

Работают с отдельными двоичными разрядами **целых** чисел.

Операция	Название	Пример (5 = 0101, 3 = 0011)	Результат
`&`	побитовое И	`5 & 3`	1 (0001)
`|`	побитовое ИЛИ	`5 | 3`	7 (0111)
`^`	исключающее ИЛИ	`5 ^ 3`	6 (0110)
`~`	побитовое НЕ	`~5`	-6
`<<`	сдвиг влево (×2 за каждый шаг)	`1 << 3`	8
`>>`	сдвиг вправо (÷2 за шаг)	`16 >> 2`	4

Фрагмент

```
cout << (5 & 3) << " " << (5 | 3) << " " << (5 ^ 3) << " " << (~5) << endl;
cout << (1 << 3) << " " << (16 >> 2) << endl;
cout << (6 & 1) << " " << (6 && 1) << endl;      // 0 и 1: & и && — разные операции!
int n = 13;
cout << ((n & 1) ? "нечётное" : "чётное") << endl;     // n & 1 == 1 для нечётных
```

Результат выполнения

```
1 7 6 -6
8 4
0 1
нечётное
```

> **Нюанс.** `&` и `&&`, `|` и `||` — разные операции. В `cout << a & b` нужны скобки `cout << (a & b)`, иначе `<<` (более высокий приоритет) сработает первым.

2.7 Операция «запятая»

Запятая объединяет выражения: они вычисляются **слева направо**, а значением всего выражения становится **последнее**. Чаще всего встречается в заголовке `for`.

Фрагмент

```
int a, b, x;
x = (a = 2, b = 3, a + b);            // x = 5
cout << x << endl;
for (int i = 0, j = 5; i < j; i++, j--)
    cout << i << "-" << j << "  ";
cout << endl;
```

Результат выполнения

```
5
0-5  1-4  2-3
```

Запятая в объявлении `int a = 1, b = 2;` или между аргументами функции — **не** операция, а просто разделитель.

2.8 Условная операция `?:`

Единственная **тернарная** операция: `условие ? значение_если_истина : значение_если_ложь`. Это краткая замена `if-else`, когда нужно получить значение.

Фрагмент

```
int a = 3, b = 7;
int mx = (a > b) ? a : b;                          // максимум из двух
cout << mx << endl;
int n = 8;
cout << (n % 2 == 0 ? "чётное" : "нечётное") << endl;   // в cout нужны скобки
int z = -4;
int sign = z > 0 ? 1 : (z < 0 ? -1 : 0);           // можно вкладывать
cout << sign << endl;
```

Результат выполнения

```
7
чётное
-1
```

Приоритет у `?:` очень низкий (чуть выше присваивания), поэтому выражение полностью заключают в скобки, когда используют внутри других выражений.

2.9 Операция `sizeof`

`sizeof` возвращает размер **в байтах**. Для типа скобки обязательны: `sizeof(int)`; для переменной можно без скобок: `sizeof x`.

Фрагмент

```
int a[10];
double d;
cout << sizeof(a) << " " << sizeof(a) / sizeof(a[0]) << " " << sizeof d << endl;
```

Результат выполнения

```
40 10 8
```

Приём `sizeof(a) / sizeof(a[0])` — число элементов массива (работает там, где массив объявлен, но не внутри функции, получившей массив параметром).

2.10 Операция разрешения области видимости `::`

С её помощью обращаются к имени из нужной области. Два типичных применения: **глобальная переменная, закрытая локальной с тем же именем**, и обращение к члену пространства имён или класса (`std::cout`, позже `Dog::bark`).

Код

```
#include <iostream.h>

int x = 100;                    // глобальная переменная

int main()
{
    int x = 5;                  // локальная скрывает глобальную
    cout << x << " " << ::x << endl;     // 5 100
    return 0;
}
```

Результат выполнения

```
5 100
```

2.11 Операции доступа к элементам: `.` и `->`

Для составных данных (структур и классов). Точка — когда есть сам объект, стрелка — когда есть **указатель** на него.

Код

```
#include <iostream.h>

struct Point { int x; int y; };

int main()
{
    Point p;
    p.x = 3;  p.y = 4;              // точка: обращение через объект
    Point *q = &p;
    q->x = 10;                      // стрелка: обращение через указатель
    (*q).y = 20;                    // то же самое, что q->y = 20
    cout << p.x << " " << p.y << endl;
    return 0;
}
```

Результат выполнения

```
10 20
```

2.12 Приоритет и ассоциативность

От самого высокого к самому низкому:

Ур.	Операции	Направление
1	`::`	
2	`() [] . ->` постфиксные `++ --`	→
3	`! ~` унарные `+ -` префиксные `++ --` `*` (разыменование) `&` (адрес) `(тип)` `sizeof` `new` `delete`	←
4	`* / %`	→
5	`+ -`	→
6	`<< >>`	→
7	`< <= > >=`	→
8	`== !=`	→
9	`&`	→
10	`^`	→
11	`|`	→
12	`&&`	→
13	`||`	→
14	`?:`	←
15	`= += -= *= /= %=`	←
16	`,`	→

Фрагмент

```
cout << 2 + 3 * 4 << " " << (2 + 3) * 4 << " " << 10 - 4 - 3 << endl;
cout << 2 + 3 * 4 % 5 << " " << 10 / 4 * 2.0 << endl;
int a, b, c;
a = b = c = 7;                    // справа налево
cout << a + b + c << endl;
```

Результат выполнения

```
14 20 3
4 4
21
```

> **Нюанс.** `a & b == c` читается как `a & (b == c)`, потому что `==` выше по приоритету, чем `&`. Если сомневаешься — ставь скобки. Скобки ничего не стоят, а ошибку приоритета искать долго.

2.13 Преобразование типов в выражениях

- В арифметике оба операнда приводятся к более «широкому» типу: `short + int → int`, `int + float → float`, `long + float → float`, `float + double → double`.
- Целочисленное деление, пока оба операнда целые, даже если результат потом кладут в `double`.
- Приведение вручную: `(double)a / b`, `(int)x`.

Разберём `k = (3/2) * (3%2)`: `3/2 = 1`, `3%2 = 1`, произведение 1. А `10 / 4 * 2.0`: `10/4` = 2 (целые), потом `2 * 2.0 = 4`.

2.14 Задачи

Задача 2.1. Формулы: среднее и гипотенуза

Дано два положительных числа `a` и `b`. Найти среднее арифметическое, среднее геометрическое `sqrt(a*b)` и гипотенузу прямоугольного треугольника с катетами `a`, `b`. Вывод — с тремя знаками после запятой.

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <iomanip.h>
#include <math.h>
#include <conio.h>

int main()
{
    double a, b;
    cout << "Введите a и b: -> ";
    cin >> a >> b;
    cout << fixed << setprecision(3);
    cout << "Среднее арифметическое = " << (a + b) / 2 << endl;
    cout << "Среднее геометрическое = " << sqrt(a * b) << endl;
    cout << "Гипотенуза = " << sqrt(a * a + b * b) << endl;
    getch();
    return 0;
}
```

Пример работы программы

```
Введите a и b: -> 3 4
Среднее арифметическое = 3.500
Среднее геометрическое = 3.464
Гипотенуза = 5.000
```

Задача 2.2. Размен суммы на купюры

Сумма в тенге вводится целым числом. Выдать её наименьшим числом купюр по 5000, 1000, 500 и 100 и вывести, сколько купюр каждого номинала и сколько осталось. Использовать `/` и `%`.

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <conio.h>

int main()
{
    int s, n5000, n1000, n500, n100;
    cout << "Введите сумму: -> ";
    cin >> s;
    n5000 = s / 5000;   s = s % 5000;
    n1000 = s / 1000;   s = s % 1000;
    n500 = s / 500;     s = s % 500;
    n100 = s / 100;     s = s % 100;
    cout << "По 5000: " << n5000 << endl;
    cout << "По 1000: " << n1000 << endl;
    cout << "По 500: " << n500 << endl;
    cout << "По 100: " << n100 << endl;
    cout << "Остаток: " << s << endl;
    getch();
    return 0;
}
```

Пример работы программы

```
Введите сумму: -> 12750
По 5000: 2
По 1000: 2
По 500: 1
По 100: 2
Остаток: 50
```

Задача 2.3. Максимум, минимум и треугольник

Ввести три числа. С помощью операции `?:` найти наибольшее и наименьшее. Проверить, существует ли треугольник с такими сторонами (каждая сторона меньше суммы двух других).

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <conio.h>

int main()
{
    int a, b, c, mx, mn;
    cout << "Введите три числа: -> ";
    cin >> a >> b >> c;
    mx = a > b ? a : b;
    mx = mx > c ? mx : c;
    mn = a < b ? a : b;
    mn = mn < c ? mn : c;
    cout << "Максимум = " << mx << ", минимум = " << mn << endl;
    bool tri = (a + b > c) && (a + c > b) && (b + c > a);
    cout << (tri ? "Треугольник существует" : "Треугольника нет") << endl;
    getch();
    return 0;
}
```

Пример работы программы

```
Введите три числа: -> 3 4 5
Максимум = 5, минимум = 3
Треугольник существует
```

Задача 2.4. Побитовые операции

Ввести целое число `n` и номер бита `k`. Вывести `n*8` (через сдвиг), `n/4` (через сдвиг), признак чётности (через `&`) и значение k-го бита числа (биты нумеруются с 0 справа).

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <conio.h>

int main()
{
    int n, k;
    cout << "Введите n и k: -> ";
    cin >> n >> k;
    cout << "n * 8 = " << (n << 3) << endl;
    cout << "n / 4 = " << (n >> 2) << endl;
    cout << (n & 1 ? "n нечётное" : "n чётное") << endl;
    cout << k << "-й бит равен " << ((n >> k) & 1) << endl;
    getch();
    return 0;
}
```

Пример работы программы

```
Введите n и k: -> 13 2
n * 8 = 104
n / 4 = 3
n нечётное
2-й бит равен 1
```

Задача 2.5. Трассировка выражений

Определи значения до запуска, потом сверь. Выражения: `a = 7; b = 2;` затем `a / b`, `a % b`, `a / (double)b`, `a++ + b`, `++a * b`, `a > b && b > 5 || a == 8`.

Решение — Borland C++ 6.0

```
int a = 7, b = 2;
cout << a / b << " " << a % b << " " << a / (double)b << endl;     // 3 1 3.5
cout << a++ + b << " ";                          // 7 + 2 = 9, потом a = 8
cout << ++a * b << " ";                          // a = 9, 9 * 2 = 18
cout << (a > b && b > 5 || a == 8) << endl;   // (1 && 0) || 0 = 0
```

Результат выполнения

```
3 1 3.5
9 18 0
```

2.15 Тест по теме

Тест по теме — лекция 2

1. Как записывается операция взятия остатка от деления?

а) `&`
б) `%`
в) `/`
г) `\`

2. К каким числам применяется операция `%`?

а) только к вещественным
б) только к целым
в) только к дробным
г) к целым и вещественным

3. Чему равно значение выражения `5/2` в C++ (оба числа целые)?

а) 3
б) 1
в) 2.5
г) 2

4. Чему равно значение выражения `7%2`?

а) 0
б) 3.5
в) 1
г) 3

5. Какое выражение **не** увеличивает значение переменной `x` на 1?

а) `x *= 1`
б) `x++`
в) `x = x + 1`
г) `x += 1`

6. Чему равно `x` после `x = 2; x *= (x + 1);`?

а) 5
б) 3
в) 6
г) 4

7. Какому оператору эквивалентно `n %= 3`?

а) `n = n * 3`
б) `n = n / 3`
в) `n = n - 3`
г) `n = n % 3`

8. Чему равно значение `k = (3/2) * (3%2);`?

а) 3
б) 1
в) 0.5
г) 2

9. Что выведет программа?

```
int i = 10;
cout << i++;
cout << ++i;
```

а) 1112
б) 1010
в) 1011
г) 1012

10. Какой знак обозначает логическое «И» (логическое умножение)?

а) `&`
б) `!`
в) `&&`
г) `||`

11. Какой знак обозначает «не равно»?

а) `!=`
б) `|=`
в) `<>`
г) `=!`

12. Что **не** является знаком сравнения?

а) `<`
б) `!=`
в) `==`
г) `=`

13. Как правильно записано применение `sizeof` к типу?

а) `sizeof [int]`
б) `sizeof int`
в) `sizeof: int`
г) `sizeof(int)`

14. Какая операция соответствует логическому отрицанию?

а) `!`
б) `=`
в) `?`
г) `&`

15. Чему равно `(1 != 1)`?

а) 2
б) истина (1)
в) выражение не имеет смысла
г) ложь (0)

16. Что выведет программа?

```
int a = 3, b = 7;
cout << (a > b ? a : b);
```

а) 7
б) 1
в) 0
г) 3

17. Значение `1 << 4`:

а) 4
б) 16
в) 14
г) 8

18. Значение выражения `2 + 3 * 4 % 5`:

а) 5
б) 2
в) 14
г) 4

19. Чему равно `10 / 4 * 2.0`?

а) 5
б) 1.25
в) 4
г) 0.8

20. Какая запись даёт корень пятой степени из `x`?

а) `pow(x, 1.0/5)`
б) `sqrt(x, 5)`
в) `sqrt(x/5)`
г) `pow(x, 1/5)`

21. Какая функция округляет число вверх?

а) `ceil(x)`
б) `fabs(x)`
в) `log(x)`
г) `floor(x)`

22. Какое значение получит `k` после `k = floor(3.63);`?

а) 3.5
б) 3
в) 2
г) 4

23. Функция модуля **целого** числа:

а) `fabs(x)`
б) `mod(x)`
в) `abs(x)`
г) `abc(x)`

24. Как записывается десятичный логарифм?

а) `lg(x)`
б) `log10(x)`
в) `log(10x)`
г) `ln(x)`

25. Какое число выдаёт `random(10)` в Borland C++?

а) целое от 0 до 10
б) целое от 1 до 10
в) вещественное от 0 до 1
г) целое от 0 до 9

26. В каком заголовочном файле описана функция `pow`?

а) `conio.h`
б) `stdlib.h`
в) `iostream.h`
г) `math.h`

Ключи (проверяй себя только после того, как ответил сам): 1–б, 2–б, 3–г, 4–в, 5–а, 6–в, 7–г, 8–б, 9–г, 10–в, 11–а, 12–г, 13–г, 14–а, 15–г, 16–а, 17–б, 18–г, 19–в, 20–а, 21–а, 22–б, 23–в, 24–б, 25–г, 26–г

2.16 Контрольные вопросы из программы (проверь себя устно)

1. Что такое выражение в C++? Что такое операция и операнд? (2.1)
2. Какие арифметические операции есть в C++? (2.2)
3. Какие операции сравнения и логические операции используются? (2.3, 2.5)
4. Что такое присваивание и составные присваивания? (2.4)
5. Что такое приоритет и ассоциативность операций? (2.12)
6. Для чего нужны операции `sizeof`, `?:`, `::`, `.`, `->`, запятая? (2.7–2.11)
7. Как работают побитовые операции? (2.6)
8. Как происходит преобразование типов в выражениях? (2.13)

Лекция 3. Ветвление: if, switch, goto. Цикл for

Здесь программа впервые «думает»: выбирает путь и повторяет действия. Именно на этих темах строится большинство задач РК — поэтому разберём не только синтаксис, но и типичные ловушки.

3.1 Управляющие структуры

Любой алгоритм собирается из трёх структур: **следование** (операторы по порядку), **ветвление** (выбор пути по условию: `if`, `switch`) и **цикл** (повторение: `for`, `while`, `do...while`). Операторы, меняющие порядок выполнения, называются **операторами управления**.

> **Запомни. Оператор ветвления** — оператор, который делит алгоритм на несколько путей: какой из них выполнится, зависит от того, выполнено условие или нет. Условный оператор — `if`, оператор выбора — `switch`.

3.2 Условный оператор if

Две формы записи:

Код

```
if (условие) оператор;                  // неполная форма: если ложь — ничего не делаем
if (условие) оператор1; else оператор2; // полная форма: либо первое, либо второе
```

Если по условию нужно выполнить **несколько** операторов, их заключают в фигурные скобки (составной оператор): `if (x > 0) { a = 1; b = 2; }`.

Условие — любое выражение: сравнение, логическая связка, число (ненулевое — истина), переменная `bool`.

Код

```
#include <iostream.h>
#include <conio.h>

int main()
{
    int x;
    cout << "Введите целое число: -> ";
    cin >> x;
    if (x > 0)
        cout << "Положительное" << endl;
    else if (x < 0)
        cout << "Отрицательное" << endl;
    else
        cout << "Ноль" << endl;
    getch();
    return 0;
}
```

Пример работы программы

```
Введите целое число: -> -7
Отрицательное
```

> **Нюанс.** Программа выше запущена только с первым числом (вторую и третью проверь сам). Цепочка `if ... else if ... else` проверяет условия сверху вниз и останавливается на первом истинном — остальные ветви пропускаются.

Вложенные if и «висячий else»

Внутри ветви `if` может быть другой `if`. `else` **всегда относится к ближайшему** `if` **без пары.** Отступы компилятору безразличны.

Фрагмент

```
int a = 1, b = 0;
if (a)
    if (b)
        cout << "X";
    else                    // относится к if (b), а не к if (a)!
        cout << "Y";
cout << endl;
```

Результат выполнения

```
Y
```

Типичные ошибки с if

Нюанс. Точка с запятой после условия. `if (x > 0); cout << "A";` — после `if` стоит пустой оператор, поэтому `cout` выполнится **всегда**. Аналогично для циклов: `for (...);` и `while (...);`.

Нюанс. Присваивание вместо сравнения. `if (x = 5)` записывает в `x` число 5 и всегда истинно. Нужно `if (x == 5)`.

Нюанс. Несколько операторов без скобок. В `if (x > 0) a = 1; b = 2;` второй оператор от условия **не зависит** — выполнится всегда.

Нюанс. Сравнение сразу двух границ: `if (0 < x < 10)` — ошибка логики, пиши `if (x > 0 && x < 10)`.

Примеры

Максимум из трёх чисел — через вложенное условие или через `max` в переменной:

Код

```
int max = a;
if (b > max) max = b;
if (c > max) max = c;
```

Високосный год — делится на 4 и не делится на 100, либо делится на 400:

Код

```
bool leap = (y % 4 == 0 && y % 100 != 0) || (y % 400 == 0);
```

Оценка по баллам — цепочка проверок от большего к меньшему:

Код

```
if (p >= 90) g = 5;
else if (p >= 75) g = 4;
else if (p >= 50) g = 3;
else g = 2;
```

3.3 Оператор выбора switch

`switch` выбирает вариант по значению **целочисленного выражения** (или `char`). Формат:

Код

```
switch (выражение)
{
    case значение1: операторы; break;
    case значение2: операторы; break;
    ...
    default: операторы;      // если ни один case не подошёл (необязательная ветвь)
}
```

Правила и особенности:

- выражение в скобках — **целое** (`int`, `char`, `enum`), **не** `float`, **не** строка;
- значения после `case` — **константы**, повторяться не могут, диапазонов (`case 1..5`) нет;
- `break` выходит из `switch`. Если `break` забыть, выполнение **«провалится»** в следующие ветви;

- `default` может стоять в любом месте, обычно в конце.

Фрагмент

```
int n = 45;
switch (n)
{
    case 23: cout << "aaa";
    case 45: cout << "bbb";      // нет break — проваливаемся дальше
    default: cout << "vvv";
             break;
}
cout << endl;
```

Результат выполнения

```
bbbvvv
```

Для `n = 3` и ветвей `case 2: ... break; case 3: ... break; default: ... break;` выполнится только ветвь `case 3`. А для `n = 45` из примера выше напечатается `bbbvvv` (в тестах — «bbbvvv»).

Один и тот же код для нескольких значений: перечисляем `case` подряд.

Фрагмент

```
int month = 4, days;
switch (month)
{
    case 1: case 3: case 5: case 7: case 8: case 10: case 12:
        days = 31; break;
    case 4: case 6: case 9: case 11:
        days = 30; break;
    case 2:
        days = 28; break;
    default:
        days = 0;
}
cout << days << endl;
```

Результат выполнения

```
30
```

Когда что. `switch` удобнее, когда много вариантов точного значения (меню, команда, номер месяца). `if` нужен для диапазонов (`x > 10`), вещественных чисел и сложных условий.

> **Нюанс.** Объявлять переменные прямо внутри `case` нужно в фигурных скобках `case 1: { int t = 5; ... break; }` — иначе компилятор выдаст ошибку «перепрыгивание через инициализацию».

3.4 Оператор безусловного перехода goto

`goto метка;` передаёт управление на оператор, помеченный `метка:`. Метка — обычный идентификатор с двоеточием.

Фрагмент

```
int i = 1;
povtor:
    cout << i << " ";
    i++;
    if (i <= 5) goto povtor;
cout << endl;
```

Результат выполнения

```
1 2 3 4 5
```

Реальная польза `goto` одна: **выйти сразу из нескольких вложенных циклов**.

Фрагмент

```
int i, j;
for (i = 1; i <= 5; i++)
    for (j = 1; j <= 5; j++)
        if (i * j == 12)
        {
            cout << "Нашли: " << i << " * " << j << endl;
            goto konec;
        }
konec:
cout << "Конец" << endl;
```

Результат выполнения

```
Нашли: 3 * 4
Конец
```

> **Нюанс.** `break` выходит только из **одного** цикла (внутреннего). Использовать `goto` часто не рекомендуется: он ухудшает читаемость («спагетти-код»), поэтому в остальных случаях пользуйся циклами и `if`. Прыгать нельзя в другую функцию и «через» объявление переменной с инициализацией.

3.5 Понятие цикла

Цикл — многократное выполнение одной и той же группы операторов (**тела цикла**). Одно выполнение тела — **итерация**. В C++ три оператора цикла: `for`, `while`, `do...while` (оператора вида `repeat ... until`, как в Pascal, **нет**).

3.6 Цикл со счётчиком for

Код

```
for (инициализация; условие; изменение)
    тело;
```

Порядок работы:

1. **Инициализация** выполняется **один раз** в самом начале (`i = 1`).
2. Проверяется **условие**. Если ложно — выход из цикла.
3. Выполняется **тело**.
4. Выполняется **изменение** (`i++`), возврат к шагу 2.

Фрагмент

```
for (int i = 1; i <= 5; i++)
    cout << i << " ";
cout << endl;
for (int i = 1; i < 10; i += 2)          // шаг 2: 1 3 5 7 9
    cout << i << " ";
cout << endl;
for (int i = 100; i > 0; i -= 30)        // в обратную сторону: 100 70 40 10
    cout << i << " ";
cout << endl;
```

Результат выполнения

```
1 2 3 4 5
1 3 5 7 9
100 70 40 10
```

Сколько раз выполнится цикл `for (i = a; i <= b; i += s)`: `(b - a) / s + 1` (при целой дроби). Например, `for (x = 1; x <= 10; x = x + 2)` — значения 1, 3, 5, 7, 9, итого **5** итераций.

Полезные формы заголовка:

Код

```
for (int i = 0, j = 10; i < j; i++, j--) { ... }   // два счётчика (запятая)
for (;;) { ... }                                    // бесконечный цикл: условия нет
int i = 0;
for (; i < 5; ) { i++; }                            // части можно опускать, ; остаются
```

Нюанс. Счётчик после цикла. В `for (a = 0; a < 10; a++)` последнее значение, при котором выполнялось тело, — 9, но после выхода из цикла `a` равно **10** (именно из-за `a == 10` условие стало ложным). Если в банке тестов встречается вопрос «чему равна `a` после цикла `int a; for(a = 0; a < 10; a++) cout << a;`» и варианты 9 и 10 — по логике верно 10; ответом «9» там считают последнее **напечатанное** число. Читай формулировку: «что напечатает» → до 9, «чему равна переменная после» → 10.

Нюанс. Переменная, объявленная в заголовке (`for (int i = ...)`), существует **только внутри цикла**. Нужен счётчик после цикла — объявляй его заранее: `int i; for (i = 0; ...)`.

Нюанс. Нельзя менять счётчик внутри тела и ожидать порядка, как в Pascal: в C++ это допустимо и часто приводит к зависанию. Вещественные счётчики (`for (double x = 0; x != 1; x += 0.1)`) опасны из-за ошибок округления — используй целый счётчик.

Типичные шаблоны

Фрагмент

```
// 1. Сумма 1..N
int n = 100, s = 0;
for (int i = 1; i <= n; i++)
    s += i;
cout << s << endl;                        // 5050

// 2. Произведение (факториал): 10! = 3628800 (до 12! помещается в int)
long f = 1;
for (int i = 2; i <= 10; i++)
    f *= i;
cout << f << endl;

// 3. Подсчёт: сколько чисел 1..10 делится на 3
int k = 0;
for (int x = 1; x <= 10; x++)
    if (x % 3 == 0) k++;
cout << k << endl;                        // 3 (это 3, 6, 9)
```

Результат выполнения

```
5050
3628800
3
```

Прочие результаты из тестов: количество чисел 1..10, кратных 5, равно **2**, кратных 9 — **1**.

Вложенные циклы

Для каждого значения внешнего счётчика внутренний цикл проходит **полностью**. Тело внутреннего выполняется `n * m` раз.

Фрагмент

```
for (int i = 1; i <= 4; i++)
{
    for (int j = 1; j <= i; j++)
        cout << "*";
    cout << endl;
}
```

Результат выполнения

```
*
**
***
****
```

3.7 Задачи

Задача 3.1. Знак, чётность, модуль

Ввести целое число. Вывести: положительное оно, отрицательное или ноль; чётное или нечётное (для нуля — чётное); его модуль (без `fabs`).

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <conio.h>

int main()
{
    int x;
    cout << "Введите число: -> ";
    cin >> x;
    if (x > 0)
        cout << "Положительное" << endl;
    else if (x < 0)
        cout << "Отрицательное" << endl;
    else
        cout << "Ноль" << endl;
    if (x % 2 == 0)
        cout << "Чётное" << endl;
    else
        cout << "Нечётное" << endl;        // для -7: -7 % 2 равно -1, а не 1, но != 0
    if (x < 0)
        x = -x;
    cout << "Модуль = " << x << endl;
    getch();
    return 0;
}
```

Пример работы программы

```
Введите число: -> -7
Отрицательное
Нечётное
Модуль = 7
```

Задача 3.2. Квадратное уравнение ax² + bx + c = 0

Ввести коэффициенты. Вычислить дискриминант и вывести корни или сообщение об их отсутствии. Не забыть случай `a = 0`.

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <math.h>
#include <conio.h>

int main()
{
    double a, b, c, d, x1, x2;
    cout << "Введите a, b, c: -> ";
    cin >> a >> b >> c;
    if (a == 0)
    {
        if (b == 0)
            cout << "Это не уравнение" << endl;
        else
            cout << "Линейное, x = " << -c / b << endl;
    }
    else
    {
        d = b * b - 4 * a * c;
        if (d < 0)
            cout << "Действительных корней нет" << endl;
        else if (d == 0)
            cout << "Один корень x = " << -b / (2 * a) << endl;
        else
        {
            x1 = (-b + sqrt(d)) / (2 * a);
            x2 = (-b - sqrt(d)) / (2 * a);
            cout << "x1 = " << x1 << ", x2 = " << x2 << endl;
        }
    }
    getch();
    return 0;
}
```

Пример работы программы

```
Введите a, b, c: -> 1 -3 2
x1 = 2, x2 = 1
```

Задача 3.3. Калькулятор на switch

Ввести два числа и знак операции (`+ - * /`). Вывести результат, на ноль делить нельзя — вывести сообщение. На любой другой символ — «неизвестная операция».

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <conio.h>

int main()
{
    double a, b;
    char op;
    cout << "Введите два числа: -> ";
    cin >> a >> b;
    cout << "Введите операцию (+ - * /): -> ";
    cin >> op;
    switch (op)
    {
        case '+': cout << a << " + " << b << " = " << a + b << endl; break;
        case '-': cout << a << " - " << b << " = " << a - b << endl; break;
        case '*': cout << a << " * " << b << " = " << a * b << endl; break;
        case '/':
            if (b == 0)
                cout << "Делить на ноль нельзя" << endl;
            else
                cout << a << " / " << b << " = " << a / b << endl;
            break;
        default:
            cout << "Неизвестная операция" << endl;
    }
    getch();
    return 0;
}
```

Пример работы программы

```
Введите два числа: -> 12 5
Введите операцию (+ - * /): -> /
12 / 5 = 2.4
```

Задача 3.4. Дни в месяце

Ввести год и номер месяца. Вывести количество дней, учитывая високосный год (февраль: 28 или 29). Для некорректного месяца — сообщение.

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <conio.h>

int main()
{
    int y, m, d;
    cout << "Введите год и месяц: -> ";
    cin >> y >> m;
    bool leap = (y % 4 == 0 && y % 100 != 0) || (y % 400 == 0);
    switch (m)
    {
        case 1: case 3: case 5: case 7: case 8: case 10: case 12:
            d = 31; break;
        case 4: case 6: case 9: case 11:
            d = 30; break;
        case 2:
            d = leap ? 29 : 28; break;
        default:
            d = 0;
    }
    if (d == 0)
        cout << "Такого месяца нет" << endl;
    else
        cout << "Дней в месяце: " << d << endl;
    getch();
    return 0;
}
```

Пример работы программы

```
Введите год и месяц: -> 2024 2
Дней в месяце: 29
```

Задача 3.5. Сумма ряда 1 + A + A^2 + ... + A^N

Дано вещественное число `A` и целое `N > 0`. Вывести сумму `1 + A + A^2 + A^3 + ... + A^N`. Каждое следующее слагаемое получаем из предыдущего умножением на `A` — так не нужен `pow` и вложенный цикл.

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <conio.h>

int main()
{
    double a, term = 1, sum = 1;
    int n;
    cout << "Введите вещественное число: -> ";
    cin >> a;
    cout << "Введите целое число: -> ";
    cin >> n;
    cout << "Сумма ряда 1";
    for (int i = 1; i <= n; i++)
    {
        term = term * a;               // A в степени i
        sum = sum + term;
        cout << " + " << term;
    }
    cout << " = " << sum << endl;
    getch();
    return 0;
}
```

Пример работы программы

```
Введите вещественное число: -> 3
Введите целое число: -> 5
Сумма ряда 1 + 3 + 9 + 27 + 81 + 243 = 364
```

Задача 3.6. Знакопеременный ряд 1 - A + A^2 - ... + (-1)^N * A^N

Те же входные данные. Здесь слагаемое получаем умножением на `-A`, знак при выводе печатаем отдельно.

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <conio.h>

int main()
{
    double a, term = 1, sum = 1;
    int n;
    cout << "Введите вещественное число: -> ";
    cin >> a;
    cout << "Введите целое число: -> ";
    cin >> n;
    cout << "Сумма ряда 1";
    for (int i = 1; i <= n; i++)
    {
        term = term * (-a);            // (-A) в степени i: знак чередуется сам
        sum = sum + term;
        if (term < 0)
            cout << " - " << -term;
        else
            cout << " + " << term;
    }
    cout << " = " << sum << endl;
    getch();
    return 0;
}
```

Пример работы программы

```
Введите вещественное число: -> 3
Введите целое число: -> 5
Сумма ряда 1 - 3 + 9 - 27 + 81 - 243 = -182
```

Задача 3.7. Приближение числа e: 2 + 1/2! + 1/3! + ... + 1/N!

Дано целое `N > 0`. Вывести сумму `2 + 1/(2!) + 1/(3!) + ... + 1/(N!)` — приближённое значение `e = 2.71828183...`. Факториал считаем накопительно: `f = f * i`. Для вывода 10 значащих цифр — `setprecision(10)`.

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <iomanip.h>
#include <conio.h>

int main()
{
    int n;
    double sum = 2, f = 1;
    cout << "Введите целое число: -> ";
    cin >> n;
    cout << setprecision(10);
    cout << "Сумма 2";
    for (int i = 2; i <= n; i++)
    {
        f = f * i;                      // f = i!
        sum = sum + 1.0 / f;            // 1.0, а не 1, иначе деление целых даст 0
        cout << " + 1/" << f;
    }
    cout << " = " << sum << endl;
    getch();
    return 0;
}
```

Пример работы программы

```
Введите целое число: -> 5
Сумма 2 + 1/2 + 1/6 + 1/24 + 1/120 = 2.716666667
```

Задача 3.8. Приближение exp(x): 1 + X + X^2/2! + ... + X^N/N!

Дано вещественное `X` и целое `N > 0`. Степень `p = X^i` и факториал `f = i!` наращиваем в цикле вместе.

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <iomanip.h>
#include <conio.h>

int main()
{
    double x, p = 1, f = 1, sum = 1;
    int n;
    cout << "Введите вещественное число: -> ";
    cin >> x;
    cout << "Введите целое число: -> ";
    cin >> n;
    cout << setprecision(7);
    cout << "Сумма 1";
    for (int i = 1; i <= n; i++)
    {
        p = p * x;                      // x в степени i
        f = f * i;                      // i!
        sum = sum + p / f;
        if (i == 1)
            cout << " + " << p;
        else
            cout << " + " << p << "/" << f;
    }
    cout << " = " << setprecision(10) << sum << endl;
    getch();
    return 0;
}
```

Пример работы программы

```
Введите вещественное число: -> 2.3
Введите целое число: -> 5
Сумма 1 + 2.3 + 5.29/2 + 12.167/6 + 27.9841/24 + 64.36343/120 = 9.675199417
```

Задача 3.9. Таблица умножения и «лесенка» из звёздочек

Ввести число `n` (от 1 до 9). Вывести таблицу умножения на `n` и треугольник из `n` строк, где в i-й строке `i` звёздочек.

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <conio.h>

int main()
{
    int n;
    cout << "Введите n: -> ";
    cin >> n;
    for (int i = 1; i <= 9; i++)
        cout << n << " x " << i << " = " << n * i << endl;
    for (int i = 1; i <= n; i++)
    {
        for (int j = 1; j <= i; j++)
            cout << "*";
        cout << endl;
    }
    getch();
    return 0;
}
```

Пример работы программы

```
Введите n: -> 4
4 x 1 = 4
4 x 2 = 8
4 x 3 = 12
4 x 4 = 16
4 x 5 = 20
4 x 6 = 24
4 x 7 = 28
4 x 8 = 32
4 x 9 = 36
*
**
***
****
```

3.8 Тест по теме

Тест по теме — лекция 3

1. Какой оператор в C++ называется условным?

а) `while`
б) `switch`
в) `for`
г) `if`

2. Какой оператор называется оператором выбора?

а) `while`
б) `for`
в) `switch`
г) `if`

3. Каким оператором выходят из `switch`?

а) `do`
б) `case`
в) `break`
г) `default`

4. Что выведет фрагмент при `n = 3`?

```
int n = 3;
switch (n)
{
    case 2: cout << "aaa"; break;
    case 3: cout << "bbb"; break;
    default: cout << "vvv"; break;
}
```

а) aaa
б) bbb
в) vvv
г) bbbvvv

5. Что выведет фрагмент при `n = 45`?

```
int n = 45;
switch (n)
{
    case 23: cout << "aaa";
    case 45: cout << "bbb";
    default: cout << "vvv"; break;
}
```

а) vvv
б) aaa
в) bbb
г) bbbvvv

6. Сколько раз выполнится тело цикла `for (x = 1; x <= 10; x = x + 2)`?

а) 10
б) 4
в) 5
г) 6

7. Какие значения принимает `i` в `for (i = 1; i < 10; i += 2)`?

а) 2 4 6 8 10
б) 1 2 3 4 5
в) 1 3 5 7 9
г) 1 3 5 7 9 10

8. Какие значения принимает `i` в `for (int i = 100; i > 0; i -= 10)`?

а) 90, 80, …, 10, 0
б) 50, 40, 30, 20, 10
в) 100, 80, 60, 40, 20
г) 100, 90, 80, …, 10

9. Что выведет фрагмент?

```
for (int i = 0; i < 3; i++)
    cout << i;
```

а) 321
б) 012
в) 0123
г) 123

10. Чему равно `s` после цикла?

```
int s = 0;
for (int i = 1; i <= 4; i++)
    s += i;
cout << s;
```

а) 24
б) 15
в) 10
г) 4

11. Чему равно `k` после выполнения?

```
int k = 0;
for (int x = 1; x <= 10; x++)
    if (x % 3 == 0) k++;
cout << k;
```

а) 2
б) 5
в) 4
г) 3

12. Чему равно `a` **после** завершения цикла `int a; for (a = 0; a < 10; a++) { }`?

а) 9
б) 11
в) 10
г) 0

13. Что напечатает фрагмент?

```
int a = 1, b = 0;
if (a)
    if (b) cout << "X";
    else cout << "Y";
```

а) X
б) Y
в) XY
г) ничего

14. Сколько раз будет выведено «A»?

```
for (int i = 0; i < 3; i++)
    for (int j = 0; j < 4; j++)
        cout << "A";
```

а) 3
б) 12
в) 7
г) 4

15. Что такое `for (;;)`?

а) Цикл, выполняющийся один раз
б) Бесконечный цикл (условия нет — оно считается истинным)
в) Цикл, который не выполнится ни разу
г) Синтаксическая ошибка

16. Что произойдёт при выполнении `if (x > 0); cout << "A";`?

а) Ошибка компиляции
б) «A» выведется при любом `x`, потому что после `if` пустой оператор
в) «A» не выведется никогда
г) «A» выведется только при `x > 0`

17. Что делает `if (x = 5)`?

а) Проверяет, что `x` не равен 5
б) Присваивает `x` значение 5, условие всегда истинно
в) Выдаёт ошибку компиляции
г) Сравнивает `x` с 5

18. Что произойдёт, если в ветви `case` забыть `break`?

а) Выполнится только `default`
б) Цикл завершится досрочно
в) Выполнение «провалится» в следующие ветви
г) Программа не скомпилируется

19. Какой тип выражения нельзя использовать в `switch`?

а) символьный (`char`)
б) вещественный (`float`, `double`)
в) целый (`int`)
г) перечисление (`enum`)

20. Что выполняет оператор `goto метка;`?

а) Условный переход, если метка истинна
б) Безусловный переход на оператор с этой меткой
в) Вызов функции с этим именем
г) Завершение программы

21. Какого оператора цикла в C++ **нет**?

а) `repeat until`
б) `for`
в) `do while`
г) `while`

22. Сколько существует операторов цикла в C++?

а) 4
б) 5
в) 3
г) 2

Ключи (проверяй себя только после того, как ответил сам): 1–г, 2–в, 3–в, 4–б, 5–г, 6–в, 7–в, 8–г, 9–б, 10–в, 11–г, 12–в, 13–б, 14–б, 15–б, 16–б, 17–б, 18–в, 19–б, 20–б, 21–а, 22–в

3.9 Контрольные вопросы из программы (проверь себя устно)

1. Что такое условный оператор и зачем он нужен? (3.2)
2. Синтаксис `if`. (3.2)
3. Чем отличаются `if`, `if...else` и вложенные `if`? (3.2)
4. Назначение `switch`; для чего нужны `case`, `break`, `default`? (3.3)
5. Основные различия `if` и `switch`. (3.3)
6. Что такое цикл со счётчиком? Синтаксис и структура `for`. (3.6)
7. Что такое инициализация, условие и итерационное выражение в `for`? (3.6)
8. Что такое вложенные циклы и когда они применяются? (3.6)
9. Как может возникнуть бесконечный цикл? (3.6)

Лекция 4A. Циклы while и do...while, break и continue

В лекции 3 мы повторяли действия, когда число повторений известно (`for`). Если же известно только **условие продолжения** («пока не введён ноль», «пока сумма меньше A»), удобнее `while` и `do...while`.

4A.1 Цикл с предусловием while

Код

```
while (условие)
{
    тело цикла;
}
```

Условие проверяется **до** каждого выполнения тела. Если оно ложно с самого начала, тело не выполнится **ни разу**. Внутри цикла обязательно должно что-то менять условие — иначе цикл будет бесконечным.

Фрагмент

```
int x = 2, y, count = 0;
while (x < 10)
{
    y = x * x;
    x = x + 2;
    count++;
}
cout << count << endl;           // x принимал значения 2, 4, 6, 8 -> четыре повтора
```

Результат выполнения

```
4
```

Ещё случаи из тестов: `x = 7; while (x <= 10) { cout << x; x++; }` — тело выполнится **4** раза (7, 8, 9, 10); `x = 7; while (x < 10) { ... }` — **3** раза (7, 8, 9).

4A.2 Цикл с постусловием do...while

Код

```
do
{
    тело цикла;
}
while (условие);       // после while(...) в этом цикле ставится ;
```

Сначала выполняется тело, потом проверяется условие. Значит, тело выполнится **минимум один раз**.

Фрагмент

```
int x = 10;
do
{
    cout << x << " ";           // выведется один раз, хотя условие уже ложно
    x++;
}
while (x < 5);
cout << endl;
while (x < 5)                   // а этот цикл не выполнится ни разу
    cout << "не выведется";
```

Результат выполнения

```
10
```

Классическое применение `do...while` — **ввод с проверкой**: пользователя нужно спросить хотя бы один раз.

Код

```
#include <iostream.h>
#include <conio.h>

int main()
{
    int mark;
    do
    {
        cout << "Введите оценку (от 1 до 5): -> ";
        cin >> mark;
    }
    while (mark < 1 || mark > 5);
    cout << "Принято: " << mark << endl;
    getch();
    return 0;
}
```

Пример работы программы

```
Введите оценку (от 1 до 5): -> 7
Введите оценку (от 1 до 5): -> 0
Введите оценку (от 1 до 5): -> 4
Принято: 4
```

Сравнение

	`while`	`do...while`	`for`
Название	с предусловием	с постусловием	со счётчиком (параметром)
Проверка условия	до тела	после тела	до тела
Минимум повторов	0	1	0
Когда выбирать	число повторов неизвестно, тело может не понадобиться	тело нужно выполнить хотя бы раз (ввод, меню)	число повторов известно заранее
Запись	`while (условие) {тело}`	`do {тело} while (условие);`	`for (начало; условие; шаг)`

Нюанс. В C++ **оба** цикла `while` и `do...while` повторяются, пока условие **истинно**, и выходят при **ложном** условии. В банке тестов вашей кафедры встречаются два вопроса в стиле Pascal: «в каком цикле выходим, когда условие истинно» (ответ банка — `do {} while(condition)`) и «когда условие не истинно» (ответ банка — `while (condition) {}`). Строго по языку это неверно, но на тесте нужно знать именно ожидаемый ответ: **постусловие — `do...while`, предусловие — `while`**.

Нюанс. `while (x = 5)` — присваивание вместо сравнения, цикл бесконечный. `while (x < 10);` — пустой оператор после условия: цикл «крутится» вхолостую, пока `x` не изменится сам (то есть вечно).

4A.3 Операторы прерывания цикла

- `break` — немедленный выход из цикла (из самого внутреннего, если циклы вложенные; также выходит из `switch`).
- `continue` — пропускает остаток **текущей итерации** и переходит к следующей (в `for` — выполняется шаг и проверка условия; в `while` — проверка условия).

Фрагмент

```
for (int i = 1; i <= 5; i++)
{
    if (i == 3) continue;       // пропустили 3
    cout << i << " ";
}
cout << endl;
for (int i = 1; i <= 5; i++)
{
    if (i == 3) break;          // выходим из цикла при i == 3
    cout << i << " ";
}
cout << endl;
```

Результат выполнения

```
1 2 4 5
1 2
```

Бесконечный цикл с выходом по `break` — удобная заготовка для «читать числа, пока не введён признак конца»:

Код

```
#include <iostream.h>
#include <conio.h>

int main()
{
    int x, sum = 0;
    while (true)                  // или while (1)
    {
        cout << "Введите число (0 - конец): -> ";
        cin >> x;
        if (x == 0)
            break;
        sum += x;
    }
    cout << "Сумма = " << sum << endl;
    getch();
    return 0;
}
```

Пример работы программы

```
Введите число (0 - конец): -> 5
Введите число (0 - конец): -> 3
Введите число (0 - конец): -> 8
Введите число (0 - конец): -> 0
Сумма = 16
```

4A.4 Типичные задачи на while

Цифры числа — берём последнюю цифру `n % 10`, отбрасываем её `n /= 10`, пока `n > 0`.

Алгоритм Евклида (НОД) — пока `b != 0`: `r = a % b; a = b; b = r;`. Ответ — `a`.

Количество повторов, пока не достигнут результат — именно так решаются задачи «найти наименьшее N, при котором сумма превысит A».

4A.5 Задачи

Задача 4А.1. Цифры числа: сумма, количество и обратное число

Ввести натуральное число. Найти сумму его цифр, количество цифр и число, записанное в обратном порядке. Циклом `while`.

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <conio.h>

int main()
{
    long n, m;
    int sum = 0, cnt = 0;
    long rev = 0;
    cout << "Введите натуральное число: -> ";
    cin >> n;
    m = n;
    while (m > 0)
    {
        int d = m % 10;          // последняя цифра
        sum += d;
        cnt++;
        rev = rev * 10 + d;      // дописываем цифру справа
        m = m / 10;              // отбрасываем последнюю цифру
    }
    cout << "Сумма цифр = " << sum << endl;
    cout << "Количество цифр = " << cnt << endl;
    cout << "Обратное число = " << rev << endl;
    getch();
    return 0;
}
```

Пример работы программы

```
Введите натуральное число: -> 40513
Сумма цифр = 13
Количество цифр = 5
Обратное число = 31504
```

Задача 4А.2. НОД и НОК двух чисел

Алгоритм Евклида: пока `b` не равно 0, заменяем пару `(a, b)` на `(b, a % b)`. НОК = `a * b / НОД`.

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <conio.h>

int main()
{
    int a, b, x, y, r;
    cout << "Введите два числа: -> ";
    cin >> a >> b;
    x = a;
    y = b;
    while (y != 0)
    {
        r = x % y;
        x = y;
        y = r;
    }
    cout << "НОД = " << x << endl;
    cout << "НОК = " << a / x * b << endl;
    getch();
    return 0;
}
```

Пример работы программы

```
Введите два числа: -> 84 36
НОД = 12
НОК = 252
```

Задача 4А.3. Наименьшее N, при котором 1 + 1/2 + ... + 1/N > A

Дано вещественное `A > 1`. Вывести наименьшее целое `N`, для которого сумма `1 + 1/2 + ... + 1/N` будет больше `A`, и саму сумму. Пока сумма не превысила `A`, добавляем следующее слагаемое.

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <iomanip.h>
#include <conio.h>

int main()
{
    double a, s = 1;
    int n = 1;
    cout << "Введите вещественное число: -> ";
    cin >> a;
    cout << "Наименьшее N для которого сумма 1";
    while (s <= a)
    {
        n++;
        s = s + 1.0 / n;
        cout << " + 1/" << n;
    }
    cout << " > " << a << " равно " << n << endl;
    cout << fixed << setprecision(3);
    cout << "Сумма равна " << s << endl;
    getch();
    return 0;
}
```

Пример работы программы

```
Введите вещественное число: -> 1.51
Наименьшее N для которого сумма 1 + 1/2 + 1/3 > 1.51 равно 3
Сумма равна 1.833
```

Задача 4А.4. Наибольшее N, при котором 1 + 1/2 + ... + 1/N < A

Здесь цикл идёт, пока **следующее** слагаемое ещё не сделает сумму больше или равной `A`. Поэтому в условии проверяем `s + 1.0/(n+1) < a`, а добавляем потом.

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <iomanip.h>
#include <conio.h>

int main()
{
    double a, s = 1;
    int n = 1;
    cout << "Введите вещественное число: -> ";
    cin >> a;
    cout << "Наибольшее N для которого сумма 1";
    while (s + 1.0 / (n + 1) < a)
    {
        n++;
        s = s + 1.0 / n;
        cout << " + 1/" << n;
    }
    cout << " < " << a << " равно " << n << endl;
    cout << fixed << setprecision(3);
    cout << "Сумма равна " << s << endl;
    getch();
    return 0;
}
```

Пример работы программы

```
Введите вещественное число: -> 1.84
Наибольшее N для которого сумма 1 + 1/2 + 1/3 < 1.84 равно 3
Сумма равна 1.833
```

Задача 4А.5. Последовательность A1 = 2, AN = 2 + 1/A(N-1)

Дано `D > 0`. Найти первый номер `K`, для которого `|A(K) - A(K-1)| < D`, и вывести `K`, а также `A(K-1)` и `A(K)`. Вид экрана здесь придуман самостоятельно (в задании так и сказано: «спроектируйте вид экрана сами»).

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <iomanip.h>
#include <math.h>
#include <conio.h>

int main()
{
    double d, prev, cur;
    int k;
    cout << "Введите D (D > 0): -> ";
    cin >> d;
    prev = 2;                           // A1
    cur = 2 + 1 / prev;                 // A2
    k = 2;
    while (fabs(cur - prev) >= d)       // пока разность не стала меньше D
    {
        prev = cur;
        cur = 2 + 1 / prev;
        k++;
    }
    cout << fixed << setprecision(5);
    cout << "Первый подходящий номер K = " << k << endl;
    cout << "A(" << k - 1 << ") = " << prev << endl;
    cout << "A(" << k << ") = " << cur << endl;
    getch();
    return 0;
}
```

Пример работы программы

```
Введите D (D > 0): -> 0.01
Первый подходящий номер K = 5
A(4) = 2.41667
A(5) = 2.41379
```

Задача 4А.6. Простые числа до N

Вывести все простые числа от 2 до `N` и их количество. Для каждого `i` ищем делитель `j` от 2 до `√i`; нашли — число составное, выходим `break`.

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <conio.h>

int main()
{
    int n, k = 0;
    cout << "Введите N: -> ";
    cin >> n;
    for (int i = 2; i <= n; i++)
    {
        bool prime = true;
        for (int j = 2; j * j <= i; j++)
            if (i % j == 0)
            {
                prime = false;
                break;                  // дальше проверять нет смысла
            }
        if (prime)
        {
            cout << i << " ";
            k++;
        }
    }
    cout << endl << "Всего: " << k << endl;
    getch();
    return 0;
}
```

Пример работы программы

```
Введите N: -> 30
2 3 5 7 11 13 17 19 23 29
Всего: 10
```

4A.6 Тест по теме

Тест по теме — лекция 4A

1. Как записывается цикл с предусловием?

а) `while (условие) { тело }`
б) `for (начало; конец; шаг)`
в) `do { тело } while (условие);`
г) `switch () { }`

2. Как записывается цикл с постусловием?

а) `for (начало; конец; шаг)`
б) `do { тело } while (условие);`
в) `while (условие) { тело }`
г) `repeat { тело } until (условие)`

3. Как записывается цикл с параметром?

а) `while (условие) { тело }`
б) `for (начало; условие; шаг)`
в) `loop (условие)`
г) `do { тело } while (условие);`

4. Сколько раз выполнится тело цикла?

```
int x = 2, y, c = 0;
while (x < 10) { y = x * x; x = x + 2; c++; }
cout << c;
```

а) 3
б) 4
в) 5
г) 10

5. Сколько раз выполнится тело цикла `x = 7; while (x <= 10) { cout << x; x++; }`?

а) 3
б) 5
в) 6
г) 4

6. Сколько раз выполнится тело цикла `x = 7; while (x < 10) { cout << x; x++; }`?

а) 5
б) 4
в) 3
г) 6

7. Какой цикл выполняется **хотя бы один раз** независимо от условия?

а) `do ... while`
б) ни один
в) `while`
г) `for`

8. Что выведет фрагмент?

```
int x = 10;
do { x++; } while (x < 5);
cout << x;
```

а) 5
б) ничего
в) 11
г) 10

9. Что делает оператор `break` внутри цикла?

а) Начинает цикл заново с первой итерации
б) Пропускает текущую итерацию
в) Завершает всю программу
г) Немедленно завершает цикл (самый внутренний)

10. Что делает оператор `continue`?

а) Пропускает остаток текущей итерации и переходит к следующей
б) Перезапускает цикл с начала
в) Завершает программу
г) Завершает цикл

11. Что выведет фрагмент?

```
for (int i = 1; i <= 5; i++)
{
    if (i == 3) continue;
    cout << i;
}
```

а) 12
б) 12345
в) 45
г) 1245

12. Чему равно `s` в конце?

```
int n = 12345, s = 0;
while (n > 0) { s += n % 10; n /= 10; }
cout << s;
```

а) 12345
б) 15
в) 54321
г) 5

13. Какой цикл выбрать, если число повторений заранее известно?

а) любой, кроме `for`
б) только `do...while`
в) `for`
г) `goto`

14. Что произойдёт при `while (x < 10);` (точка с запятой в конце), если `x` не меняется?

а) Тело выполнится один раз
б) Ошибка компиляции
в) Цикл с пустым телом будет бесконечным
г) Цикл не выполнится ни разу

Ключи (проверяй себя только после того, как ответил сам): 1–а, 2–б, 3–б, 4–б, 5–г, 6–в, 7–а, 8–в, 9–г, 10–а, 11–г, 12–б, 13–в, 14–в

Лекция 4Б. Одномерные массивы

4Б.1 Что такое массив

Массив — последовательность значений **одного типа** под одним именем. К отдельному элементу обращаются по **индексу** — номеру элемента (в квадратных скобках). Индексы в C++ начинаются с **0**: у массива из `n` элементов допустимы индексы от `0` до `n-1`.

Схема

```
int a[5];       индекс:   0     1     2     3     4
                значение: 7     3    12     0     5
                элемент: a[0]  a[1]  a[2]  a[3]  a[4]    (элементов 5, последний — a[4])
```

Зачем нужны массивы: хранить много однотипных данных (оценки группы, температуры за месяц) и обрабатывать их циклом, не заводя 100 переменных. Преимущество: удобная обработка циклом. Ограничения: размер фиксируется при объявлении, все элементы одного типа, нет проверки выхода за границы.

4Б.2 Объявление

Код

```
тип имя[размер];
int a[10];            // 10 целых
float b[12];          // 12 вещественных
char s[20];           // 20 символов (строка, лекция 6)
const int N = 100;
double d[N];          // размер — константа
```

Нюанс. Размер массива должен быть константой (число, `const int` или `#define`), известной при компиляции. Код `int n; cin >> n; int a[n];` в **Borland C++ 6.0 не компилируется** («Constant expression required»). Что делать: объявить с запасом `const int MAXN = 100; int a[MAXN];` и работать только с первыми `n` элементов, где `n` ввели с клавиатуры (проверив, что `n <= MAXN`); либо использовать `new` (лекция 1).

4Б.3 Инициализация

Фрагмент

```
int a[5] = {1, 2, 3, 4, 5};      // все пять значений заданы
int b[5] = {1, 2};               // остальные элементы обнуляются: 1 2 0 0 0
int c[5] = {0};                  // весь массив нулевой: 0 0 0 0 0
int d[] = {7, 8, 9};             // размер определяется по числу значений: 3
for (int i = 0; i < 5; i++) cout << b[i] << " ";
cout << endl;
for (int i = 0; i < 5; i++) cout << c[i] << " ";
cout << endl;
cout << sizeof(d) / sizeof(d[0]) << endl;
```

Результат выполнения

```
1 2 0 0 0
0 0 0 0 0
3
```

- `int a[5] = {0};` — быстрый способ обнулить весь массив; то же `int a[5] = {};` в современных компиляторах.
- `int a[5];` внутри функции — **не инициализирован**: в нём мусор. Глобальные массивы (вне функций) автоматически заполняются нулями.
- `int a[3] = {1, 2, 3, 4};` — ошибка («слишком много инициализаторов»).
- Массивы нельзя присваивать целиком: `b = a;` — ошибка; копирование — только циклом по элементам. Сравнение `a == b` сравнивает не содержимое, а адреса. Каждое действие над массивом — это цикл.

> **Нюанс.** Выход за границы (`a[5]` при размере 5, `a[-1]`) компилятор **не ловит**: программа читает или портит чужую память — результат непредсказуем. Типичная ошибка: цикл `for (i = 0; i <= n; i++)` вместо `i < n`.

4Б.4 Шаблон ввода и вывода

Это основа всех задач ниже: ввод количества элементов, ввод значений в цикле, вывод через запятую.

Код

```
const int MAXN = 100;
int a[MAXN], n, i;

do                                      // повторять, пока n не в допустимых пределах
{
    cout << "Введите число элементов массива: -> ";
    cin >> n;
}
while (n < 1 || n > MAXN);

for (i = 0; i < n; i++)
{
    cout << "Введите число " << i + 1 << ": -> ";     // для человека нумерация с 1
    cin >> a[i];
}

for (i = 0; i < n; i++)                 // вывод через запятую
{
    if (i > 0) cout << ", ";
    cout << a[i];
}
cout << endl;
```

4Б.5 Типовые алгоритмы

Во всех фрагментах массив `a` из `n = 6` элементов: `{4, -2, 9, 0, 7, 3}`.

Сумма, среднее арифметическое. Среднее — вещественное: делим на `(double)n`, иначе деление целых.

Фрагмент

```
int a[6] = {4, -2, 9, 0, 7, 3}, n = 6, s = 0;
for (int i = 0; i < n; i++)
    s += a[i];
cout << s << " " << (double)s / n << endl;       // 21 3.5
```

Результат выполнения

```
21 3.5
```

Минимум и максимум вместе с индексом — берём первый элемент за начальный и сравниваем остальные.

Фрагмент

```
int a[6] = {4, -2, 9, 0, 7, 3}, n = 6;
int mn = a[0], mx = a[0], imn = 0, imx = 0;
for (int i = 1; i < n; i++)
{
    if (a[i] < mn) { mn = a[i]; imn = i; }
    if (a[i] > mx) { mx = a[i]; imx = i; }
}
cout << mn << " на месте " << imn << ", " << mx << " на месте " << imx << endl;
```

Результат выполнения

```
-2 на месте 1, 9 на месте 2
```

Подсчёт по условию и сумма по условию. (чётные индексы, положительные, кратные 3 и т.д.)

Фрагмент

```
int a[6] = {4, -2, 9, 0, 7, 3}, n = 6, kPos = 0, sEven = 0;
for (int i = 0; i < n; i++)
{
    if (a[i] > 0) kPos++;                 // сколько положительных
    if (i % 2 == 0) sEven += a[i];        // сумма элементов с чётными ИНДЕКСАМИ 0, 2, 4
}
cout << kPos << " " << sEven << endl;     // 4 и 4 + 9 + 7 = 20
```

Результат выполнения

```
4 20
```

В тестах фрагмент `if (i % 2 == 0) s2 = s2 + a[i];` описывают как «сумма только чётных (по номеру) элементов», то есть элементов с **чётными индексами** `0, 2, 4...`, а `if (i % 2 != 0)` — с нечётными. Чётность проверяем у индекса `i`, а не у самого элемента `a[i]`.

Поиск первого элемента, удовлетворяющего условию. Запоминаем индекс и выходим по `break`; если не нашли — ответ «нет» (часто 0 или -1).

Фрагмент

```
int a[6] = {4, -2, 9, 0, 7, 3}, n = 6, pos = -1;
for (int i = 0; i < n; i++)
    if (a[i] > 8)
    {
        pos = i;
        break;
    }
cout << pos << endl;                  // 2
```

Результат выполнения

```
2
```

Обращение массива (реверс) на месте — меняем местами `a[i]` и `a[n-1-i]` только в первой половине.

Фрагмент

```
int a[6] = {4, -2, 9, 0, 7, 3}, n = 6, t;
for (int i = 0; i < n / 2; i++)
{
    t = a[i];  a[i] = a[n - 1 - i];  a[n - 1 - i] = t;
}
for (int i = 0; i < n; i++) cout << a[i] << " ";
cout << endl;                           // 3 7 0 9 -2 4
```

Результат выполнения

```
3 7 0 9 -2 4
```

> **Нюанс.** Если пройти по **всему** массиву до `n`, а не до `n / 2`, каждая пара поменяется дважды — массив вернётся в исходное состояние. Классическая ошибка.

Циклический сдвиг влево на одну позицию — сохраняем первый элемент, сдвигаем остальные, первый кладём в конец.

Фрагмент

```
int a[6] = {4, -2, 9, 0, 7, 3}, n = 6;
int first = a[0];
for (int i = 0; i < n - 1; i++)
    a[i] = a[i + 1];
a[n - 1] = first;
for (int i = 0; i < n; i++) cout << a[i] << " ";    // -2 9 0 7 3 4
cout << endl;
```

Результат выполнения

```
-2 9 0 7 3 4
```

Заполнение случайными числами — `randomize();` один раз, затем `random(n)` (Borland):

Код

```
randomize();
for (int i = 0; i < n; i++)
    a[i] = random(21) - 10;           // от -10 до 10
```

4Б.6 Сортировка

Сортировка — расстановка элементов по порядку (по возрастанию или убыванию). Самый простой метод — **пузырьковый**: многократно проходим по массиву, сравнивая соседние элементы и меняя их местами, если они стоят «не в том порядке». После каждого прохода самый большой из неупорядоченных «всплывает» в конец.

Код

```
#include <iostream.h>

int main()
{
    int a[6] = {4, -2, 9, 0, 7, 3}, n = 6, t;
    for (int p = 0; p < n - 1; p++)             // n - 1 проходов
        for (int i = 0; i < n - 1 - p; i++)     // хвост уже упорядочен
            if (a[i] > a[i + 1])                // для убывания: a[i] < a[i + 1]
            {
                t = a[i];  a[i] = a[i + 1];  a[i + 1] = t;
            }
    for (int i = 0; i < n; i++) cout << a[i] << " ";
    cout << endl;
    return 0;
}
```

Результат выполнения

```
-2 0 3 4 7 9
```

После **первого прохода** массив `4 2 3 1` (по возрастанию) превращается в `2 3 1 4`: сравниваем (4,2) — меняем; (4,3) — меняем; (4,1) — меняем. Максимум стал последним.

Сортировка выбором: на шаге `p` находим минимум среди `a[p..n-1]` и меняем его с `a[p]`.

Код

```
#include <iostream.h>

int main()
{
    int a[6] = {4, -2, 9, 0, 7, 3}, n = 6, t, imin;
    for (int p = 0; p < n - 1; p++)
    {
        imin = p;
        for (int i = p + 1; i < n; i++)
            if (a[i] < a[imin]) imin = i;
        t = a[p];  a[p] = a[imin];  a[imin] = t;
    }
    for (int i = 0; i < n; i++) cout << a[i] << " ";
    cout << endl;
    return 0;
}
```

Результат выполнения

```
-2 0 3 4 7 9
```

> **Запомни.** Обмен двух элементов — всегда через третью переменную: `t = a[i]; a[i] = a[j]; a[j] = t;`. Запись `a[i] = a[j]; a[j] = a[i];` теряет значение.

4Б.7 Задачи (лабораторная по одномерным массивам)

Приглашение «Введите число 1: -> » нумеруется с единицы, поэтому в `cout` пишем `i + 1`. Для компактности примеры запущены с короткими массивами.

Задача 4Б.1. Вывод массива в обратном порядке (Задание 1а)

Дан массив размера N. Вывести его элементы в обратном порядке.

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <conio.h>

int main()
{
    const int MAXN = 100;
    int a[MAXN], n, i;
    do
    {
        cout << "Введите число элементов массива: -> ";
        cin >> n;
    }
    while (n < 1 || n > MAXN);
    for (i = 0; i < n; i++)
    {
        cout << "Введите число " << i + 1 << ": -> ";
        cin >> a[i];
    }
    for (i = n - 1; i >= 0; i--)           // идём с конца к началу
    {
        cout << a[i];
        if (i > 0) cout << ", ";
    }
    cout << endl;
    getch();
    return 0;
}
```

Пример работы программы

```
Введите число элементов массива: -> 5
Введите число 1: -> 3
Введите число 2: -> 7
Введите число 3: -> 1
Введите число 4: -> 8
Введите число 5: -> 4
4, 8, 1, 7, 3
```

Задача 4Б.2. Сначала чётные, затем нечётные индексы (Задание 1б)

Дан массив размера N. Вывести вначале его элементы с чётными индексами, а затем — с нечётными. В условии индексы нумеруются **с 1** (как в приглашении «Введите число 2»): «чётные» — это 2-й, 4-й, 6-й элементы.

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <conio.h>

int main()
{
    const int MAXN = 100;
    int a[MAXN], n, i;
    bool first;
    do
    {
        cout << "Введите число элементов массива: -> ";
        cin >> n;
    }
    while (n < 1 || n > MAXN);
    for (i = 0; i < n; i++)
    {
        cout << "Введите число " << i + 1 << ": -> ";
        cin >> a[i];
    }
    cout << "Чётные индексы: ";
    first = true;
    for (i = 1; i < n; i += 2)            // позиции 2, 4, 6... (индексы массива 1, 3, 5...)
    {
        if (!first) cout << ", ";
        cout << a[i];
        first = false;
    }
    cout << endl << "Не чётные индексы: ";
    first = true;
    for (i = 0; i < n; i += 2)            // позиции 1, 3, 5...
    {
        if (!first) cout << ", ";
        cout << a[i];
        first = false;
    }
    cout << endl;
    getch();
    return 0;
}
```

Пример работы программы

```
Введите число элементов массива: -> 5
Введите число 1: -> 3
Введите число 2: -> 7
Введите число 3: -> 1
Введите число 4: -> 8
Введите число 5: -> 4
Чётные индексы: 7, 8
Не чётные индексы: 3, 1, 4
```

Нюанс. В C++ индекс первого элемента — 0, поэтому «чётные» по нумерации с единицы — это `a[1], a[3], ...`. Если преподаватель считает индексы по-C++ (с нуля), нужно поменять местами начальные значения в циклах (`i = 0` для чётных, `i = 1` для нечётных). Уточни на паре и поправь одну цифру.

Задача 4Б.3. Первый элемент с A[1] < A[i] < A[10] (Задание 2а)

Дан целочисленный массив A размера 10. Вывести номер первого из элементов `A[i]`, которые удовлетворяют двойному неравенству `A[1] < A[i] < A[10]`. Если таких нет — вывести 0. (Здесь `A[1]` — первый элемент, т.е. `a[0]`; `A[10]` — последний, т.е. `a[9]`.)

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <conio.h>

int main()
{
    int a[10], i, k = 0;
    for (i = 0; i < 10; i++)
    {
        cout << "Введите число " << i + 1 << ": -> ";
        cin >> a[i];
    }
    for (i = 0; i < 10; i++)
        if (a[0] < a[i] && a[i] < a[9])    // двойное неравенство записываем через &&
        {
            k = i + 1;                     // номер элемента с единицы
            break;                         // нужен только первый
        }
    cout << "Номер первого подходящего элемента: " << k << endl;
    if (k > 0)
        cout << "Это элемент " << a[k - 1] << endl;
    getch();
    return 0;
}
```

Пример работы программы

```
Введите число 1: -> 5
Введите число 2: -> 3
Введите число 3: -> 4
Введите число 4: -> 9
Введите число 5: -> 7
Введите число 6: -> 2
Введите число 7: -> 8
Введите число 8: -> 6
Введите число 9: -> 1
Введите число 10: -> 7
Номер первого подходящего элемента: 8
Это элемент 6
```

Задача 4Б.4. Прибавить последний элемент к нечётным (Задание 2б)

Дан целочисленный массив размера N. Прибавить к нечётным числам последний элемент. Первый и последний элементы не изменять.

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <conio.h>

int main()
{
    const int MAXN = 100;
    int a[MAXN], n, i;
    do
    {
        cout << "Введите число элементов массива: -> ";
        cin >> n;
    }
    while (n < 2 || n > MAXN);
    for (i = 0; i < n; i++)
    {
        cout << "Введите число " << i + 1 << ": -> ";
        cin >> a[i];
    }
    for (i = 1; i < n - 1; i++)            // только средние: без a[0] и a[n-1]
        if (a[i] % 2 != 0)                 // != 0, а не == 1: для отрицательных остаток -1
            a[i] = a[i] + a[n - 1];
    for (i = 0; i < n; i++)
    {
        cout << a[i];
        if (i < n - 1) cout << ", ";
    }
    cout << endl;
    getch();
    return 0;
}
```

Пример работы программы

```
Введите число элементов массива: -> 6
Введите число 1: -> 3
Введите число 2: -> 7
Введите число 3: -> 4
Введите число 4: -> 9
Введите число 5: -> -2
Введите число 6: -> 7
3, 14, 4, 16, -2, 7
```

Задача 4Б.5. Поменять местами минимальный и максимальный (Задание 3а)

Поменять местами минимальный и максимальный элементы массива размера 10. Запоминаем **индексы** min и max, а затем меняем элементы через третью переменную. Если минимумов несколько, берётся первый.

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <conio.h>

int main()
{
    int a[10], i, imin = 0, imax = 0, t;
    for (i = 0; i < 10; i++)
    {
        cout << "Введите число " << i + 1 << ": -> ";
        cin >> a[i];
    }
    for (i = 1; i < 10; i++)
    {
        if (a[i] < a[imin]) imin = i;
        if (a[i] > a[imax]) imax = i;
    }
    t = a[imin];  a[imin] = a[imax];  a[imax] = t;
    for (i = 0; i < 10; i++)
    {
        cout << a[i];
        if (i < 9) cout << ", ";
    }
    cout << endl;
    getch();
    return 0;
}
```

Пример работы программы

```
Введите число 1: -> 3
Введите число 2: -> 7
Введите число 3: -> 4
Введите число 4: -> 6
Введите число 5: -> 5
Введите число 6: -> 8
Введите число 7: -> 7
Введите число 8: -> 6
Введите число 9: -> 5
Введите число 10: -> 9
9, 7, 4, 6, 5, 8, 7, 6, 5, 3
```

Задача 4Б.6. Положительные заменить максимальным (Задание 3б)

Заменить все положительные элементы целочисленного массива размера 10 на значение максимального. Сначала находим максимум по исходному массиву, потом заменяем.

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <conio.h>

int main()
{
    int a[10], i, mx;
    for (i = 0; i < 10; i++)
    {
        cout << "Введите число " << i + 1 << ": -> ";
        cin >> a[i];
    }
    mx = a[0];
    for (i = 1; i < 10; i++)
        if (a[i] > mx) mx = a[i];
    for (i = 0; i < 10; i++)
        if (a[i] > 0)                       // ноль не положительный: он не заменяется
            a[i] = mx;
    for (i = 0; i < 10; i++)
    {
        cout << a[i];
        if (i < 9) cout << ", ";
    }
    cout << endl;
    getch();
    return 0;
}
```

Пример работы программы

```
Введите число 1: -> 3
Введите число 2: -> -7
Введите число 3: -> 4
Введите число 4: -> -1
Введите число 5: -> 0
Введите число 6: -> 6
Введите число 7: -> -2
Введите число 8: -> 5
Введите число 9: -> -3
Введите число 10: -> 9
9, -7, 9, -1, 0, 9, -2, 9, -3, 9
```

Задача 4Б.7. Сортировка по возрастанию и по убыванию

Ввести массив и вывести его отсортированным по возрастанию, затем по убыванию (пузырьковый метод). Для убывания достаточно сменить знак сравнения.

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <conio.h>

int main()
{
    const int MAXN = 100;
    int a[MAXN], n, i, p, t;
    cout << "Введите число элементов массива: -> ";
    cin >> n;
    for (i = 0; i < n; i++)
    {
        cout << "Введите число " << i + 1 << ": -> ";
        cin >> a[i];
    }
    for (p = 0; p < n - 1; p++)
        for (i = 0; i < n - 1 - p; i++)
            if (a[i] > a[i + 1])
            {
                t = a[i];  a[i] = a[i + 1];  a[i + 1] = t;
            }
    cout << "По возрастанию:";
    for (i = 0; i < n; i++) cout << " " << a[i];
    cout << endl;
    for (p = 0; p < n - 1; p++)
        for (i = 0; i < n - 1 - p; i++)
            if (a[i] < a[i + 1])
            {
                t = a[i];  a[i] = a[i + 1];  a[i + 1] = t;
            }
    cout << "По убыванию:";
    for (i = 0; i < n; i++) cout << " " << a[i];
    cout << endl;
    getch();
    return 0;
}
```

Пример работы программы

```
Введите число элементов массива: -> 5
Введите число 1: -> 9
Введите число 2: -> 2
Введите число 3: -> 7
Введите число 4: -> 1
Введите число 5: -> 5
По возрастанию: 1 2 5 7 9
По убыванию: 9 7 5 2 1
```

Задача 4Б.8. Среднее арифметическое без минимума и максимума

Вычислить среднее арифметическое элементов массива, не учитывая минимальное и максимальное значения. Сумма всех минус min минус max, делённая на `n - 2`. Нужно хотя бы 3 элемента.

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <conio.h>

int main()
{
    const int MAXN = 100;
    int a[MAXN], n, i, mn, mx, s = 0;
    do
    {
        cout << "Введите число элементов массива (не менее 3): -> ";
        cin >> n;
    }
    while (n < 3 || n > MAXN);
    for (i = 0; i < n; i++)
    {
        cout << "Введите число " << i + 1 << ": -> ";
        cin >> a[i];
    }
    mn = mx = a[0];
    for (i = 0; i < n; i++)
    {
        s += a[i];
        if (a[i] < mn) mn = a[i];
        if (a[i] > mx) mx = a[i];
    }
    cout << "Среднее без min и max = " << (double)(s - mn - mx) / (n - 2) << endl;
    getch();
    return 0;
}
```

Пример работы программы

```
Введите число элементов массива (не менее 3): -> 5
Введите число 1: -> 4
Введите число 2: -> 9
Введите число 3: -> 1
Введите число 4: -> 7
Введите число 5: -> 6
Среднее без min и max = 5.66667
```

Задача 4Б.9. Подсчёт знаков и циклический сдвиг

Ввести массив, посчитать количество положительных, отрицательных и нулевых элементов и циклически сдвинуть массив влево на одну позицию.

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <conio.h>

int main()
{
    const int MAXN = 100;
    int a[MAXN], n, i, pos = 0, neg = 0, zero = 0, first;
    cout << "Введите число элементов массива: -> ";
    cin >> n;
    for (i = 0; i < n; i++)
    {
        cout << "Введите число " << i + 1 << ": -> ";
        cin >> a[i];
        if (a[i] > 0) pos++;
        else if (a[i] < 0) neg++;
        else zero++;
    }
    cout << "Положительных: " << pos << ", отрицательных: " << neg
         << ", нулей: " << zero << endl;
    first = a[0];
    for (i = 0; i < n - 1; i++)
        a[i] = a[i + 1];
    a[n - 1] = first;
    cout << "После сдвига:";
    for (i = 0; i < n; i++) cout << " " << a[i];
    cout << endl;
    getch();
    return 0;
}
```

Пример работы программы

```
Введите число элементов массива: -> 6
Введите число 1: -> 4
Введите число 2: -> -2
Введите число 3: -> 0
Введите число 4: -> 9
Введите число 5: -> -5
Введите число 6: -> 0
Положительных: 2, отрицательных: 2, нулей: 2
После сдвига: -2 0 9 -5 0 4
```

4Б.8 Тест по теме

Тест по теме — лекция 4Б

1. Что такое массив?

а) Набор элементов разных типов
б) Последовательность значений одного типа под одним именем
в) Тип данных, определяющий размер памяти программы
г) Одна переменная, хранящая только целые числа

2. С какого числа начинается нумерация индексов массива в C++?

а) -1
б) 1
в) 0
г) 2

3. Какое объявление создаёт массив из 12 целых чисел?

а) `int a(12);`
б) `int a[12];`
в) `float a[12];`
г) `int a[1..12];`

4. Что означает запись `float a[10];`?

а) Массив `a` из 10 вещественных чисел
б) Одно вещественное число, равное 10
в) Матрицу 10 на 10
г) Последовательность целых чисел от 1 до 10

5. Какой индекс имеет последний элемент массива `int a[8]`?

а) 9
б) 7
в) 8
г) 6

6. Чему равно значение `a[3] + a[4]`?

```
int a[5] = {1, 2, 3};
cout << a[3] + a[4];
```

а) 7
б) 0
в) 5
г) неопределено (мусор)

7. Что содержит локальный массив `int a[5];` без инициализации?

а) Единицы
б) Он пустой (0 элементов)
в) Случайные значения («мусор»)
г) Нули

8. Как обнулить сразу весь массив при объявлении?

а) `int a(10) = 0;`
б) `int a[10] = 0;`
в) `int a[10] = {0};`
г) `int a[0..10];`

9. Почему в Borland C++ 6.0 нельзя написать `int n; cin >> n; int a[n];`?

а) Нельзя вводить `n` с клавиатуры
б) Массивы не поддерживают тип `int`
в) Имя `n` зарезервировано
г) Размер массива должен быть константой, известной при компиляции

10. Что выведет фрагмент?

```
int a[4] = {5, 6, 7, 8};
cout << a[a[0] - 4];
```

а) 5
б) 6
в) 1
г) 7

11. Что выведет фрагмент?

```
int a[5] = {1, 2, 3, 4, 5};
int t;
for (int i = 0; i < 5 / 2; i++)
{
    t = a[i]; a[i] = a[4 - i]; a[4 - i] = t;
}
for (int i = 0; i < 5; i++) cout << a[i];
```

а) 51423
б) 54321
в) 12345
г) 15243

12. Что вычисляет фрагмент?

```
int a[6] = {1, 2, 3, 4, 5, 6}, s2 = 0;
for (int i = 0; i < 6; i++)
    if (i % 2 == 0) s2 = s2 + a[i];
cout << s2;
```

а) Сумму элементов с чётными индексами (результат 9)
б) Сумму элементов с нечётными индексами (результат 12)
в) Количество чётных элементов (результат 3)
г) Сумму всех элементов (результат 21)

13. Как называется процесс расстановки элементов массива по порядку?

а) проверка
б) поиск
в) сортировка
г) подстановка

14. Как называется число, с помощью которого обращаются к одному элементу массива?

а) индекс
б) среднее арифметическое
в) матрица
г) тип

15. Как после первого прохода «пузырька» по возрастанию выглядит массив `4 2 3 1`?

а) `2 3 1 4`
б) `1 2 3 4`
в) `4 2 3 1`
г) `2 4 3 1`

16. Как правильно поменять местами `a[i]` и `a[j]`?

а) `a[i] = a[j]; a[j] = a[i];`
б) `t = a[i]; a[j] = t; a[i] = a[j];`
в) `a[i] = a[j];`
г) `t = a[i]; a[i] = a[j]; a[j] = t;`

17. Что произойдёт, если при реверсе массива пройти циклом по **всем** n элементам (а не по n/2) с обменом `a[i]` и `a[n-1-i]`?

а) Массив вернётся к исходному виду: каждая пара обменяется дважды
б) Программа аварийно завершится
в) Все элементы станут равными
г) Массив развернётся, как нужно

Ключи (проверяй себя только после того, как ответил сам): 1–б, 2–в, 3–б, 4–а, 5–б, 6–б, 7–в, 8–в, 9–г, 10–б, 11–б, 12–а, 13–в, 14–а, 15–а, 16–г, 17–а

Лекция 4B. Двумерные массивы (матрицы)

4B.1 Что это

Двумерный массив — «массив массивов»: таблица из строк и столбцов. Его ещё называют **матрицей**. Каждый элемент определяется двумя индексами: `a[i][j]`, где `i` — номер строки, `j` — номер столбца (оба с 0).

Код

```
int a[3][4];          // 3 строки, 4 столбца: всего 12 элементов
const int N = 5;
double m[N][N];       // квадратная матрица 5 x 5
```

Схема

```
int a[3][4]        j=0   j=1   j=2   j=3
         i=0      a[0][0] a[0][1] a[0][2] a[0][3]
         i=1      a[1][0] a[1][1] a[1][2] a[1][3]
         i=2      a[2][0] a[2][1] a[2][2] a[2][3]
```

В памяти элементы лежат **по строкам**: сначала вся строка 0, затем строка 1 и т.д.

Инициализация

Фрагмент

```
int a[2][3] = {{1, 2, 3}, {4, 5, 6}};     // по строкам, в фигурных скобках
int b[2][3] = {1, 2, 3, 4, 5, 6};         // то же самое, плоским списком
int c[2][3] = {0};                        // всё нулями
int d[][3]  = {{1, 2, 3}, {4, 5, 6}};     // первую размерность можно не указывать
cout << a[1][0] << " " << b[1][2] << " " << c[1][1] << " ";
cout << sizeof(d) / sizeof(int) << endl;
```

Результат выполнения

```
4 6 0 6
```

Пропустить можно **только первую** размерность: `int d[][3]` допустимо, `int d[2][]` — нет.

> **Нюанс.** Ограничения те же, что и у одномерных: размеры — константы (в Borland 6), нет проверки границ, присваивание матриц целиком запрещено — всё через вложенные циклы.

4B.2 Шаблон: ввод, вывод и вложенные циклы

Для обхода матрицы нужны **два вложенных цикла**: внешний по строкам `i`, внутренний по столбцам `j`.

Код

```
const int MAXN = 10;
int a[MAXN][MAXN], n, m, i, j;

cout << "Введите число строк и столбцов: -> ";
cin >> n >> m;
for (i = 0; i < n; i++)
{
    cout << "Строка " << i + 1 << ": -> ";
    for (j = 0; j < m; j++)
        cin >> a[i][j];                  // элементы строки вводим через пробел
}
for (i = 0; i < n; i++)                  // вывод таблицей: setw(4) задаёт ширину поля
{
    for (j = 0; j < m; j++)
        cout << setw(4) << a[i][j];      // нужен #include <iomanip.h>
    cout << endl;                        // после каждой строки — новая строка
}
```

4B.3 Типовые алгоритмы

Матрица `a` размером 3 × 3: `{{1,2,3},{4,5,6},{7,8,9}}`.

Главная диагональ — элементы, у которых номер строки равен номеру столбца: `i == j` (то есть `a[i][i]`). **Побочная диагональ** — `i + j == n - 1`, то есть `a[i][n-1-i]`. **Выше главной диагонали** — `j > i`; **ниже** — `j < i`.

Фрагмент

```
int a[3][3] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}}, n = 3;
int main_d = 0, sec_d = 0, up = 0, low = 0;
for (int i = 0; i < n; i++)
    for (int j = 0; j < n; j++)
    {
        if (i == j) main_d += a[i][j];          // главная: 1 + 5 + 9
        if (i + j == n - 1) sec_d += a[i][j];   // побочная: 3 + 5 + 7
        if (j > i) up += a[i][j];               // выше главной: 2 + 3 + 6
        if (j < i) low += a[i][j];              // ниже главной: 4 + 7 + 8
    }
cout << main_d << " " << sec_d << " " << up << " " << low << endl;
```

Результат выполнения

```
15 15 11 19
```

(Именно так выглядит тестовое задание «`if (j < i) s += a[i][j]` — сумма элементов **под** главной диагональю», а «`s += a[i][i]` — сумма элементов главной диагонали».)

Суммы по строкам и по столбцам. По строкам: сумму обнуляем в начале каждой строки (внутри внешнего цикла). По столбцам: внешний цикл по `j`, внутренний по `i`.

Фрагмент

```
int a[3][3] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}}, n = 3, s;
for (int i = 0; i < n; i++)
{
    s = 0;                                          // обнуляем для каждой строки!
    for (int j = 0; j < n; j++)
        s += a[i][j];
    cout << "строка " << i + 1 << ": " << s << endl;
}
for (int j = 0; j < n; j++)
{
    s = 0;
    for (int i = 0; i < n; i++)
        s += a[i][j];
    cout << "столбец " << j + 1 << ": " << s << endl;
}
```

Результат выполнения

```
строка 1: 6
строка 2: 15
строка 3: 24
столбец 1: 12
столбец 2: 15
столбец 3: 18
```

Транспонирование — замена строк столбцами: меняем `a[i][j]` и `a[j][i]`, но только для `j > i` (над диагональю), иначе вернём всё назад.

Фрагмент

```
int a[3][3] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}}, n = 3, t;
for (int i = 0; i < n; i++)
    for (int j = i + 1; j < n; j++)
    {
        t = a[i][j];  a[i][j] = a[j][i];  a[j][i] = t;
    }
for (int i = 0; i < n; i++)
{
    for (int j = 0; j < n; j++) cout << a[i][j] << " ";
    cout << endl;
}
```

Результат выполнения

```
1 4 7
2 5 8
3 6 9
```

Изменение побочной диагонали (частая задача): записываем нужное значение в `a[i][n-1-i]`.

Фрагмент

```
int a[3][3] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}}, n = 3;
for (int i = 0; i < n; i++)
    a[i][n - 1 - i] = 0;                          // обнулили побочную диагональ
for (int i = 0; i < n; i++)
{
    for (int j = 0; j < n; j++) cout << a[i][j] << " ";
    cout << endl;
}
```

Результат выполнения

```
1 2 0
4 0 6
0 8 9
```

Произведение матриц (`C = A * B`, обе квадратные): `c[i][j]` — сумма произведений `a[i][k] * b[k][j]` по `k`. Накапливающую сумму обнуляем перед внутренним циклом по `k`.

4В.4 Задачи (лабораторная по многомерным массивам)

Нумерация — как в твоём списке заданий.

Задача 4В.1. №1. Сложить две матрицы

Написать программу, которая складывает два двумерных массива и результат записывает в третий. Складывать можно матрицы **одинакового размера**: `c[i][j] = a[i][j] + b[i][j]`.

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <iomanip.h>
#include <conio.h>

int main()
{
    const int MAXN = 10;
    int a[MAXN][MAXN], b[MAXN][MAXN], c[MAXN][MAXN], n, m, i, j;
    cout << "Введите число строк и столбцов: -> ";
    cin >> n >> m;
    cout << "Матрица A (по строкам, через пробел)" << endl;
    for (i = 0; i < n; i++)
    {
        cout << "-> ";
        for (j = 0; j < m; j++) cin >> a[i][j];
    }
    cout << "Матрица B (по строкам, через пробел)" << endl;
    for (i = 0; i < n; i++)
    {
        cout << "-> ";
        for (j = 0; j < m; j++) cin >> b[i][j];
    }
    for (i = 0; i < n; i++)
        for (j = 0; j < m; j++)
            c[i][j] = a[i][j] + b[i][j];
    cout << "Сумма A + B:" << endl;
    for (i = 0; i < n; i++)
    {
        for (j = 0; j < m; j++) cout << setw(5) << c[i][j];
        cout << endl;
    }
    getch();
    return 0;
}
```

Пример работы программы

```
Введите число строк и столбцов: -> 2 3
Матрица A (по строкам, через пробел)
-> 1 2 3
-> 4 5 6
Матрица B (по строкам, через пробел)
-> 10 20 30
-> 40 50 60
Сумма A + B:
   11   22   33
   44   55   66
```

Задача 4В.2. №2. Матрица A[5][5] по формуле A[i][j] = (i + 10) * j

Объявить двумерный массив 5 × 5, заполнить по формуле (индексы от 1 до 5) и вывести в виде `A[i][j] = значение`. Чтобы индексы совпадали с формулой, циклы идут от 1 до 5, а в массиве используем `a[i-1][j-1]`.

Решение — Borland C++ 6.0

```cpp
#include <iostream.h>
#include <conio.h>

int main()
{
    int a[5][5], i, j;
    for (i = 1; i <= 5; i++)
        for (j = 1; j <= 5; j++)
            a[i - 1][j - 1] = (i + 10) * j;
    for (i = 1; i <= 5; i++)
    {
        for (j = 1; j <= 5; j++)
            cout << "A[" << i << "][" << j << "] = " << a[i - 1][j - 1] << "  ";
        cout << endl;
    }
    getch();
    return 0;
}
```

Результат выполнения

```
A[1][1] = 11  A[1][2] = 22  A[1][3] = 33  A[1][4] = 44  A[1][5] = 55
A[2][1] = 12  A[2][2] = 24  A[2][3] = 36  A[2][4] = 48  A[2][5] = 60
A[3][1] = 13  A[3][2] = 26  A[3][3] = 39  A[3][4] = 52  A[3][5] = 65
A[4][1] = 14  A[4][2] = 28  A[4][3] = 42  A[4][4] = 56  A[4][5] = 70
A[5][1] = 15  A[5][2] = 30  A[5][3] = 45  A[5][4] = 60  A[5][5] = 75
```

Задача 4В.3. №3. Среднее арифметическое элементов матрицы

Ввести по строкам двумерный массив и вычислить среднее арифметическое всех его элементов (сумма / `n*m`, вещественное деление).

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <conio.h>

int main()
{
    const int MAXN = 10;
    int a[MAXN][MAXN], n, m, i, j, s = 0;
    cout << "Введите число строк и столбцов: -> ";
    cin >> n >> m;
    for (i = 0; i < n; i++)
    {
        cout << "-> ";
        for (j = 0; j < m; j++)
        {
            cin >> a[i][j];
            s += a[i][j];
        }
    }
    cout << "Среднее арифметическое = " << (double)s / (n * m) << endl;
    getch();
    return 0;
}
```

Пример работы программы

```
Введите число строк и столбцов: -> 2 3
-> 1 2 3
-> 4 5 6
Среднее арифметическое = 3.5
```

Задача 4В.4. №4. Определитель матрицы второго порядка

Экран из задания: сначала два пояснения, затем ввод строк после приглашения `->`, затем результат с двумя знаками после запятой. Определитель: `a11*a22 - a12*a21`.

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <iomanip.h>
#include <conio.h>

int main()
{
    double a[2][2], det;
    int i, j;
    cout << "Введите матрицу второго порядка." << endl;
    cout << "После ввода элементов строки нажимайте <Enter>" << endl;
    for (i = 0; i < 2; i++)
    {
        cout << "-> ";
        for (j = 0; j < 2; j++)
            cin >> a[i][j];
    }
    det = a[0][0] * a[1][1] - a[0][1] * a[1][0];
    cout << "Определитель матрицы" << endl;
    cout << fixed << setprecision(2);
    cout << a[0][0] << " " << a[0][1] << endl;
    cout << a[1][0] << " " << a[1][1] << " Равен " << det << endl;
    getch();
    return 0;
}
```

Пример работы программы

```
Введите матрицу второго порядка.
После ввода элементов строки нажимайте <Enter>
-> 5 -7
-> 1 3
Определитель матрицы
5.00 -7.00
1.00 3.00 Равен 22.00
```

Задача 4В.5. №5. Суммы элементов по столбцам

Ввести по строкам двумерный массив, вычислить сумму по каждому столбцу: внешний цикл — по столбцам.

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <conio.h>

int main()
{
    const int MAXN = 10;
    int a[MAXN][MAXN], n, m, i, j, s;
    cout << "Введите число строк и столбцов: -> ";
    cin >> n >> m;
    for (i = 0; i < n; i++)
    {
        cout << "-> ";
        for (j = 0; j < m; j++) cin >> a[i][j];
    }
    for (j = 0; j < m; j++)
    {
        s = 0;
        for (i = 0; i < n; i++)
            s += a[i][j];
        cout << "Сумма столбца " << j + 1 << " = " << s << endl;
    }
    getch();
    return 0;
}
```

Пример работы программы

```
Введите число строк и столбцов: -> 2 3
-> 1 2 3
-> 4 5 6
Сумма столбца 1 = 5
Сумма столбца 2 = 7
Сумма столбца 3 = 9
```

Задача 4В.6. №6. Суммы элементов по строкам

То же, но внешний цикл — по строкам, сумма сбрасывается в начале каждой строки.

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <conio.h>

int main()
{
    const int MAXN = 10;
    int a[MAXN][MAXN], n, m, i, j, s;
    cout << "Введите число строк и столбцов: -> ";
    cin >> n >> m;
    for (i = 0; i < n; i++)
    {
        cout << "-> ";
        for (j = 0; j < m; j++) cin >> a[i][j];
    }
    for (i = 0; i < n; i++)
    {
        s = 0;
        for (j = 0; j < m; j++)
            s += a[i][j];
        cout << "Сумма строки " << i + 1 << " = " << s << endl;
    }
    getch();
    return 0;
}
```

Пример работы программы

```
Введите число строк и столбцов: -> 2 3
-> 1 2 3
-> 4 5 6
Сумма строки 1 = 6
Сумма строки 2 = 15
```

Задача 4В.7. №7. Строка квадратной матрицы с максимальной суммой

Определить номер строки квадратной матрицы, сумма элементов которой максимальна. Считаем сумму каждой строки и запоминаем лучшую.

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <conio.h>

int main()
{
    const int MAXN = 10;
    int a[MAXN][MAXN], n, i, j, s, best, ibest = 0;
    cout << "Введите размер квадратной матрицы: -> ";
    cin >> n;
    for (i = 0; i < n; i++)
    {
        cout << "-> ";
        for (j = 0; j < n; j++) cin >> a[i][j];
    }
    for (i = 0; i < n; i++)
    {
        s = 0;
        for (j = 0; j < n; j++) s += a[i][j];
        if (i == 0 || s > best)          // первую строку принимаем за лучшую
        {
            best = s;
            ibest = i;
        }
    }
    cout << "Номер строки с максимальной суммой: " << ibest + 1;
    cout << " (сумма " << best << ")" << endl;
    getch();
    return 0;
}
```

Пример работы программы

```
Введите размер квадратной матрицы: -> 3
-> 1 2 3
-> 9 8 7
-> 4 5 6
Номер строки с максимальной суммой: 2 (сумма 24)
```

Задача 4В.8. №8. Минимальный и максимальный элементы и их индексы

Найти минимум и максимум в двумерном массиве и их индексы (номера строки и столбца, с единицы).

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <conio.h>

int main()
{
    const int MAXN = 10;
    int a[MAXN][MAXN], n, m, i, j;
    int mn, mx, imn = 0, jmn = 0, imx = 0, jmx = 0;
    cout << "Введите число строк и столбцов: -> ";
    cin >> n >> m;
    for (i = 0; i < n; i++)
    {
        cout << "-> ";
        for (j = 0; j < m; j++) cin >> a[i][j];
    }
    mn = mx = a[0][0];
    for (i = 0; i < n; i++)
        for (j = 0; j < m; j++)
        {
            if (a[i][j] < mn) { mn = a[i][j]; imn = i; jmn = j; }
            if (a[i][j] > mx) { mx = a[i][j]; imx = i; jmx = j; }
        }
    cout << "Минимум " << mn << " в строке " << imn + 1;
    cout << ", столбце " << jmn + 1 << endl;
    cout << "Максимум " << mx << " в строке " << imx + 1;
    cout << ", столбце " << jmx + 1 << endl;
    getch();
    return 0;
}
```

Пример работы программы

```
Введите число строк и столбцов: -> 2 3
-> 4 -1 7
-> 9 0 3
Минимум -1 в строке 1, столбце 2
Максимум 9 в строке 2, столбце 1
```

Задача 4В.9. №9. Двумерный массив в одномерный

Переписать элементы двумерного массива построчно в одномерный. Счётчик `k` растёт на 1 при каждой записи.

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <conio.h>

int main()
{
    const int MAXN = 10;
    int a[MAXN][MAXN], b[MAXN * MAXN], n, m, i, j, k = 0;
    cout << "Введите число строк и столбцов: -> ";
    cin >> n >> m;
    for (i = 0; i < n; i++)
    {
        cout << "-> ";
        for (j = 0; j < m; j++) cin >> a[i][j];
    }
    for (i = 0; i < n; i++)
        for (j = 0; j < m; j++)
            b[k++] = a[i][j];               // записали и сдвинули счётчик
    cout << "Одномерный массив:";
    for (i = 0; i < k; i++) cout << " " << b[i];
    cout << endl;
    getch();
    return 0;
}
```

Пример работы программы

```
Введите число строк и столбцов: -> 2 3
-> 1 2 3
-> 4 5 6
Одномерный массив: 1 2 3 4 5 6
```

Задача 4В.10. №10. Среднее арифметическое без минимума и максимума

Для двумерного массива: (сумма всех − min − max) / (количество − 2). Для одномерного массива решение такое же (см. лекцию 4Б). Нужно минимум 3 элемента.

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <conio.h>

int main()
{
    const int MAXN = 10;
    int a[MAXN][MAXN], n, m, i, j, s = 0, mn, mx;
    cout << "Введите число строк и столбцов: -> ";
    cin >> n >> m;
    for (i = 0; i < n; i++)
    {
        cout << "-> ";
        for (j = 0; j < m; j++) cin >> a[i][j];
    }
    mn = mx = a[0][0];
    for (i = 0; i < n; i++)
        for (j = 0; j < m; j++)
        {
            s += a[i][j];
            if (a[i][j] < mn) mn = a[i][j];
            if (a[i][j] > mx) mx = a[i][j];
        }
    cout << "Среднее без min и max = " << (double)(s - mn - mx) / (n * m - 2) << endl;
    getch();
    return 0;
}
```

Пример работы программы

```
Введите число строк и столбцов: -> 2 3
-> 4 -1 7
-> 9 0 3
Среднее без min и max = 3.5
```

Задача 4В.11. №11. Умножение двух квадратных матриц

Квадратные матрицы одного размера `n`. `c[i][j] = a[i][0]*b[0][j] + a[i][1]*b[1][j] + ...`.

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <iomanip.h>
#include <conio.h>

int main()
{
    const int MAXN = 10;
    int a[MAXN][MAXN], b[MAXN][MAXN], c[MAXN][MAXN], n, i, j, k;
    cout << "Введите размер матриц: -> ";
    cin >> n;
    cout << "Матрица A" << endl;
    for (i = 0; i < n; i++)
    {
        cout << "-> ";
        for (j = 0; j < n; j++) cin >> a[i][j];
    }
    cout << "Матрица B" << endl;
    for (i = 0; i < n; i++)
    {
        cout << "-> ";
        for (j = 0; j < n; j++) cin >> b[i][j];
    }
    for (i = 0; i < n; i++)
        for (j = 0; j < n; j++)
        {
            c[i][j] = 0;                        // накопитель обнуляем!
            for (k = 0; k < n; k++)
                c[i][j] += a[i][k] * b[k][j];
        }
    cout << "Произведение A * B:" << endl;
    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++) cout << setw(5) << c[i][j];
        cout << endl;
    }
    getch();
    return 0;
}
```

Пример работы программы

```
Введите размер матриц: -> 2
Матрица A
-> 1 2
-> 3 4
Матрица B
-> 5 6
-> 7 8
Произведение A * B:
   19   22
   43   50
```

Задача 4В.12. №12. Магический квадрат

Проверить, является ли введённая квадратная матрица «магическим» квадратом: суммы по каждой строке, каждому столбцу и обеим диагоналям одинаковы. За эталон берём сумму главной диагонали и сравниваем с остальными.

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <conio.h>

int main()
{
    const int MAXN = 10;
    int a[MAXN][MAXN], n, i, j, target = 0, s, s2 = 0;
    bool magic = true;
    cout << "Введите размер квадратной матрицы: -> ";
    cin >> n;
    for (i = 0; i < n; i++)
    {
        cout << "-> ";
        for (j = 0; j < n; j++) cin >> a[i][j];
    }
    for (i = 0; i < n; i++)
    {
        target += a[i][i];                      // главная диагональ
        s2 += a[i][n - 1 - i];                  // побочная диагональ
    }
    if (s2 != target) magic = false;
    for (i = 0; i < n; i++)
    {
        s = 0;
        for (j = 0; j < n; j++) s += a[i][j];  // строка i
        if (s != target) magic = false;
        s = 0;
        for (j = 0; j < n; j++) s += a[j][i];  // столбец i
        if (s != target) magic = false;
    }
    if (magic)
        cout << "Это магический квадрат, сумма = " << target << endl;
    else
        cout << "Это не магический квадрат" << endl;
    getch();
    return 0;
}
```

Пример работы программы

```
Введите размер квадратной матрицы: -> 3
-> 2 9 4
-> 7 5 3
-> 6 1 8
Это магический квадрат, сумма = 15
```

Задача 4В.13. Дополнительно: изменить побочную диагональ и отсортировать строки

Квадратная матрица: заменить элементы побочной диагонали суммой элементов главной диагонали, затем отсортировать каждую строку по возрастанию (пузырьком).

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <conio.h>

int main()
{
    const int MAXN = 10;
    int a[MAXN][MAXN], n, i, j, p, t, d = 0;
    cout << "Введите размер квадратной матрицы: -> ";
    cin >> n;
    for (i = 0; i < n; i++)
    {
        cout << "-> ";
        for (j = 0; j < n; j++) cin >> a[i][j];
    }
    for (i = 0; i < n; i++)
        d += a[i][i];                              // сумма главной диагонали
    for (i = 0; i < n; i++)
        a[i][n - 1 - i] = d;                       // меняем побочную диагональ
    for (i = 0; i < n; i++)                        // сортировка каждой строки
        for (p = 0; p < n - 1; p++)
            for (j = 0; j < n - 1 - p; j++)
                if (a[i][j] > a[i][j + 1])
                {
                    t = a[i][j];  a[i][j] = a[i][j + 1];  a[i][j + 1] = t;
                }
    cout << "Результат:" << endl;
    for (i = 0; i < n; i++)
    {
        for (j = 0; j < n; j++) cout << a[i][j] << " ";
        cout << endl;
    }
    getch();
    return 0;
}
```

Пример работы программы

```
Введите размер квадратной матрицы: -> 3
-> 5 1 3
-> 9 4 2
-> 7 8 6
Результат:
1 5 15
2 9 15
6 8 15
```

4B.5 Тест по теме

Тест по теме — лекция 4B

1. Как по-другому называют массив, у которого два индекса?

а) матрица
б) динамический массив
в) разнородный массив
г) одномерный массив

2. Сколько элементов в массиве `int a[3][4];`?

а) 3
б) 4
в) 7
г) 12

3. Чему равно значение `a[1][0]`?

```
int a[2][2] = {{1, 2}, {3, 4}};
cout << a[1][0];
```

а) 4
б) 2
в) 3
г) 1

4. Какое условие выбирает элементы главной диагонали квадратной матрицы?

а) `i > j`
б) `i == j`
в) `i + j == n - 1`
г) `i < j`

5. Какое условие выбирает элементы побочной диагонали матрицы `n x n` (индексы с 0)?

а) `i == j`
б) `i - j == 1`
в) `i + j == n`
г) `i + j == n - 1`

6. Что вычисляет фрагмент?

```
int a[3][3] = {{1,2,3},{4,5,6},{7,8,9}}, s = 0;
for (int i = 0; i < 3; i++)
    for (int j = 0; j < 3; j++)
        if (j < i) s += a[i][j];
cout << s;
```

а) Сумму элементов главной диагонали (15)
б) Сумму элементов побочной диагонали (15)
в) Сумму элементов ниже главной диагонали (19)
г) Сумму элементов выше главной диагонали (11)

7. Что вычисляет фрагмент?

```
int a[3][3] = {{1,2,3},{4,5,6},{7,8,9}}, s = 0;
for (int i = 0; i < 3; i++)
    s += a[i][i];
cout << s;
```

а) Сумму элементов главной диагонали матрицы (15)
б) Сумму нечётных элементов массива
в) Сумму всех элементов матрицы
г) Сумму чётных элементов массива

8. Какое объявление допустимо?

а) `int d[2][] = {{1,2,3},{4,5,6}};`
б) `int d[][3] = {{1,2,3},{4,5,6}};`
в) `int d[3,2];`
г) `int d[][] = {{1,2,3},{4,5,6}};`

9. Сколько раз выполнится оператор в теле `for (i = 0; i < 3; i++) for (j = 0; j < 5; j++) { ... }`?

а) 15
б) 5
в) 3
г) 8

10. Как записать сумму элементов `i`-й строки матрицы, если `s` обнулена?

а) `for (j = 0; j < m; j++) s += a[j][i];`
б) `for (j = 0; j < m; j++) s = a[i][j];`
в) `for (j = 0; j < m; j++) s += a[i][j];`
г) `s += a[i][m];`

11. Что надо сделать перед суммированием очередной строки (или столбца)?

а) Отсортировать строку
б) Увеличить размер матрицы
в) Обнулить переменную суммы
г) Ничего: сумма накапливается сама

12. Как делается транспонирование квадратной матрицы на месте?

а) Все элементы умножаются на -1
б) Меняются местами первая и последняя строки
в) Меняются местами `a[i][j]` и `a[j][i]` только при `j > i`
г) Меняются местами `a[i][j]` и `a[j][i]` при любых `i`, `j`

13. Чем является матрица, у которой суммы чисел в каждой строке, каждом столбце и по диагоналям одинаковы?

а) Единичной матрицей
б) Симметричной матрицей
в) Нулевой матрицей
г) Магическим квадратом

14. Чему равен определитель матрицы 2 × 2 с элементами `5 -7 / 1 3`?

а) -22
б) 8
в) 22
г) 15

Ключи (проверяй себя только после того, как ответил сам): 1–а, 2–г, 3–в, 4–б, 5–г, 6–в, 7–а, 8–б, 9–а, 10–в, 11–в, 12–в, 13–г, 14–в

4В.6 Контрольные вопросы из программы (проверь себя устно)

1. Что такое претест- и посттест-цикл? Чем отличаются `while` и `do...while`? Когда какой применять? (4А)
2. Для чего нужны `break` и `continue`? (4А.3)
3. Что такое массив? Как объявляется и инициализируется одномерный массив? Как обращаться к элементам? (4Б)
4. Что такое двумерный массив? Как он объявляется и инициализируется? Как вложенные циклы обрабатывают двумерный массив? (4В)
5. Преимущества и ограничения массивов. (4Б.1, 4Б.3)

Лекция 5. Функции

Функция — это именованный кусок программы, который можно вызывать много раз с разными данными. До сих пор мы писали всё в `main()`; на РК нужно уметь разбивать задачу на функции: `int a() { return ...}`, `void f(...)`, передача параметров, массивы в функциях.

5.1 Зачем нужны функции

- **Повторное использование**: один раз написали — вызываем сколько угодно.
- **Структурирование**: большая программа делится на логические части (ввод, вычисление, вывод).
- **Читаемость и отладка**: проще проверить маленькую функцию, чем 300 строк подряд.

> **Запомни. Функция** — определённая последовательность операций и описаний, выполняющая некоторую последовательность действий и имеющая имя. Стандартные функции (`sqrt`, `strlen`...) лежат в **библиотечных файлах** (`.lib`), а их описания — в заголовочных (`.h`); свои функции пишет программист.

5.2 Определение функции и вызов

Код

```
тип_результата имя(тип параметр1, тип параметр2, ...)
{
    тело функции;
    return значение;
}
```

- **Тип результата** — какого типа значение вернёт функция (`int`, `double`, `char`, `bool`...). Если значения нет — `void` («пусто»).
- **Имя** — идентификатор.
- **Параметры** — данные, которые функция получает «на вход».
- `return выражение;` — вычисляет выражение, **возвращает его значение туда, откуда функция вызвана**, и **завершает** функцию. В `void`-функции пишут просто `return;` или вообще ничего.
- **Тело** функции заключено в фигурные скобки `{ }`.

Код

```
#include <iostream.h>

int sum(int a, int b)               // функция возвращает целое
{
    return a + b;
}

void hello()                        // ничего не возвращает и не принимает
{
    cout << "Привет из функции!" << endl;
}

int main()
{
    int x = sum(3, 4);              // вызов: значение попадает в x
    cout << x << " " << sum(10, 20) << endl;
    hello();
    return 0;
}
```

Результат выполнения

```
7 30
Привет из функции!
```

Правила вызова: имя функции и в скобках **аргументы** через запятую (для функции без параметров — пустые скобки `hello()`). Вызов функции, возвращающей значение, — это

выражение: его можно присвоить, вывести, использовать в расчёте.

> **Нюанс.** У функции с типом результата (`int`, `double`...) в **каждой** ветви должен быть `return`. Если вместо значения забыть `return`, получишь мусор или предупреждение. Только `void`-функция может обойтись без него. Выполнение любой программы начинается с функции `main()` — она сама тоже функция (возвращает `int`; в учебных примерах пишут `void main()` и `return;`).

Пример: максимум и минимум двух чисел

Код

```
int maxi(int x, int y)             // большее из двух целых
{
    if (x > y) return x;
    else return y;
}

int mini(int x, int y)             // меньшее из двух целых
{
    int t;
    if (x < y) t = x;
    else t = y;
    return t;
}
```

Как отвечать на вопросы вроде «что определяет функция»: если сравнение `x < y` и возвращается меньшее — функция находит **меньшее** из двух целых чисел; если `x > y` и возвращается больший — **большее**.

5.3 Объявление (прототип) и определение

Определение — полный текст функции с телом. **Объявление (прототип)** — только заголовок с точкой с запятой: `int sum(int a, int b);` (имена параметров можно опустить: `int sum(int, int);`). Прототип сообщает компилятору имя функции, число и типы параметров и тип результата — **прототип функции помогает компилятору определить число и тип передаваемых аргументов**.

Правило: функцию нужно описать **до** первого вызова. Два способа:

Способ 1. Определение функции стоит выше `main`.

Код

```
#include <iostream.h>

double sqr(double x)               // определение ДО main
{
    return x * x;
}

int main()
{
    cout << sqr(1.5) << endl;
    return 0;
}
```

Результат выполнения

```
2.25
```

Способ 2. Вверху — прототип, определение — ниже `main` (так принято в больших программах: `main` читается первым).

Код

```
#include <iostream.h>

double sqr(double x);              // прототип: заголовок и ;

int main()
{
    cout << sqr(1.5) << endl;
    return 0;
}

double sqr(double x)               // определение ПОСЛЕ main
{
    return x * x;
}
```

Результат выполнения

```
2.25
```

> **Нюанс.** Если определение ниже `main`, а прототипа нет — компилятор в точке вызова ещё не знает о функции и выдаёт ошибку («Function should have a prototype» / «Call to undefined function»).

5.4 Параметры и аргументы

- **Формальные параметры** — переменные в заголовке при определении (`int a, int b`). Это локальные переменные функции.
- **Фактические параметры (аргументы)** — значения при вызове (`sum(3, 4)`, `sum(x, y + 1)`). Могут быть константами, переменными, выражениями.
- Число и порядок аргументов должны совпадать с параметрами; тип приводится автоматически, если возможно (`sqr(2)` для `double` — число 2 станет 2.0).
- Функция без параметров: `int f()` или `int f(void)`.

5.5 Передача параметров по значению

По умолчанию функция получает **копию** значения. Изменение параметра внутри функции **не затрагивает** исходную переменную.

Код

```
#include <iostream.h>

int inc2(int vv)
{
    vv++;                          // увеличили копию
    return vv;
}

void swapBad(int a, int b)         // меняет копии, оригиналы не тронуты
{
    int t = a;  a = b;  b = t;
}

int main()
{
    int a, b = 5;
    a = inc2(b);
    cout << a << " " << b << endl;         // 6 5: b не изменилась
    int x = 1, y = 2;
    swapBad(x, y);
    cout << x << " " << y << endl;         // 1 2: обмена не произошло
    return 0;
}
```

Результат выполнения

```
6 5
1 2
```

В тесте: `int inc2(int vv) { vv++; return vv; }` и `a = inc2(b)` при `b = 5` даёт `a = 6`, а `b` остаётся 5.

5.6 Передача параметров по ссылке

Ссылка — второе имя той же переменной. Если параметр объявлен как `int &a`, функция работает с **самой переменной** из вызова, а не с копией. Значок `&` ставится в заголовке функции; при вызове пишем обычные имена.

Код

```
#include <iostream.h>

void swapOk(int &a, int &b)        // ссылки: меняем оригиналы
{
    int t = a;  a = b;  b = t;
}

void divide(int a, int b, int &q, int &r)   // функция «возвращает» два результата
{
    q = a / b;
    r = a % b;
}

int main()
{
    int x = 1, y = 2, q, r;
    swapOk(x, y);
    cout << x << " " << y << endl;         // 2 1
    divide(17, 5, q, r);
    cout << q << " " << r << endl;         // 3 2
    return 0;
}
```

Результат выполнения

```
2 1
3 2
```

	По значению	По ссылке
Заголовок	`void f(int a)`	`void f(int &a)`
Что получает функция	копию	саму переменную (псевдоним)
Изменение внутри функции	оригинал не меняется	оригинал **меняется**
Аргумент при вызове	любое выражение (`f(5)`, `f(x+1)`)	только переменная (`f(x)`)
Когда применять	функция только читает данные	нужно изменить аргумент или вернуть несколько результатов; большие данные

В стиле C меняют оригинал через указатели: `void swapP(int *a, int *b)` и вызов `swapP(&x, &y)` — внутри `*a`, `*b`.

5.7 Спецификатор const у параметров

`const` запрещает функции менять параметр: любая попытка изменить его — ошибка компиляции. Так защищают данные, передаваемые по ссылке или как массив.

Код

```
int maxElem(const int a[], int n);        // функция может читать a[], но не менять
void show(const int &x);                  // передаём без копирования, но менять нельзя
```

Если же функция **должна менять** исходный массив (сортировка, заполнение) — `const` не пишем. Таким образом, одна и та же заготовка решает две задачи: с `const` — «запретить изменение исходного массива», без `const` — «разрешить изменение».

5.8 Параметры со значениями по умолчанию

Параметру можно задать значение, которое подставится, если при вызове аргумента нет. Такие параметры идут **в конце** списка, а значение пишут один раз — **в прототипе** (если он есть).

Код

```
#include <iostream.h>

void line(int n = 10, char c = '*')    // линия из n символов c
{
    for (int i = 0; i < n; i++)
        cout << c;
    cout << endl;
}

int main()
{
    line();                // 10 звёздочек
    line(5);               // 5 звёздочек
    line(8, '-');          // 8 минусов
    return 0;
}
```

Результат выполнения

```
**********
*****
--------
```

> **Нюанс.** Нельзя пропустить параметр в середине: `line(, '-')` — ошибка. Нельзя ставить параметр с умолчанием **перед** параметром без умолчания: `void f(int a = 1, int b);` — ошибка.

5.9 Встроенные (inline) функции

`inline` — просьба компилятору подставить тело функции прямо в место вызова, чтобы не тратить время на вызов. Подходит для коротких функций. Определение inline-функции должно быть видно до использования.

Код

```
inline int sqr(int x) { return x * x; }    // вызов sqr(5) компилятор может заменить на 5 * 5
```

Решение, подставлять код или нет, принимает компилятор. На результат программы `inline` не влияет, только на скорость и размер.

5.10 Локальные и глобальные переменные. Область видимости

- **Локальная** переменная объявлена внутри функции (или блока `{ }`) и **известна только внутри неё**. В другой функции то же имя — другая переменная. Параметры — тоже локальные. Живёт, пока выполняется функция.
- **Глобальная** переменная объявлена **вне любой функции** (обычно в начале файла); видна во всех функциях ниже по тексту. Живёт всё время работы программы и по умолчанию равна 0.
- Если локальная имеет то же имя, что глобальная, она **скрывает** глобальную; к глобальной тогда можно обратиться через `::имя`.
- **Область видимости** — часть программы, где имя доступно. Блок `{ }` создаёт свою область.

Код

```
#include <iostream.h>

int total = 100;                // глобальная

int counter()
{
    static int c = 0;           // статическая локальная: хранит значение между вызовами
    c++;
    return c;
}

void f()
{
    int total = 5;              // локальная скрывает глобальную
    cout << total << " " << ::total << endl;
}

int main()
{
    f();                        // 5 100
    counter();  counter();
    cout << counter() << endl;  // 3
    return 0;
}
```

Результат выполнения

```
5 100
3
```

> **Запомни.** Старайся не пользоваться глобальными переменными для передачи данных между функциями — это скрытые связи и источник ошибок. Данные лучше передавать через параметры и возвращаемое значение.

5.11 Массивы как параметры функций

Массив в функцию передаётся **по адресу** (по сути, по ссылке): копии нет, функция работает с **исходным** массивом. Размер массива в одномерном случае не передаётся — его передают отдельным параметром.

Код

```
void fill(int a[], int n);          // то же самое: void fill(int *a, int n);
```

- Вызов: `fill(arr, 10)` — только имя массива, без скобок.
- Изменения массива внутри функции видны в вызывающей программе.
- Чтобы запретить изменения, пиши `const int a[]`.
- Внутри функции `sizeof(a)` даёт размер **указателя** (4 байта), а не массива: число элементов нужно передавать явно.
- **Двумерный** массив: размер **второго** измерения обязан быть константой в заголовке: `void show(int a[][N], int n)`.
- Функция не может «вернуть массив»; она заполняет массив-параметр.

Код

```
#include <iostream.h>

const int N = 5;

void fill(int a[], int n, int start)        // заполняет: изменяет оригинал
{
    for (int i = 0; i < n; i++)
        a[i] = start + i * 2;
}

void show(const int a[], int n)             // только читает
{
    for (int i = 0; i < n; i++)
        cout << a[i] << " ";
    cout << endl;
}

int maxElem(const int a[], int n)
{
    int m = a[0];
    for (int i = 1; i < n; i++)
        if (a[i] > m) m = a[i];
    return m;
}

int main()
{
    int arr[N];
    fill(arr, N, 3);
    show(arr, N);                            // 3 5 7 9 11
    cout << maxElem(arr, N) << endl;         // 11
    return 0;
}
```

Результат выполнения

```
3 5 7 9 11
11
```

5.12 Перегрузка функций

В C++ можно создать несколько функций с **одним именем**, но с **разными списками параметров** (разное число или разные типы). Компилятор выбирает нужную по аргументам вызова. Различие только в типе результата — **не** перегрузка (ошибка).

Код

```
#include <iostream.h>

int sq(int x)       { return x * x; }
double sq(double x) { return x * x; }
int sum(int a, int b)        { return a + b; }
int sum(int a, int b, int c) { return a + b + c; }

int main()
{
    cout << sq(3) << " " << sq(1.5) << endl;      // 9 2.25
    cout << sum(1, 2) << " " << sum(1, 2, 3) << endl;     // 3 6
    return 0;
}
```

Результат выполнения

```
9 2.25
3 6
```

5.13 Рекурсия

Рекурсия — функция, которая **вызывает саму себя**. У рекурсивного решения всегда есть: **базовый случай** (когда вызовов больше нет) и **шаг рекурсии** (сведение задачи к такой же задаче меньшего размера). Без базового случая — бесконечные вызовы и переполнение стека.

Виды: **прямая** (функция вызывает себя) и **косвенная** (`f` вызывает `g`, а `g` вызывает `f`).

Код

```
#include <iostream.h>

long fact(int n)                  // n! = n * (n-1)!
{
    if (n <= 1) return 1;         // базовый случай
    return n * fact(n - 1);       // шаг рекурсии
}

int sumDigits(int n)              // сумма цифр
{
    if (n == 0) return 0;
    return n % 10 + sumDigits(n / 10);
}

int fib(int n)                    // Фибоначчи: 1 1 2 3 5 8 ...
{
    if (n <= 2) return 1;
    return fib(n - 1) + fib(n - 2);
}

int main()
{
    cout << fact(5) << " " << sumDigits(4821) << " " << fib(6) << endl;
    return 0;
}
```

Результат выполнения

```
120 15 8
```

Рекурсия красива, но дороже цикла по времени и памяти (каждый вызов занимает место в стеке), а `fib` в таком виде вызывается экспоненциально много раз. Для простых задач выбирай цикл.

5.14 Задачи

Задача 5.1. Максимум и минимум через функции

Написать функции `maxi(a, b)` и `mini(a, b)`; с их помощью найти наибольшее и наименьшее из трёх чисел.

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <conio.h>

int maxi(int x, int y)
{
    if (x > y) return x;
    else return y;
}

int mini(int x, int y)
{
    return x < y ? x : y;
}

int main()
{
    int a, b, c;
    cout << "Введите три числа: -> ";
    cin >> a >> b >> c;
    cout << "Максимум = " << maxi(maxi(a, b), c) << endl;
    cout << "Минимум = " << mini(mini(a, b), c) << endl;
    getch();
    return 0;
}
```

Пример работы программы

```
Введите три числа: -> 8 3 5
Максимум = 8
Минимум = 3
```

Задача 5.2. Простые числа: функция isPrime

Функция `bool isPrime(int n)` возвращает `true`, если число простое. С её помощью вывести все простые числа от `a` до `b`.

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <conio.h>

bool isPrime(int n)
{
    if (n < 2) return false;
    for (int d = 2; d * d <= n; d++)
        if (n % d == 0) return false;
    return true;
}

int main()
{
    int a, b;
    cout << "Введите границы a и b: -> ";
    cin >> a >> b;
    for (int i = a; i <= b; i++)
        if (isPrime(i))
            cout << i << " ";
    cout << endl;
    getch();
    return 0;
}
```

Пример работы программы

```
Введите границы a и b: -> 10 40
11 13 17 19 23 29 31 37
```

Задача 5.3. Обмен: по значению и по ссылке

Сравнить две функции обмена и показать, какая из них реально меняет переменные.

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <conio.h>

void swapVal(int a, int b)         // по значению: не работает
{
    int t = a;  a = b;  b = t;
}

void swapRef(int &a, int &b)       // по ссылке: работает
{
    int t = a;  a = b;  b = t;
}

int main()
{
    int x, y;
    cout << "Введите x и y: -> ";
    cin >> x >> y;
    swapVal(x, y);
    cout << "После swapVal: " << x << " " << y << endl;
    swapRef(x, y);
    cout << "После swapRef: " << x << " " << y << endl;
    getch();
    return 0;
}
```

Пример работы программы

```
Введите x и y: -> 4 9
После swapVal: 4 9
После swapRef: 9 4
```

Задача 5.4. Обработка массива через функции

Ввод, вывод, поиск максимума (без права изменения массива) и умножение всех элементов на `k` (с изменением оригинала) — каждое отдельной функцией.

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <conio.h>

const int MAXN = 100;

void inputArr(int a[], int n)                  // ввод: изменяет массив
{
    for (int i = 0; i < n; i++)
    {
        cout << "Введите число " << i + 1 << ": -> ";
        cin >> a[i];
    }
}

void outputArr(const int a[], int n)           // вывод: менять нельзя
{
    for (int i = 0; i < n; i++)
    {
        cout << a[i];
        if (i < n - 1) cout << ", ";
    }
    cout << endl;
}

int maxArr(const int a[], int n)               // поиск максимума: менять нельзя
{
    int m = a[0];
    for (int i = 1; i < n; i++)
        if (a[i] > m) m = a[i];
    return m;
}

void scale(int a[], int n, int k)              // умножение: изменяет оригинал
{
    for (int i = 0; i < n; i++)
        a[i] *= k;
}

int main()
{
    int arr[MAXN], n, k;
    cout << "Введите число элементов массива: -> ";
    cin >> n;
    inputArr(arr, n);
    cout << "Максимум = " << maxArr(arr, n) << endl;
    cout << "Введите множитель k: -> ";
    cin >> k;
    scale(arr, n, k);
    outputArr(arr, n);
    getch();
    return 0;
}
```

Пример работы программы

```
Введите число элементов массива: -> 4
Введите число 1: -> 3
Введите число 2: -> -1
Введите число 3: -> 7
Введите число 4: -> 2
Максимум = 7
Введите множитель k: -> 10
30, -10, 70, 20
```

Задача 5.5. Несколько результатов через ссылки

Функция `stats` получает массив и «возвращает» минимум, максимум и среднее — через три параметра-ссылки.

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <conio.h>

void stats(const int a[], int n, int &mn, int &mx, double &avg)
{
    int s = 0;
    mn = mx = a[0];
    for (int i = 0; i < n; i++)
    {
        s += a[i];
        if (a[i] < mn) mn = a[i];
        if (a[i] > mx) mx = a[i];
    }
    avg = (double)s / n;
}

int main()
{
    int a[100], n, mn, mx;
    double avg;
    cout << "Введите число элементов массива: -> ";
    cin >> n;
    for (int i = 0; i < n; i++)
    {
        cout << "Введите число " << i + 1 << ": -> ";
        cin >> a[i];
    }
    stats(a, n, mn, mx, avg);
    cout << "min = " << mn << ", max = " << mx << ", среднее = " << avg << endl;
    getch();
    return 0;
}
```

Пример работы программы

```
Введите число элементов массива: -> 5
Введите число 1: -> 4
Введите число 2: -> 9
Введите число 3: -> 1
Введите число 4: -> 7
Введите число 5: -> 6
min = 1, max = 9, среднее = 5.4
```

Задача 5.6. Рекурсия: степень, сумма и факториал

Рекурсивные функции: `power(x, n)` (степень), `fact(n)` и `sumTo(n)` (сумма 1..n). У каждой — базовый случай.

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <conio.h>

double power(double x, int n)
{
    if (n == 0) return 1;
    return x * power(x, n - 1);
}

long fact(int n)
{
    if (n <= 1) return 1;
    return n * fact(n - 1);
}

int sumTo(int n)
{
    if (n == 0) return 0;
    return n + sumTo(n - 1);
}

int main()
{
    double x;
    int n, m;
    cout << "Введите x и n (степень): -> ";
    cin >> x >> n;
    cout << x << " в степени " << n << " = " << power(x, n) << endl;
    cout << "Введите m: -> ";
    cin >> m;
    cout << m << "! = " << fact(m) << ", сумма 1.." << m << " = " << sumTo(m) << endl;
    getch();
    return 0;
}
```

Пример работы программы

```
Введите x и n (степень): -> 2 10
2 в степени 10 = 1024
Введите m: -> 6
6! = 720, сумма 1..6 = 21
```

Задача 5.7. Перегрузка и параметры по умолчанию

`area` с одним параметром — площадь круга, с двумя — площадь прямоугольника. Функция `line` рисует линию, параметры по умолчанию.

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <conio.h>

double area(double r)                    // круг
{
    return 3.14159 * r * r;
}

double area(double a, double b)          // прямоугольник
{
    return a * b;
}

void line(int n = 15, char c = '=')
{
    for (int i = 0; i < n; i++) cout << c;
    cout << endl;
}

int main()
{
    line();
    cout << "Круг R=2: " << area(2.0) << endl;
    cout << "Прямоугольник 3x4: " << area(3.0, 4.0) << endl;
    line(6, '-');
    getch();
    return 0;
}
```

Результат выполнения

```
===============
Круг R=2: 12.5664
Прямоугольник 3x4: 12
------
```

Задача 5.8. Матрица в функции: сумма главной диагонали и вывод

Функции с двумерным массивом: второе измерение — константа `N`.

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <iomanip.h>
#include <conio.h>

const int N = 3;

int diagSum(const int a[][N], int n)
{
    int s = 0;
    for (int i = 0; i < n; i++)
        s += a[i][i];
    return s;
}

void showMat(const int a[][N], int n)
{
    for (int i = 0; i < n; i++)
    {
        for (int j = 0; j < n; j++)
            cout << setw(4) << a[i][j];
        cout << endl;
    }
}

int main()
{
    int m[N][N] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
    showMat(m, N);
    cout << "Сумма главной диагонали = " << diagSum(m, N) << endl;
    getch();
    return 0;
}
```

Результат выполнения

```
   1   2   3
   4   5   6
   7   8   9
Сумма главной диагонали = 15
```

5.15 Тест по теме

Тест по теме — лекция 5

1. Как называется определённая последовательность операций и описаний, выполняющая некоторую последовательность действий?

а) константа
б) рекурсия
в) функция
г) комментарий

2. Как называются параметры в заголовке при описании функции?

а) формальные
б) глобальные
в) рекурсивные
г) фактические

3. Что делает оператор `return выражение;` в функции?

а) Выводит значение выражения на экран
б) Запускает функцию заново
в) Объявляет переменную
г) Возвращает значение выражения туда, откуда функция вызвана, и завершает функцию

4. Для чего в заголовке функции используется слово `void`?

а) Показывает, что у функции нет имени
б) Показывает, что функция рекурсивная
в) Показывает, что функция возвращает целое число
г) Показывает, что функция не возвращает значения

5. Что помогает компилятору определить число и тип аргументов, передаваемых в функцию?

а) прототип функции
б) комментарий
в) локальная переменная
г) идентификатор функции

6. Как называется функция, вызывающая саму себя?

а) рекурсия
б) процедура
в) оператор
г) константа

7. Как называется переменная, известная только внутри функции, где она описана?

а) рекурсивная
б) локальная
в) внешняя
г) глобальная

8. Как называется переменная, объявленная вне любых блоков и функций?

а) формальная
б) глобальная
в) унарная
г) локальная

9. Что делает функция?

```
int m(int x, int y)
{
    int t;
    if (x < y) t = x;
    else t = y;
    return t;
}
```

а) Проверяет равенство двух чисел
б) Определяет большее из двух целых чисел
в) Определяет меньшее из двух вещественных чисел
г) Определяет меньшее из двух целых чисел

10. Что делает функция?

```
int m(int x, int y)
{
    if (x > y) return x;
    else return y;
}
```

а) Определяет меньшее из двух целых чисел
б) Определяет сумму двух чисел
в) Определяет большее из двух целых чисел
г) Проверяет равенство двух чисел

11. Что выведет программа?

```
#include <iostream.h>
int inc2(int vv) { vv++; return vv; }
int main()
{
    int a, b = 5;
    a = inc2(b);
    cout << a << b;
    return 0;
}
```

а) 65
б) 6
в) 66
г) 55

12. Что произойдёт после вызова `swap(x, y)`, если функция объявлена `void swap(int a, int b)` и внутри меняет `a` и `b` местами?

а) `x` и `y` поменяются местами
б) Ничего: переменные `x` и `y` не изменятся (передача по значению)
в) Ошибка компиляции
г) `x` и `y` станут равны нулю

13. Что нужно изменить в заголовке, чтобы обмен реально менял переменные?

а) Объявить параметры ссылками: `void swap(int &a, int &b)`
б) Заменить `void` на `int`
в) Переименовать функцию
г) Добавить `const` перед параметрами

14. Что выведет программа?

```
#include <iostream.h>
void f(int n = 3, char c = '#')
{
    for (int i = 0; i < n; i++) cout << c;
}
int main()
{
    f(); f(2); f(1, '-');
    return 0;
}
```

а) `######-`
б) `###-`
в) `##-`
г) `#####-`

15. Что произойдёт, если внутри функции с параметром `const int a[]` попытаться присвоить `a[0] = 1;`?

а) Ошибка компиляции: менять константный параметр нельзя
б) Программа выведет предупреждение и продолжит
в) Изменится только копия массива
г) Первый элемент массива станет 1

16. Как массив передаётся в функцию?

а) Нельзя передавать массивы
б) Копируется целиком
в) Всегда как константа
г) По адресу: функция работает с исходным массивом

17. Что напечатает программа?

```
#include <iostream.h>
int f(int n)
{
    if (n <= 1) return 1;
    return n * f(n - 1);
}
int main()
{
    cout << f(5);
    return 0;
}
```

а) 120
б) 24
в) 5
г) 15

18. Что напечатает программа?

```
#include <iostream.h>
int c()
{
    static int k = 0;
    k++;
    return k;
}
int main()
{
    c(); c();
    cout << c();
    return 0;
}
```

а) 2
б) 1
в) 3
г) 0

19. Какие две функции допустимы в одной программе (перегрузка)?

а) `int f(int a);` и `double f(int a);`
б) `void f();` и `void f();`
в) `int f(int a);` и `int f(int b);`
г) `int f(int a);` и `int f(int a, int b);`

20. Какое объявление с параметрами по умолчанию допустимо?

а) `void f(int a = 5; int b = 6);`
б) `void f(int a = 5, int b);`
в) `void f(a = 5, b = 6);`
г) `void f(int a, int b = 5);`

21. Что будет, если у рекурсивной функции нет базового случая?

а) Бесконечные вызовы и переполнение стека
б) Функция выполнится один раз
в) Компилятор исправит ошибку сам
г) Результат всегда 0

Ключи (проверяй себя только после того, как ответил сам): 1–в, 2–а, 3–г, 4–г, 5–а, 6–а, 7–б, 8–б, 9–г, 10–в, 11–а, 12–б, 13–а, 14–г, 15–а, 16–г, 17–а, 18–в, 19–г, 20–г, 21–а

5.16 Контрольные вопросы из программы (проверь себя устно)

1. Что такое функция и зачем она нужна? (5.1)
2. Чем отличается объявление функции от определения? Что такое прототип? (5.3)
3. Что такое параметры и аргументы? Как вызывается функция? (5.2, 5.4)

4. Для чего нужен `return`? (5.2)
5. Что значит передача параметра по значению и по ссылке? В чём разница? Когда по ссылке? (5.5, 5.6)
6. Что такое ссылка в C++? Как изменить исходную переменную через параметр-ссылку? (5.6)
7. Зачем `const` в параметрах? Параметры по умолчанию, `inline`. (5.7–5.9)
8. Как передать массив в функцию? (5.11)
9. Локальные и глобальные переменные, область видимости. (5.10)
10. Перегрузка и рекурсия. (5.12, 5.13)

Лекция 6. Тип char и строки

Строка в C++ — это массив символов, заканчивающийся специальным нулевым символом. Эта простая мысль объясняет почти все «странные» вопросы про строки: почему нельзя сравнить строки через `==`, почему нужно `strcpy`, почему размер массива на единицу больше длины строки. Есть и второй путь — класс `string`, он удобнее, но на РК спрашивают оба.

6.1 Тип char

`char` — тип размером **1 байт**, хранящий **код символа**. Символ и число — это одно и то же: `'A'` и `65` лежат в памяти одинаково. Что именно получится на экране, зависит от того, как мы выводим.

- Символьная константа пишется в **одинарных** кавычках: `'A'`, `'7'`, `'\n'`.
- Диапазон `char`: от **-128 до 127**; `unsigned char`: от 0 до 255. Русские буквы в Windows-кодировке занимают коды 192–255, то есть для обычного `char` они отрицательные.
- Таблица кодов ASCII: пробел — 32; цифры `'0'`…`'9'` — коды 48…57; заглавные `'A'`…`'Z'` — 65…90; строчные `'a'`…`'z'` — 97…122. Разница между строчной и заглавной буквой — 32.

Фрагмент

```
char c = 'A';
cout << c << " " << (int)c << " " << c + 1 << " " << (char)(c + 1) << endl;
cout << (int)'0' << " " << '7' - '0' << endl;
```

Результат выполнения

```
A 65 66 B
48 7
```

Запомни: операции над `char` выполняются как над числами (`c + 1` — это `int`), для вывода символа нужно привести обратно `(char)(c + 1)`. Цифру-символ превращают в число вычитанием `'0'`: `'7' - '0'` равно 7.

Работа с символами

Фрагмент

```
char c = 'g', d = '5';
if (c >= 'a' && c <= 'z')
    c = c - 32;                          // в верхний регистр вручную (разница 32)
cout << c << " ";
cout << (char)toupper('x') << (char)tolower('Q') << " ";    // из <ctype.h>
cout << (isdigit(d) ? "цифра" : "не цифра") << " ";
cout << (isalpha(d) ? "буква" : "не буква") << endl;
```

Результат выполнения

```
G Xq цифра не буква
```

Функции из `<ctype.h>`: `isdigit(c)` — цифра ли; `isalpha(c)` — латинская буква ли; `isupper`, `islower`, `isspace`; `toupper(c)`, `tolower(c)` — смена регистра.

Ввод символа

- `cin >> c;` — читает один символ, **пропуская пробелы** и переводы строк.
- `cin.get(c);` — читает ровно один символ, **включая пробел**.
- `getch();` — ждёт нажатия клавиши и возвращает её код, на экране символ не показывается.

> **Нюанс.** `'A'` и `"A"` — **разные вещи**. `'A'` — один символ (1 байт). `"A"` — строка из двух байт: символ `A` и завершающий `\0`. Присвоить `char c = "A";` нельзя.

Перенос строки и другие управляющие символы — escape-последовательности (см. лекцию 1): `\n`, `\t`, `\0`, `\\`, `\'`, `\"`.

6.2 Строки как массивы символов

Строка в стиле C — это массив `char`, в конце которого находится **нулевой символ** `'\0'` (код 0). Именно по нему функции определяют конец строки.

Схема

```
char s[10] = "Hello";

  индекс:   0    1    2    3    4    5    6    7    8    9
  значение: 'H'  'e'  'l'  'l'  'o'  \0   \0   \0   \0   \0
                                    ^
                                    конец строки (strlen(s) = 5, sizeof(s) = 10)
```

Длина строки (`strlen`) — число символов **до** `\0`. Размер массива (`sizeof`) — число байт, выделенных под него. Для строки из `k` символов нужен массив минимум из `k + 1` элементов.

Объявление и инициализация

Фрагмент

```
char s1[10] = "Hello";              // остальные элементы заполняются нулями
char s2[] = "Hello";                // размер выбирается сам: 6 (5 букв + \0)
char s3[] = {'H', 'i', '\0'};       // посимвольно: \0 нужно добавлять вручную
char s4[10] = "";                   // пустая строка
cout << s1 << " " << sizeof(s2) << " " << strlen(s2) << " " << s3 << "|" << s4 << "|" << endl;
```

Результат выполнения

```
Hello 6 5 Hi||
```

> **Нюанс.** `char s[5] = "Hello";` — **ошибка**: для пяти букв и `\0` нужно 6 элементов. Если `\0` в массиве нет, `cout << s` не знает, где остановиться, и выводит «мусор» из памяти до первого случайного нуля.

Строки нельзя **присваивать** (`s1 = s2;`) и **сравнивать** (`s1 == s2`) операциями: массив — это адрес, поэтому `==` сравнивает адреса, а не содержимое. Для этого есть функции из `<string.h>`.

6.3 Ввод и вывод строк

Способ	Особенности
`cout << s;`	вывод до `\0`
`cin >> s;`	читает **одно слово**: до пробела или Enter
`cin.getline(s, N);`	читает **всю строку** с пробелами, не более `N-1` символов
`gets(s);` / `puts(s);`	из `<stdio.h>`: строка целиком; `gets` не ограничивает длину (опасно)
`scanf("%s", s);` / `printf("%s", s);`	в стиле C; `scanf` тоже читает одно слово, а для строки перед именем `&` **не** ставится

Код

```
#include <iostream.h>
#include <conio.h>

int main()
{
    char s[50];
    cout << "Введите строку: -> ";
    cin.getline(s, 50);
    cout << "Вы ввели: " << s << endl;
    getch();
    return 0;
}
```

Пример работы программы

```
Введите строку: -> Hello big world
Вы ввели: Hello big world
```

Если ввести `Hello big world` через `cin >> s;`, в `s` окажется только `Hello`.

> **Нюанс. Ловушка:** после `cin >> n;` в буфере остаётся символ перевода строки. Следующий `cin.getline(...)` прочитает пустую строку. Перед ним ставят `cin.ignore();` (пропустить один символ) или `cin.get();`.

Код

```
int n;
char s[50];
cin >> n;
cin.ignore();               // убрали оставшийся Enter
cin.getline(s, 50);         // теперь читается нормальная строка
```

6.4 Доступ к символам и перебор строки

К символу обращаются как к элементу массива: `s[0]` — первый, `s[strlen(s) - 1]` — последний. Строку перебирают, пока не встретится `\0`.

Фрагмент

```
char s[] = "Programming";
int i = 0, cnt = 0;
while (s[i] != '\0')               // можно короче: while (s[i])
{
    if (s[i] == 'm') cnt++;
    i++;
}
cout << "Длина " << i << ", букв m: " << cnt << endl;
s[0] = 'p';                        // менять символы можно
cout << s << endl;
```

Результат выполнения

```
Длина 11, букв m: 2
programming
```

6.5 Библиотека `<string.h>`

Подключается через `#include <string.h>` — это библиотека **строковых функций**.

Функция	Что делает
`strlen(s)`	длина строки (без `\0`)
`strcpy(s1, s2)`	копирует строку `s2` в `s1` (вместе с `\0`) — «присваивание» строк
`strncpy(s1, s2, k)`	копирует первые `k` символов `s2` в `s1`
`strcat(s1, s2)`	дописывает `s2` в конец `s1` (результат в `s1`) — конкатенация
`strncat(s1, s2, k)`	дописывает к `s1` первые `k` символов `s2`
`strcmp(s1, s2)`	сравнение: **0**, если строки равны; меньше 0, если `s1` меньше; больше 0, если `s1` больше
`strncmp(s1, s2, k)`	сравнивает только первые `k` символов
`stricmp(s1, s2)`	как `strcmp`, но **без учёта регистра** (Borland)
`strchr(s, c)`	ищет символ `c` **слева направо**; возвращает указатель на него или `NULL`
`strrchr(s, c)`	ищет символ `c` **с конца к началу**
`strstr(s1, s2)`	ищет подстроку `s2` внутри `s1`; возвращает указатель или `NULL`
`strupr(s)`	переводит символы в верхний регистр (Borland)
`strlwr(s)`	переводит символы в нижний регистр (Borland)
`strrev(s)`	разворачивает строку (Borland)
`strnset(s, c, k)`	заполняет `k` символов строки символом `c` (Borland)

Фрагмент

```
char a[40] = "Hello", b[40], c[40] = "world";
strcpy(b, a);                                  // b = "Hello"
strcat(b, ", ");                               // b = "Hello, "
strcat(b, c);                                  // b = "Hello, world"
cout << b << " " << strlen(b) << endl;
cout << strcmp("Salem", "Salem") << " " << (strcmp("abc", "abd") < 0) << " ";
cout << (strcmp("b", "a") > 0) << endl;
char *p = strstr(b, "wor");                    // указатель на подстроку или NULL
if (p != NULL)
    cout << "найдено с позиции " << p - b << ": " << p << endl;
cout << strchr(b, 'l') - b << " " << strrchr(b, 'l') - b << endl;   // первая и последняя l
strupr(b);
cout << b << endl;
strrev(b);
cout << b << endl;
```

Результат выполнения

```
Hello, world 12
0 1 1
найдено с позиции 7: world
2 10
HELLO, WORLD
DLROW ,OLLEH
```

Нюанс. `strcmp(s1, s2) == 0` — **строки равны** (ноль означает «нет разницы»). `strcmp("Salem", "Salem")` в тесте даёт 0. Для различающихся строк конкретное число зависит от компилятора — проверяй только знак.

Нюанс. `strcpy(s1, s2)` не проверяет размер: если `s1` короче `s2` вместе с `\0`, программа затрёт чужую память. Размеры массивов выбирай с запасом.

6.6 Класс string

В C++ есть готовый тип `string`, который сам управляет памятью, поддерживает `=`, `+`, `==`. Подключение: `#include <string>` и `using namespace std;`.

Нюанс. Для программ со `string` в Borland C++ 6.0 надёжнее подключать **новую** библиотеку ввода-вывода: `#include <iostream>`, `#include <string>`, `using namespace std;` (а не смесь `<iostream.h>` и `<string>`).

Операция	Запись
объявление	`string s; string t = "abc";`
присваивание, копия	`s = t;`
склейка	`s = s + t; s += "xyz";`
сравнение	`s == t`, `s != t`, `s < t` (по алфавиту)
длина	`s.length()` или `s.size()`
символ	`s[i]`
подстрока	`s.substr(pos, len)`
поиск	`s.find("ab")` — позиция или `string::npos`, если нет
вставка, удаление	`s.insert(pos, "text")`, `s.erase(pos, len)`
пустая ли	`s.empty()`
ввод	`cin >> s;` (слово), `getline(cin, s);` (вся строка)
в `char*`	`s.c_str()`

Код

```cpp
#include <iostream>
#include <string>
using namespace std;

int main()
{
    string a = "Hello", b = "world", s;
    s = a + ", " + b;                          // склейка: операция + работает
    cout << s << " " << s.length() << endl;
    if (a != b)
        cout << "строки разные" << endl;
    cout << s.substr(7, 5) << " " << s.find("wor") << endl;     // world 7
    s.erase(5, 7);                             // убрали ", world"
    s.insert(5, "!!!");
    cout << s << " " << s[0] << endl;
    int p = b.find("zzz");
    if (p == string::npos)
        cout << "не найдено" << endl;
    return 0;
}
```

Результат выполнения

```
Hello, world 12
строки разные
world 7
Hello!!! H
не найдено
```

	`char`-массив	`string`
Размер	фиксирован при объявлении	растёт сам
Присваивание	`strcpy(a, b)`	`a = b`
Склейка	`strcat(a, b)`	`a + b` или `a += b`
Сравнение	`strcmp(a, b)`	`a == b`, `a < b`
Длина	`strlen(a)`	`a.length()`
Ввод строки с пробелами	`cin.getline(a, N)`	`getline(cin, a)`
Подходит для	учебных задач на «руками», работы с функциями C	повседневной работы

6.7 Типичные ошибки

- Нет места под `\0`: `char s[5] = "Hello";`.
- Сравнение/присваивание строк через `==` и `=` вместо `strcmp`/`strcpy`.
- Путаница `'a'` и `"a"`.
- `cin >> s` для ввода строки с пробелами.
- Забытый `cin.ignore()` между `cin >> n` и `cin.getline`.
- Выход за границы массива при `strcpy`, `strcat`, `cin >> s`.
- Отсутствие `#include <string.h>` при использовании `strlen`, `strcpy`, ...

6.8 Задачи

Задача 6.1. Длина, гласные и цифры

Ввести строку (с пробелами). Определить её длину, количество гласных латинских букв (`a e i o u`, регистр не важен) и количество цифр. Перебираем символы до `\0`.

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <string.h>
#include <ctype.h>
#include <conio.h>

int main()
{
    char s[100];
    int i, vowels = 0, digits = 0;
    cout << "Введите строку: -> ";
    cin.getline(s, 100);
    for (i = 0; s[i] != '\0'; i++)
    {
        char c = tolower(s[i]);
        if (c == 'a' || c == 'e' || c == 'i' || c == 'o' || c == 'u')
            vowels++;
        if (isdigit(s[i]))
            digits++;
    }
    cout << "Длина = " << strlen(s) << endl;
    cout << "Гласных = " << vowels << ", цифр = " << digits << endl;
    getch();
    return 0;
}
```

Пример работы программы

```
Введите строку: -> Hello World 2026
Длина = 16
Гласных = 3, цифр = 4
```

Задача 6.2. Палиндром

Слово — палиндром, если читается одинаково в обе стороны. Сравниваем `s[i]` и `s[n-1-i]` в первой половине. Альтернатива — скопировать строку, развернуть `strrev` и сравнить `strcmp`.

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <string.h>
#include <conio.h>

int main()
{
    char s[100];
    int n, i;
    bool pal = true;
    cout << "Введите слово: -> ";
    cin >> s;
    n = strlen(s);
    for (i = 0; i < n / 2; i++)
        if (s[i] != s[n - 1 - i])
        {
            pal = false;
            break;
        }
    cout << (pal ? "Палиндром" : "Не палиндром") << endl;
    getch();
    return 0;
}
```

Пример работы программы

```
Введите слово: -> level
Палиндром
```

Задача 6.3. Слова в строке, удаление пробелов, верхний регистр

Ввести предложение. Подсчитать количество слов (слово начинается там, где пробел сменяется не-пробелом), убрать пробелы и перевести всё в верхний регистр.

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <string.h>
#include <ctype.h>
#include <conio.h>

int main()
{
    char s[100], t[100];
    int i, k = 0, words = 0;
    cout << "Введите предложение: -> ";
    cin.getline(s, 100);
    for (i = 0; s[i] != '\0'; i++)
    {
        if (s[i] != ' ' && (i == 0 || s[i - 1] == ' '))
            words++;                        // начало нового слова
        if (s[i] != ' ')
        {
            t[k] = toupper(s[i]);           // копируем без пробелов, в верхнем регистре
            k++;
        }
    }
    t[k] = '\0';                            // не забыть завершить строку!
    cout << "Слов: " << words << endl;
    cout << "Без пробелов: " << t << endl;
    getch();
    return 0;
}
```

Пример работы программы

```
Введите предложение: -> the quick  brown fox
Слов: 4
Без пробелов: THEQUICKBROWNFOX
```

Задача 6.4. Функции из string.h

Ввести имя и фамилию (два слова). Составить строку «Фамилия Имя», вывести её длину, сравнить имя и фамилию по алфавиту (`strcmp`) и найти позицию буквы `a` (`strchr`).

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <string.h>
#include <conio.h>

int main()
{
    char name[30], surname[30], full[70];
    cout << "Введите имя: -> ";
    cin >> name;
    cout << "Введите фамилию: -> ";
    cin >> surname;
    strcpy(full, surname);                  // копируем фамилию
    strcat(full, " ");                      // добавляем пробел
    strcat(full, name);                     // добавляем имя
    cout << full << ", длина " << strlen(full) << endl;
    if (strcmp(name, surname) < 0)
        cout << name << " идёт раньше по алфавиту" << endl;
    else if (strcmp(name, surname) > 0)
        cout << surname << " идёт раньше по алфавиту" << endl;
    else
        cout << "Имя и фамилия совпадают" << endl;
    char *p = strchr(full, 'a');
    if (p != NULL)
        cout << "Первая буква a на позиции " << p - full + 1 << endl;
    getch();
    return 0;
}
```

Пример работы программы

```
Введите имя: -> Aidar
Введите фамилию: -> Nurlan
Nurlan Aidar, длина 12
Aidar идёт раньше по алфавиту
Первая буква a на позиции 5
```

Задача 6.5. Решение через класс string

Та же обработка для `string`: ввод строки целиком, количество пробелов, разворот строки циклом, поиск слова и удаление найденного.

Решение — Borland C++ 6.0

```
#include <iostream>
#include <string>
using namespace std;

int main()
{
    string s, r;
    int i, spaces = 0;
    cout << "Введите строку: -> ";
    getline(cin, s);
    for (i = 0; i < s.length(); i++)
        if (s[i] == ' ') spaces++;
    cout << "Длина " << s.length() << ", пробелов " << spaces << endl;
    for (i = s.length() - 1; i >= 0; i--)
        r += s[i];                          // собираем перевёрнутую строку
    cout << "Наоборот: " << r << endl;
    int p = s.find("not");
    if (p != string::npos)
    {
        s.erase(p, 4);                      // удаляем «not » (3 буквы и пробел)
        cout << "Без not: " << s << endl;
    }
    return 0;
}
```

Пример работы программы

```
Введите строку: -> to be or not to be
Длина 18, пробелов 5
Наоборот: eb ot ton ro eb ot
Без not: to be or to be
```

Задача 6.6. Шифр Цезаря

Каждую латинскую букву сдвигаем по алфавиту на `k` позиций вперёд с «заворачиванием» (после `z` снова `a`). Остальные символы не меняются. Формула для строчных: `(c - 'a' + k) % 26 + 'a'`.

Решение — Borland C++ 6.0

```
#include <iostream.h>
#include <conio.h>

int main()
{
    char s[100];
    int k, i;
    cout << "Введите текст: -> ";
    cin.getline(s, 100);
    cout << "Введите сдвиг: -> ";
    cin >> k;
    for (i = 0; s[i] != '\0'; i++)
    {
        if (s[i] >= 'a' && s[i] <= 'z')
            s[i] = (s[i] - 'a' + k) % 26 + 'a';
        else if (s[i] >= 'A' && s[i] <= 'Z')
            s[i] = (s[i] - 'A' + k) % 26 + 'A';
    }
    cout << "Результат: " << s << endl;
    getch();
    return 0;
}
```

Пример работы программы

```
Введите текст: -> Hello, World
Введите сдвиг: -> 3
Результат: Khoor, Zruog
```

6.9 Тест по теме

Тест по теме — лекция 6

1. Как объявляется символьный тип данных?

а) `string`
б) `symbol`
в) `byte`
г) `char`

2. Какой диапазон значений у типа `char`?

а) от -128 до 127
б) от 0 до 65535
в) от -32768 до 32767
г) от 0 до 1000

3. Что значит управляющая последовательность `\t`?

а) Горизонтальная табуляция
б) Печать символа `t` один раз
в) Переход на новую строку
г) Переход на следующую страницу

4. Каким символом завершается строка в массиве `char`?

а) `'\n'`
б) пробелом
в) `'.'`
г) `'\0'`

5. Что выведет фрагмент?

```
cout << 'A' + 1;
```

а) B
б) 65
в) 66
г) A1

6. Чем `"A"` отличается от `'A'`?

а) Ничем, это записи одного и того же
б) `"A"` — строка из двух байт (`A` и `\0`), `'A'` — один символ
в) `'A'` занимает 2 байта
г) `"A"` — число, `'A'` — строка

7. Чему равна длина массива `char s[] = "Hello";` (по `sizeof`)?

а) 4
б) 7
в) 5
г) 6

8. Какая функция определяет длину строки?

а) `strcat`
б) `strcpy`
в) `strlen`
г) `strcmp`

9. В каком заголовочном файле описана `strlen`?

а) `<conio.h>`
б) `<iostream.h>`
в) `<stdlib.h>`
г) `<string.h>`

10. Что делает `strcat(stroka1, stroka2)`?

а) Ищет в первой строке любой символ второй
б) Копирует первую строку во вторую
в) Присоединяет `stroka2` к концу `stroka1` (результат в `stroka1`)
г) Сравнивает две строки

11. Что делает `strcpy(str1, str2)`?

а) Копирует строку из `str2` в `str1`
б) Склеивает строки
в) Сравнивает строки
г) Ищет `str2` внутри `str1`

12. Что выведет программа?

```
printf("%d\n", strcmp("Salem", "Salem"));
```

а) 7
б) 1
в) 2
г) 0

13. Что делает `strstr(str1, str2)`?

а) Склеивает две строки
б) Сравнивает две строки
в) Копирует строку
г) Ищет вторую строку внутри первой

14. Что делает `strrev(str)` (Borland)?

а) Разворачивает символы строки в обратном порядке
б) Удаляет строку
в) Сравнивает строки без учёта регистра
г) Копирует строку

15. Что делает `strncpy(S1, CS2, k)`?

а) Удаляет `k` символов из обеих строк
б) Копирует последние `k` символов `CS2` в `S1`
в) Копирует первые `k` символов строки `CS2` в `S1`
г) Сравнивает `k` символов

16. Что делает `strupr(S)`?

а) Переводит символы в римские цифры
б) Переводит символы в нижний регистр
в) Удаляет пробелы
г) Переводит символы строки в верхний регистр

17. Что делает `stricmp(CS1, CS2)`?

а) Сравнивает строки без учёта регистра букв
б) Сравнивает строки с учётом регистра
в) Склеивает строки
г) Ищет символ в строке

18. Что делает `strrchr(CS, ch)`?

а) Сравнивает две строки
б) Ищет символ `ch` в строке с конца к началу
в) Склеивает две строки
г) Ищет символ `ch` слева направо

19. Что выведет программа?

```
char s1[50] = "jack";
char s2[50] = "jill";
cout << strlen(s1) + strlen(s2) << endl;
```

а) 10
б) 8
в) 9
г) 7

20. Что будет в `s` при `cin >> s;` и вводе `Hello world`?

а) `Hello world`
б) `world`
в) пустая строка
г) `Hello`

21. Какой способ читает целую строку с пробелами в массив `char s[50]`?

а) `cin.getline(s, 50);`
б) `cin << s;`
в) `cin >> s;`
г) `getch(s);`

22. Как правильно сравнить две строки-массива на равенство?

а) `strcpy(a, b) == 0`
б) `a == b`
в) `strcmp(a, b) == 0`
г) `a = b`

23. Как склеить две переменные типа `string`?

а) `strcat(a, b);` для типа `string` — единственный способ
б) `s = a & b;`
в) `s = a.concat(b);`
г) `s = a + b;`

Ключи (проверяй себя только после того, как ответил сам): 1–г, 2–а, 3–а, 4–г, 5–в, 6–б, 7–г, 8–в, 9–г, 10–в, 11–а, 12–г, 13–г, 14–а, 15–в, 16–г, 17–а, 18–б, 19–б, 20–г, 21–а, 22–в, 23–г

6.10 Контрольные вопросы из программы (проверь себя устно)

1. Что такое тип `char`? Как символ хранится в памяти? (6.1)
2. Чем отличаются символ и строка? Зачем escape-последовательности? (6.1, 6.2)
3. Что такое строка в C++? Что такое класс `string`? (6.2, 6.6)
4. Как объявить и инициализировать строку? Как ввести и вывести? (6.2, 6.3)
5. Как соединить, сравнить строки, узнать длину? (6.5, 6.6)
6. Какие типичные операции выполняют над строками? (6.5, 6.8)

Лекция 7. Основные понятия и принципы ООП

До этого мы писали **процедурные** программы: данные — отдельно, функции — отдельно. Объектно-ориентированное программирование (ООП) объединяет данные и действия над ними в одном месте — **объекте**. Дальше в курсе (темы 8–15) вся работа в C++ Builder построена на этом. Здесь важно понять идеи и уметь прочитать и написать простой класс.

7.1 Что такое ООП

> **Запомни. Объектно-ориентированное программирование** — парадигма, в которой программа строится как набор **взаимодействующих объектов**; каждый объект — экземпляр некоторого **класса** и содержит свои данные (атрибуты) и действия (методы).

	Процедурный подход	Объектно-ориентированный подход
Основная единица	функция (процедура)	объект / класс
Данные и действия	данные отдельно, функции отдельно	данные и методы вместе, в классе
Доступ к данным	функции видят общие данные	доступ через методы, данные можно скрыть
Как растёт программа	больше функций, сложнее связи	новые классы, наследование
Пример	`double area(double r)`	`Circle c; c.area();`

7.2 Базовые понятия

- **Объект** — конкретная «вещь» в программе со своим **состоянием** и **поведением**. Пример: конкретная собака по кличке Бадди.
- **Класс** — описание (чертёж, шаблон) для создания объектов одного вида: какие у них данные и что они умеют. Пример: «Собака» вообще. Объект — **экземпляр** класса: из одного класса можно создать сколько угодно объектов.
- **Атрибут (поле, свойство)** — данные, хранящиеся в объекте: `name`, `breed`, `age`. Вместе значения атрибутов составляют **состояние** объекта.
- **Метод** — функция, принадлежащая классу: `bark()`, `walk()`. Методы описывают **поведение** объекта.

Аналогия: класс — чертёж дома, объект — построенный по нему дом. Чертёж один, домов много, у каждого свой цвет стен (своё состояние), но умеют они одно и то же.

Код

```
#include <iostream>
#include <string>
using namespace std;

class Dog                           // класс: описание «собаки вообще»
{
public:                             // всё ниже доступно снаружи
    string name;                    // атрибуты
    string breed;
    int age;
    void bark()                     // метод
    {
        cout << name << " (" << breed << ") говорит: Гав!" << endl;
    }
};                                  // после закрывающей скобки класса — точка с запятой!

int main()
{
    Dog d1;                         // создаём объекты — экземпляры класса
    d1.name = "Buddy";  d1.breed = "Golden Retriever";  d1.age = 3;
    Dog d2;
    d2.name = "Rex";    d2.breed = "Shepherd";          d2.age = 5;
    d1.bark();                      // обращение к методу — через точку
    d2.bark();
    cout << d1.name << " старше? " << (d1.age > d2.age ? "да" : "нет") << endl;
    return 0;
}
```

Результат выполнения

```
Buddy (Golden Retriever) говорит: Гав!
Rex (Shepherd) говорит: Гав!
Buddy старше? нет
```

Здесь `d1` и `d2` — два **разных объекта** одного класса: одинаковое поведение (`bark`), но разное состояние (`name`, `breed`, `age`). Изменение `d1.age` никак не затрагивает `d2`.

Краткий синтаксис класса

Код

```
class Имя
{
public:             // спецификатор доступа (открытая часть)
    поля;           // атрибуты
    методы;         // методы (тела можно писать внутри или снаружи: тип Имя::метод() {...})
private:            // закрытая часть: доступна только методам этого класса
    поля;
};                  // обязательная точка с запятой
```

- Создание объекта: `Dog d;` (как переменной).
- Доступ к членам через объект — `d.name`, через указатель — `p->name`.
- Если не указать доступ, в `class` по умолчанию всё **private**, а в `struct` — **public** (единственное различие).

Подробно: спецификаторы доступа, `this`, конструкторы и деструкторы — в темах 8, 13, 14.

7.3 Главные принципы ООП

Четыре принципа: **инкапсуляция, наследование, полиморфизм, абстракция**.

Инкапсуляция

Инкапсуляция — объединение данных и методов в классе и **сокрытие внутреннего устройства**: данные делают закрытыми (`private`), а доступ даётся только через открытые методы (`public`). Так объект сам следит, чтобы его состояние оставалось корректным.

Код

```
#include <iostream>
#include <string>
using namespace std;

class Dog
{
private:                                // снаружи напрямую не изменить
    string name;
    int age;
public:
    void setName(string n) { name = n; }
    string getName() { return name; }
    void setAge(int a)
    {
        if (a >= 0)                     // проверка: возраст не может быть отрицательным
            age = a;
        else
            cout << "Ошибка: возраст не может быть отрицательным" << endl;
    }
    int getAge() { return age; }
};

int main()
{
    Dog d;
    d.setName("Buddy");
    d.setAge(4);
    d.setAge(-2);                       // будет отклонено
    // d.age = -2;                      // ошибка компиляции: age закрыто (private)
    cout << d.getName() << ", " << d.getAge() << endl;
    return 0;
}
```

Результат выполнения

```
Ошибка: возраст не может быть отрицательным
Buddy, 4
```

Методы чтения называют **геттерами** (`get...`), изменения — **сеттерами** (`set...`). Плюсы: можно проверять значения, менять внутреннюю реализацию, не ломая остальной код, защитить данные от случайной порчи.

Абстракция

Абстракция — выделение **существенных** свойств объекта и **сокрытие деталей реализации**. Пользователь класса знает, **что** делает метод, но не обязан знать, **как**. Пример: нажимая на пульте «громче», вы не думаете о схеме усилителя.

Два проявления в C++: открытый интерфейс класса (один метод `performAction()` скрывает вызовы внутренних `walk()`, `fetch()`) и **абстрактные классы** — классы только с описанием «что делать», без реализации.

Код

```cpp
#include <iostream>
using namespace std;

class Shape                             // абстрактный класс: не умеет считать площадь сам
{
public:
    virtual double area() = 0;          // чисто виртуальный метод: «= 0», реализации нет
};

class Rect : public Shape
{
    double a, b;
public:
    Rect(double x, double y) { a = x; b = y; }
    double area() { return a * b; }
};

class Circle : public Shape
{
    double r;
public:
    Circle(double x) { r = x; }
    double area() { return 3.14159 * r * r; }
};

int main()
{
    // Shape s;                         // ошибка: объект абстрактного класса создать нельзя
    Rect rc(3, 4);
    Circle ci(2);
    Shape *p[2] = {&rc, &ci};
    for (int i = 0; i < 2; i++)
        cout << p[i]->area() << endl;
    return 0;
}
```

Результат выполнения

```
12
12.5664
```

> **Запомни. Отличие инкапсуляции от абстракции.** Инкапсуляция — про **защиту и упаковку**: данные спрятаны внутри класса, доступ — только через методы. Абстракция — про **упрощение**: оставляем только существенное, детали скрыты за простым интерфейсом.

Наследование

Наследование — создание нового класса (**производного**, потомка) на базе существующего (**базового**, родителя). Потомок получает все поля и методы родителя и может добавлять свои или переопределять. Отношение «**является**»: пудель **является** собакой. Смысл: повторное использование кода и построение **иерархии** классов.

Код

```
#include <iostream>
#include <string>
using namespace std;

class Dog
{
public:
    string name;
    void bark() { cout << name << ": Гав!" << endl; }
};

class Poodle : public Dog                 // Poodle наследует Dog
{
public:
    string coatType;                      // свои дополнительные атрибуты
    void doesGrooming() { cout << name << " (" << coatType << ") после стрижки" << endl; }
};

int main()
{
    Poodle p;
    p.name = "Fifi";                      // унаследованный атрибут
    p.coatType = "curly";                 // собственный атрибут
    p.bark();                             // унаследованный метод
    p.doesGrooming();                     // собственный метод
    return 0;
}
```

Результат выполнения

```
Fifi: Гав!
Fifi (curly) после стрижки
```

Полиморфизм

Полиморфизм («много форм») — один и тот же вызов ведёт себя по-разному в зависимости от того, **какому объекту** он адресован. Реализуется через **виртуальные функции**: в базовом классе метод помечают `virtual`, в потомках **переопределяют**; вызов через указатель/ссылку на базовый класс выбирает метод реального объекта.

Код

```
#include <iostream>
using namespace std;

class Animal
{
public:
    virtual void makeSound() { cout << "..." << endl; }
};

class Dog : public Animal
{
public:
    void makeSound() { cout << "Гав!" << endl; }
};

class Cat : public Animal
{
public:
    void makeSound() { cout << "Мяу!" << endl; }
};

int main()
{
    Dog d;
    Cat c;
    Animal *zoo[2] = {&d, &c};            // массив указателей на базовый класс
    for (int i = 0; i < 2; i++)
        zoo[i]->makeSound();              // для d — Гав!, для c — Мяу!
    return 0;
}
```

Результат выполнения

```
Гав!
Мяу!
```

Нюанс. Если в базовом классе **забыть** `virtual`, то вызов `zoo[i]->makeSound()` по указателю на `Animal` выполнит версию базового класса (`...`) для всех объектов. Полиморфизм работает только через `virtual` и только через указатель или ссылку на базовый класс.

7.4 Состояние и поведение. Взаимодействие объектов

- **Состояние** объекта — значения его атрибутов в данный момент (у банковского счёта — баланс). **Поведение** — набор методов, то есть что он умеет.
- Методы могут менять состояние: `deposit()` увеличивает баланс.
- Программа — это **взаимодействие** объектов: один объект вызывает метод другого, передавая данные. Это называют «передачей сообщений».
- Один объект может **содержать** другой (отношение «имеет»): у владельца есть собака.

Код

```
#include <iostream>
using namespace std;

class Account
{
private:
    double balance;                         // состояние скрыто
public:
    Account(double start) { balance = start; }   // конструктор: начальное состояние
    double getBalance() { return balance; }
    void deposit(double sum)
    {
        if (sum > 0) balance += sum;
    }
    bool withdraw(double sum)
    {
        if (sum > 0 && sum <= balance)
        {
            balance -= sum;
            return true;
        }
        return false;                       // денег не хватает
    }
    bool transfer(Account &to, double sum) // взаимодействие двух объектов
    {
        if (withdraw(sum))
        {
            to.deposit(sum);
            return true;
        }
        return false;
    }
};

int main()
{
    Account a(1000), b(200);
    a.transfer(b, 300);
    cout << a.getBalance() << " " << b.getBalance() << endl;       // 700 500
    cout << a.transfer(b, 5000) << endl;                           // 0: не получилось
    return 0;
}
```

Результат выполнения

```
700 500
0
```

7.5 Моделирование реальных сущностей

Чтобы по описанию задачи построить классы, выпиши слова из условия:

- **существительные** → кандидаты в **классы** (Книга, Читатель);
- **характеристики** («название», «автор») → **атрибуты**;
- **глаголы** («выдать», «вернуть») → **методы**;
- «является» → **наследование** (Студент — это Человек); «имеет» → **вложение** (у Библиотеки есть Книги).

7.6 Преимущества и применение

Преимущества: повторное использование кода (наследование); модульность (каждый класс — отдельный «кирпич»); защита данных (инкапсуляция); простота сопровождения и расширения; программа близка к описанию реального мира. **Недостатки:** для крошечных задач избыточно; чуть больше «обвязки» и ресурсов.

Где применяется: графические интерфейсы (в C++ Builder каждая кнопка, поле ввода и форма — объект; `Button1->Caption = "OK";` — обращение к атрибуту объекта), игры, банковские и учётные

системы, моделирование, операционные системы, библиотеки. Библиотека визуальных компонентов VCL построена на иерархии классов от `TObject` (тема 12).

7.7 Задачи

Задача 7.1. Класс Dog (из задания для самостоятельной работы)

Создать класс `Dog` с атрибутами `name` и `breed` и методом `bark()`. Создать несколько объектов и убедиться, что это разные экземпляры одного класса. Конструктор — специальный метод, который задаёт поля при создании объекта.

Решение — Borland C++ 6.0

```
#include <iostream>
#include <string>
using namespace std;

class Dog
{
public:
    string name;
    string breed;
    Dog(string n, string b)               // конструктор
    {
        name = n;
        breed = b;
    }
    void bark()
    {
        cout << name << " (" << breed << "): Гав!" << endl;
    }
};

int main()
{
    Dog myDog("Buddy", "Golden Retriever");
    Dog other("Rex", "Shepherd");
    myDog.bark();
    other.bark();
    other.name = "Max";                   // меняем один объект, второй не затронут
    other.bark();
    myDog.bark();
    return 0;
}
```

Результат выполнения

```
Buddy (Golden Retriever): Гав!
Rex (Shepherd): Гав!
Max (Shepherd): Гав!
Buddy (Golden Retriever): Гав!
```

Задача 7.2. Инкапсуляция: приватное имя и get/set

В `Dog` сделать атрибут `name` приватным, добавить `get_name()` и `set_name()` (запрет пустого имени), а также метод `perform_action()`, который вызывает внутренние приватные `walk()` и `fetch()`.

Решение — Borland C++ 6.0

```
#include <iostream>
#include <string>
using namespace std;

class Dog
{
private:
    string name;
    void walk()  { cout << name << " гуляет" << endl; }
    void fetch() { cout << name << " приносит палку" << endl; }
public:
    Dog(string n) { name = n; }
    string get_name() { return name; }
    void set_name(string n)
    {
        if (n != "")                        // защита от пустого имени
            name = n;
    }
    void perform_action()                   // пользователь не знает, что внутри
    {
        walk();
        fetch();
    }
};

int main()
{
    Dog d("Buddy");
    d.set_name("");                         // игнорируется
    d.set_name("Max");
    cout << d.get_name() << endl;
    d.perform_action();
    return 0;
}
```

Результат выполнения

```
Max
Max гуляет
Max приносит палку
```

Задача 7.3. Наследование: Poodle от Dog

Класс `Poodle` наследует `Dog`, получает свой атрибут `coat_type` и метод `does_grooming()`, а также все свойства `Dog`.

Решение — Borland C++ 6.0

```
#include <iostream>
#include <string>
using namespace std;

class Dog
{
public:
    string name;
    string breed;
    void bark() { cout << name << ": Гав!" << endl; }
};

class Poodle : public Dog
{
public:
    string coat_type;
    void does_grooming()
    {
        cout << name << ": стрижка для шерсти типа " << coat_type << endl;
    }
};

int main()
{
    Poodle p;
    p.name = "Fifi";  p.breed = "Poodle";  p.coat_type = "curly";
    p.bark();
    p.does_grooming();
    return 0;
}
```

Результат выполнения

```
Fifi: Гав!
Fifi: стрижка для шерсти типа curly
```

Задача 7.4. Полиморфизм: make_sound у Dog и Cat

Базовый класс `Animal` с виртуальным `make_sound()`; `Dog` и `Cat` переопределяют его. Вызвать для каждого объекта через указатель на `Animal`.

Решение — Borland C++ 6.0

```cpp
#include <iostream>
using namespace std;

class Animal
{
public:
    virtual void make_sound() { cout << "Какой-то звук" << endl; }
};

class Dog : public Animal
{
public:
    void make_sound() { cout << "Dog: Гав!" << endl; }
};

class Cat : public Animal
{
public:
    void make_sound() { cout << "Cat: Мяу!" << endl; }
};

int main()
{
    Animal a;
    Dog d;
    Cat c;
    Animal *list[3] = {&a, &d, &c};
    for (int i = 0; i < 3; i++)
        list[i]->make_sound();
    return 0;
}
```

Результат выполнения

```
Какой-то звук
Dog: Гав!
Cat: Мяу!
```

Задача 7.5. Абстракция: фигуры и их площади

Абстрактный класс `Shape` с чисто виртуальным `area()`; `Square` и `Triangle` реализуют его. Вычислить суммарную площадь.

Решение — Borland C++ 6.0

```
#include <iostream>
using namespace std;

class Shape
{
public:
    virtual double area() = 0;
};

class Square : public Shape
{
    double a;
public:
    Square(double x) { a = x; }
    double area() { return a * a; }
};

class Triangle : public Shape
{
    double base, h;
public:
    Triangle(double b, double height) { base = b; h = height; }
    double area() { return base * h / 2; }
};

int main()
{
    Square s(4);
    Triangle t(6, 5);
    Shape *all[2] = {&s, &t};
    double total = 0;
    for (int i = 0; i < 2; i++)
        total += all[i]->area();
    cout << "Суммарная площадь = " << total << endl;
    return 0;
}
```

Результат выполнения

```
Суммарная площадь = 31
```

Задача 7.6. Банковский счёт и перевод между объектами

Класс `Account`: баланс скрыт, `deposit`, `withdraw` (с проверкой), `transfer(Account &to, double sum)`. Показать корректный и некорректный перевод.

Решение — Borland C++ 6.0

```
#include <iostream>
using namespace std;

class Account
{
private:
    double balance;
public:
    Account(double start) { balance = start; }
    double getBalance() { return balance; }
    void deposit(double s) { if (s > 0) balance += s; }
    bool withdraw(double s)
    {
        if (s > 0 && s <= balance) { balance -= s; return true; }
        return false;
    }
    bool transfer(Account &to, double s)
    {
        if (!withdraw(s)) return false;
        to.deposit(s);
        return true;
    }
};

int main()
{
    Account a(500), b(0);
    if (a.transfer(b, 200))
        cout << "Перевод выполнен: " << a.getBalance() << " и " << b.getBalance() << endl;
    if (!a.transfer(b, 1000))
        cout << "Недостаточно средств, баланс a = " << a.getBalance() << endl;
    return 0;
}
```

Результат выполнения

```
Перевод выполнен: 300 и 200
Недостаточно средств, баланс a = 300
```

Задача 7.7. Выделить классы (без кода)

Условие: «В школе работают учителя и учатся ученики. Каждый учитель ведёт предмет и ставит оценки. У ученика есть имя, класс и список оценок; он может получить оценку и вычислить средний балл. Школа хранит список учеников». Определи классы, атрибуты, методы и связи.

Класс	Атрибуты	Методы	Связи
`Person`	имя, возраст	`getName()`	базовый для учителя и ученика (наследование)
`Teacher`	предмет	`setMark(Student&, mark)`	является `Person`
`Student`	класс, оценки	`addMark(m)`, `average()`	является `Person`; оценки ему ставит `Teacher`
`School`	список учеников	`addStudent()`, `findBest()`	содержит (имеет) объекты `Student`

Принципы здесь: **инкапсуляция** — оценки закрыты, меняются только через `addMark`; **наследование** — `Teacher` и `Student` от `Person`; **абстракция** — `School` оперирует учениками, не зная устройство списка оценок; **полиморфизм** — например, виртуальный `info()` у `Person`, который по-разному печатает учителя и ученика.

7.8 Тест по теме

Тест по теме — лекция 7

1. Что такое объектно-ориентированное программирование?

а) Подход, в котором программа состоит только из одной функции `main`
б) Способ записи алгоритмов блок-схемами
в) Язык программирования, созданный для графических программ
г) Подход, в котором программа строится из взаимодействующих объектов, объединяющих данные и методы

2. Что такое класс?

а) Только один метод
б) Описание (шаблон), по которому создаются объекты
в) Конкретный объект со своими значениями
г) Другое название переменной

3. Что такое объект?

а) Описание нового типа данных
б) Оператор цикла
в) Файл с заголовками
г) Экземпляр класса со своим состоянием и поведением

4. Что такое атрибут (поле) класса?

а) Функция класса
б) Комментарий в коде
в) Данные, хранящиеся в объекте
г) Оператор присваивания

5. Что такое метод?

а) Тип данных
б) Функция, принадлежащая классу
в) Переменная класса
г) Директива препроцессора

6. Что является состоянием объекта?

а) Значения его атрибутов в данный момент
б) Список его методов
в) Имя класса
г) Размер программы

7. Какой принцип ООП предполагает скрытие данных и доступ к ним через методы?

а) полиморфизм
б) рекурсия
в) инкапсуляция
г) наследование

8. Какой принцип ООП позволяет создать новый класс на основе существующего и получить его свойства и методы?

а) перегрузка
б) абстракция
в) наследование
г) инкапсуляция

9. Какой принцип ООП означает, что один вызов метода выполняется по-разному для объектов разных классов?

а) наследование
б) инкапсуляция
в) абстракция
г) полиморфизм

10. Что означает абстракция?

а) Выделение существенных свойств и сокрытие деталей реализации
б) Копирование кода из одного класса в другой
в) Хранение данных в файле
г) Объявление всех полей открытыми

11. Какая запись описывает наследование `Poodle` от `Dog` в C++?

а) `class Poodle(Dog) { ... };`
б) `class Poodle : public Dog { ... };`
в) `class Poodle inherits Dog { ... };`
г) `class Poodle -> Dog { ... };`

12. Как называют отношение «Пудель является собакой»?

а) вложение («имеет»)
б) наследование («является»)
в) рекурсия
г) перегрузка

13. Что нужно, чтобы вызов метода через указатель на базовый класс выполнил версию потомка?

а) Использовать `goto`
б) Объявить метод в базовом классе как `virtual`
в) Ничего, всегда выполняется версия потомка
г) Объявить метод как `static`

14. Что выведет программа?

```
#include <iostream>
using namespace std;
class A { public: void f() { cout << "A"; } };
class B : public A { public: void f() { cout << "B"; } };
int main()
{
    B b;
    A *p = &b;
    p->f();
    return 0;
}
```

а) ошибка компиляции
б) A
в) B
г) AB

15. Что произойдёт при попытке обратиться к `d.age`, если `age` объявлено в секции `private`?

а) Значение будет прочитано, но не изменено
б) Программа выведет 0
в) Ошибка компиляции: поле закрыто
г) Значение будет прочитано

16. Какой оператор используется для доступа к члену класса через **объект**, а какой — через **указатель**?

а) `&` для объекта и `*` для указателя
б) `->` для объекта и `.` для указателя
в) `.` для объекта и `->` для указателя
г) `::` для обоих

17. Какой доступ по умолчанию у членов `class`, если спецификатор не указан?

а) public
б) зависит от компилятора
в) private
г) protected

18. Что нельзя сделать с абстрактным классом (есть чисто виртуальный метод `= 0`)?

а) Создать объект этого класса
б) Наследоваться от него
в) Объявить указатель на него
г) Объявить в нём поля

19. Что общего у объектов `d1` и `d2` типа `Dog`?

а) Ничего общего
б) Одно и то же состояние
в) Один класс, одинаковый набор методов; значения атрибутов у каждого свои
г) Общие значения всех атрибутов

20. Как при проектировании найти кандидатов в методы?

а) Это глаголы из описания задачи
б) Это числа из описания
в) Это существительные из описания
г) Это прилагательные из описания

21. Какое из преимуществ относится к ООП?

а) Нельзя допустить ошибок
б) Не нужны функции
в) Программа всегда работает быстрее процедурной
г) Повторное использование кода и удобство сопровождения

Ключи (проверяй себя только после того, как ответил сам): 1–г, 2–б, 3–г, 4–в, 5–б, 6–а, 7–в, 8–в, 9–г, 10–а, 11–б, 12–б, 13–б, 14–б, 15–в, 16–в, 17–в, 18–а, 19–в, 20–а, 21–г

7.9 Контрольные вопросы из программы (проверь себя устно)

1. Что такое объектно-ориентированное программирование? (7.1)
2. Каковы основные понятия ООП? (7.2)
3. Что такое класс и объект? (7.2)
4. Что такое инкапсуляция, наследование, полиморфизм, абстракция? (7.3)
5. Как различаются инкапсуляция и абстракция? (7.3)
6. В чём преимущества ООП? (7.6)
7. Чем отличаются процедурный и объектно-ориентированный подходы? (7.1)
8. Как реальные сущности представляются объектами и классами? (7.5)

Приложения. Шпаргалка, типичные ошибки и итоговый тест по РК1

П.1 Шаблон консольной программы для Borland C++ 6.0

Начинай любую лабораторную с этого каркаса. `getch()` в конце не даёт окну закрыться, пока не нажмёшь клавишу.

Код

```
#include <iostream.h>      // cin, cout
#include <conio.h>         // getch()
#include <stdlib.h>        // randomize(), random()
#include <math.h>          // sqrt, pow, fabs, sin ...
#include <string.h>        // strlen, strcpy ...
#include <iomanip.h>       // setw, setprecision

int main()
{
    // 1. объявить переменные
    // 2. ввод данных (cout << подсказка; cin >> переменная;)
    // 3. вычисления
    // 4. вывод результата
    getch();               // задержка окна
    return 0;
}
```

Запомни. Если программа работает со строками `string`, подключай `#include <string>` и добавляй `using namespace std;` (при `<iostream.h>` в старом Borland иногда приходится писать `<iostream>` вместе с `using namespace std;`). Для русского текста в консоли — `setlocale(LC_ALL, "Russian");` (нужен `<locale.h>`).

П.2 Типичные ошибки новичка

Ошибка	Пример	Как правильно
`=` вместо `==`	`if (a = 5)`	`if (a == 5)`
`;` после `if`/`for`/`while`	`for (i=0;i<5;i++);`	убрать `;` — иначе тело пустое
Целочисленное деление	`1/2` даёт 0	`1.0/2` или `(double)1/2`
Выход за границы массива	`int a[5]; a[5]=1;`	допустимые индексы 0…4
Размер массива — переменная	`int n=5; int a[n];`	`const int N=5; int a[N];`
Нет `break` в `switch`	выполняются все следующие `case`	ставить `break;`
Не инициализирована сумма	`int s; s += x;`	`int s = 0;`
Бесконечный цикл	в `while` не меняется счётчик	менять переменную условия
Сравнение строк `==`	`if (s1 == s2)` для `char[]`	`strcmp(s1, s2) == 0`
`cin >> s` с пробелами	читает до пробела	`cin.getline(s, 80)`
Нет `;` после класса	`class A {...}`	`class A {...};`
Забыли `return` в функции	`int f(){ x=5; }`	`return x;`
`a[i,j]` вместо `a[i][j]`	`m[1,2]`	`m[1][2]`
`&` забыт при `scanf`	`scanf("%d", n)`	`scanf("%d", &n)`
`random(n)` даёт `0…n-1`	`random(10)` не даёт 10	диапазон `[a;b]` → `a + random(b-a+1)`

П.3 Шпаргалка

Формулы и шаблоны

Код

```
// диапазон [a; b] целых случайных
randomize();
x = a + random(b - a + 1);

// сумма / произведение / счётчик
s = 0; for (i = 0; i < n; i++) s += a[i];
p = 1; for (i = 0; i < n; i++) p *= a[i];
k = 0; for (i = 0; i < n; i++) if (a[i] > 0) k++;

// максимум и его индекс
mx = a[0]; im = 0;
for (i = 1; i < n; i++) if (a[i] > mx) { mx = a[i]; im = i; }

// обмен двух значений
t = a[i]; a[i] = a[j]; a[j] = t;

// чётное / кратное
if (x % 2 == 0) ...      if (x % 3 == 0) ...

// цифры числа
while (x > 0) { c = x % 10; x /= 10; }

// длина строки, сравнение, копия
n = strlen(s);   strcmp(s1, s2) == 0;   strcpy(s1, s2);
```

Двумерный массив `a[N][M]`

Код

```
for (i = 0; i < N; i++)            // обход по строкам
    for (j = 0; j < M; j++)
        ... a[i][j] ...

главная диагональ     : i == j          (a[i][i])
побочная диагональ    : i + j == N - 1  (a[i][N-1-i])
выше главной          : j > i
ниже главной          : j < i
строка k целиком      : a[k][j], j = 0…M-1
столбец k целиком     : a[i][k], i = 0…N-1
```

Сортировка пузырьком по возрастанию

Код

```
for (i = 0; i < n - 1; i++)
    for (j = 0; j < n - 1 - i; j++)
        if (a[j] > a[j + 1])
        {
            t = a[j]; a[j] = a[j + 1]; a[j + 1] = t;
        }
```

Схема цикла: что выбрать

Ситуация	Цикл
Число повторений известно	`for`
Повторяем, пока условие истинно, может не выполниться ни разу	`while`
Тело должно выполниться хотя бы раз (ввод с проверкой)	`do … while`

Функции

Код

```
int f(int a, int b) { return a + b; }       // возвращает значение
void show(int a[], int n) { ... }            // ничего не возвращает; массив — по адресу
void swap(int &a, int &b) { int t=a; a=b; b=t; }   // & — изменяем исходные переменные
```

Прототип (объявление) пишется до `main`, определение — можно после: `int f(int a, int b);`.

П.4 Примеры типа «изменить побочную диагональ / отсортировать»

Код

```
#include <iostream>
#include <iomanip>
using namespace std;

int main()
{
    const int N = 4;
    int a[N][N];
    int i, j, k = 1;
    for (i = 0; i < N; i++)
        for (j = 0; j < N; j++)
            a[i][j] = k++;

    // 1) побочная диагональ: обнулить
    for (i = 0; i < N; i++)
        a[i][N - 1 - i] = 0;

    for (i = 0; i < N; i++)
    {
        for (j = 0; j < N; j++)
            cout << setw(4) << a[i][j];
        cout << endl;
    }

    // 2) сумма элементов выше главной диагонали
    int s = 0;
    for (i = 0; i < N; i++)
        for (j = i + 1; j < N; j++)
            s += a[i][j];
    cout << "Сумма выше главной = " << s << endl;
    return 0;
}
```

Результат выполнения

```
   1   2   3   0
   5   6   0   8
   9   0  11  12
   0  14  15  16
Сумма выше главной = 25
```

Код

```
#include <iostream>
using namespace std;

int main()
{
    const int N = 5;
    int a[N], i, j, t;
    for (i = 0; i < N; i++)
    {
        cout << "-> ";
        cin >> a[i];
    }
    for (i = 0; i < N - 1; i++)                 // сортировка по убыванию
        for (j = 0; j < N - 1 - i; j++)
            if (a[j] < a[j + 1])
            {
                t = a[j]; a[j] = a[j + 1]; a[j + 1] = t;
            }
    for (i = 0; i < N; i++)
        cout << a[i] << " ";
    cout << endl;
    return 0;
}
```

Пример работы программы

```
-> 5
-> 9
-> 2
-> 7
-> 1
9 7 5 2 1
```

П.5 Итоговый тест по темам 1–7

Тест по теме — лекция П

1. Какой тип подходит для хранения значения `3.14`?

а) `char`
б) `double`
в) `bool`
г) `int`

2. Чему равно `17 % 5`?

а) 4
б) 2
в) 3
г) 0

3. Что выведет программа?

```
#include <iostream>
using namespace std;
int main()
{
    int a = 7 / 2;
    double b = 7 / 2;
    double c = 7 / 2.0;
    cout << a << " " << b << " " << c;
    return 0;
}
```

а) 3 3 3.5
б) 3.5 3.5 3.5
в) 4 3 3.5
г) 3 3.5 3.5

4. Что выведет программа?

```
#include <iostream>
using namespace std;
int main()
{
    int x = 5;
    int y = x++;
    int z = ++x;
    cout << x << " " << y << " " << z;
    return 0;
}
```

а) 6 5 6
б) 7 5 7
в) 7 6 7
г) 7 5 6

5. Какое значение получит `r` после `r = (3 > 2) && (2 > 5);`?

а) 5
б) 2
в) 1 (истина)
г) 0 (ложь)

6. Что выведет программа?

```
#include <iostream>
using namespace std;
int main()
{
    int a = 3;
    switch (a)
    {
        case 1: cout << "A";
        case 3: cout << "B";
        case 4: cout << "C"; break;
        default: cout << "D";
    }
    return 0;
}
```

а) BCD
б) BC
в) B
г) ABC

7. Что выведет программа?

```
#include <iostream>
using namespace std;
int main()
{
    int s = 0;
    for (int i = 1; i <= 5; i++)
        s += i * i;
    cout << s;
    return 0;
}
```

а) 55
б) 15
в) 225
г) 30

8. Что выведет программа?

```
#include <iostream>
using namespace std;
int main()
{
    int i = 10;
    while (i > 0)
    {
        i -= 3;
        if (i == 4) continue;
        cout << i << " ";
    }
    return 0;
}
```

а) 7 1
б) 7 4 1 -2
в) 7 1 -2
г) 10 7 4 1

9. Сколько раз выполнится тело цикла?

```
#include <iostream>
using namespace std;
int main()
{
    int k = 0, n = 5;
    do
    {
        k++;
        n += 2;
    } while (n < 5);
    cout << k;
    return 0;
}
```

а) 0
б) бесконечно
в) 1
г) 2

10. Какая запись правильно объявляет массив из 10 целых, заполненный нулями?

а) `int a[10] = {0};`
б) `int a[] = {10};`
в) `int[10] a = 0;`
г) `int a(10) = 0;`

11. Какое объявление приведёт к ошибке в Borland C++ 6.0?

а) `const int n = 5; int a[n];`
б) `int a[] = {1, 2, 3};`
в) `int a[5];`
г) `int n = 5; int a[n];`

12. Какой индекс имеет **последний** элемент массива `int a[8]`?

а) 0
б) 8
в) 9
г) 7

13. Что выведет программа?

```
#include <iostream>
using namespace std;
int main()
{
    int a[5] = {4, 8, 1, 9, 3};
    int mx = a[0];
    for (int i = 1; i < 5; i++)
        if (a[i] > mx) mx = a[i];
    cout << mx;
    return 0;
}
```

а) 4
б) 9
в) 3
г) 25

14. Что выведет программа?

```
#include <iostream>
using namespace std;
int main()
{
    int a[5] = {1, 2, 3};
    int s = 0;
    for (int i = 0; i < 5; i++)
        s += a[i];
    cout << s;
    return 0;
}
```

а) 15
б) неопределённое значение
в) 3
г) 6

15. Какое условие выбирает элементы **побочной диагонали** матрицы `a[N][N]`?

а) `i - j == N - 1`
б) `i + j == N - 1`
в) `i + j == N`
г) `i == j`

16. Что выведет программа?

```
#include <iostream>
using namespace std;
int main()
{
    int a[3][3] = {{1, 2, 3}, {4, 5, 6}, {7, 8, 9}};
    int s = 0;
    for (int i = 0; i < 3; i++)
        s += a[i][i] + a[i][2 - i];
    cout << s;
    return 0;
}
```

а) 15
б) 30
в) 45
г) 25

17. Что выведет программа?

```
#include <iostream>
using namespace std;
int main()
{
    int a[2][3] = {{1, 2, 3}, {4, 5, 6}};
    int k = 0;
    for (int i = 0; i < 2; i++)
        for (int j = 0; j < 3; j++)
            if (a[i][j] % 2 == 0) k += a[i][j];
    cout << k;
    return 0;
}
```

а) 9
б) 21
в) 6
г) 12

18. Что выведет программа?

```
#include <iostream>
using namespace std;
int f(int x)
{
    return x * 2 + 1;
}
int main()
{
    cout << f(f(2));
    return 0;
}
```

а) 5
б) 10
в) 11
г) 9

19. Что выведет программа?

```
#include <iostream>
using namespace std;
void change(int a, int &b)
{
    a = 10;
    b = 20;
}
int main()
{
    int x = 1, y = 2;
    change(x, y);
    cout << x << " " << y;
    return 0;
}
```

а) 10 20
б) 1 2
в) 1 20
г) 10 2

20. Чем функция типа `void` отличается от функции типа `int`?

а) `void` работает быстрее
б) `void` не может иметь параметры
в) `void` не возвращает значения, `int` возвращает целое через `return`
г) `int` не может содержать циклы

21. Что такое прототип функции?

а) Комментарий к функции
б) Тело функции
в) Вызов функции
г) Объявление функции (тип, имя, параметры) до её определения

22. Что выведет программа?

```
#include <iostream>
using namespace std;
int main()
{
    char s[] = "Hello";
    cout << sizeof(s) << " " << s[1];
    return 0;
}
```

а) 6 e
б) 6 l
в) 5 H
г) 5 e

23. Какая функция возвращает длину строки?

а) `sizeof_str`
б) `strcmp`
в) `strcpy`
г) `strlen`

24. Как правильно сравнить две строки `char s1[20], s2[20]` на равенство?

а) `s1 = s2`
б) `s1.equals(s2)`
в) `strcmp(s1, s2) == 0`
г) `s1 == s2`

25. Что выведет программа?

```
#include <iostream>
#include <string.h>
using namespace std;
int main()
{
    char s[20] = "abcabc";
    int k = 0;
    for (int i = 0; i < strlen(s); i++)
        if (s[i] == 'b') k++;
    cout << k;
    return 0;
}
```

а) 3
б) 2
в) 6
г) 1

26. Каков код символа `'A'` относительно `'a'` в ASCII?

а) `'A'` больше `'a'` на 32
б) `'A'` меньше `'a'` на 1
в) `'A'` меньше `'a'` на 32
г) они равны

27. Что выведет программа?

```
#include <iostream>
using namespace std;
int main()
{
    char c = 'a';
    c = c + 2;
    cout << c << " " << (int)'0';
    return 0;
}
```

а) 99 48
б) c 48
в) c 0
г) a 48

28. Какой принцип ООП означает скрытие данных внутри класса и доступ через методы?

а) перегрузка
б) полиморфизм
в) наследование
г) инкапсуляция

29. Что выведет программа?

```
#include <iostream>
using namespace std;
class A { public: virtual void f() { cout << "A"; } };
class B : public A { public: void f() { cout << "B"; } };
int main()
{
    B b;
    A *p = &b;
    p->f();
    return 0;
}
```

а) ошибка компиляции
б) AB
в) A
г) B

30. Что выведет программа?

```
#include <iostream>
using namespace std;
int main()
{
    int s = 0, n = 0;
    for (int i = 1; i <= 10; i++)
    {
        if (i % 3 == 0) continue;
        if (i > 8) break;
        s += i;
        n++;
    }
    cout << s << " " << n;
    return 0;
}
```

а) 27 7
б) 18 6
в) 27 6
г) 55 10

Ключи (проверяй себя только после того, как ответил сам): 1–б, 2–б, 3–а, 4–б, 5–г, 6–б, 7–а, 8–в, 9–в, 10–а, 11–г, 12–г, 13–б, 14–г, 15–б, 16–б, 17–г, 18–в, 19–в, 20–в, 21–г, 22–а, 23–г, 24–в, 25–б, 26–в, 27–б, 28–г, 29–г, 30–в

П.6 Задачи для тренировки (реши сам, затем проверь на компьютере)

Это задачи по образцу лабораторных. Формат экрана придумай сам: подсказка `"-> "`, затем результат.

Циклы и ряды

1. Найти сумму квадратов всех нечётных чисел от 1 до N (N вводит пользователь).
2. Вывести таблицу умножения на число K (от 1 до 10).
3. Вычислить факториал числа N (`1 * 2 * … * N`) циклом `while`.
4. Подсчитать количество цифр в целом числе и их сумму.
5. Вычислить сумму ряда `1 + 1/2 + 1/3 + … + 1/N` и вывести с 4 знаками после запятой.
6. Вводить числа, пока пользователь не введёт 0; вывести их количество и среднее значение (`do…while`).
7. Вывести числа Фибоначчи, не превышающие 100.

Одномерные массивы (размер 10, значения случайные в заданном диапазоне)

1. Поменять местами максимальный и минимальный элементы.

2. Все отрицательные элементы заменить на их модуль и подсчитать, сколько их было.
3. Найти среднее арифметическое положительных элементов.
4. Вывести элементы в обратном порядке, не создавая нового массива (обмен `a[i]` и `a[n-1-i]`).
5. Отсортировать массив по возрастанию и вывести индекс элемента, равного заданному числу.
6. Удалить из массива все элементы, равные нулю (сдвиг влево).
7. Определить, является ли массив упорядоченным по возрастанию.

Двумерные массивы (матрица 5×5)

1. Заменить элементы побочной диагонали их квадратами.
2. Найти максимальный элемент в каждой строке и вывести столбцом.
3. Поменять местами первую и последнюю строки.
4. Найти сумму элементов каждого столбца и номер столбца с наименьшей суммой.
5. Заполнить матрицу «змейкой» или таблицей умножения (`a[i][j] = (i+1)*(j+1)`).
6. Транспонировать квадратную матрицу.

Строки и функции

1. Подсчитать количество гласных в строке.
2. Проверить, является ли слово палиндромом.
3. Заменить в строке все пробелы на символ `_`.
4. Написать функцию `int mx(int a[], int n)`, возвращающую максимум массива, и вызвать её для двух разных массивов.
5. Написать функцию `void swap(int &a, int &b)` и использовать её при сортировке массива.
6. Написать функцию `bool isPrime(int n)` и вывести все простые числа до 50.

> **Запомни.** Хочешь проверить решение — пришли код, и я проверю его и объясню ошибки. Главное правило для РК: сначала разобрать задачу на шаги (ввод → цикл → условие → вывод), потом писать код.

П.7 План повторения перед РК1

1. Перечитай лекции в порядке 1 → 7, не пропуская блоки «нюансы» (красные).
2. Реши по 1–2 задачи из каждой лекции **без подглядывания**, затем сверься.
3. Пройди тест каждой лекции и итоговый тест, неправильные ответы разбери по лекции.
4. Устно ответь на «Контрольные вопросы» в конце каждой лекции.
5. Потренируйся писать от руки шаблоны: массив (ввод/вывод), поиск максимума, обмен, пузырёк, побочная диагональ, функция с возвратом и с `&`.